

```

        s := s + cl.definition[1]\class.described_by[i];
    END_REPEAT;

    IF (('ISO13584_IEC61360_ITEM_CLASS_CASE_OF_SCHEMA.'
        + 'A_PRIORI_SEMANTIC_RELATIONSHIP')
        IN TYPEOF (cl.definition[1]))
    THEN
        s := s + cl.definition[1]\a_priori_semantic_relationship.referenc
            ed_properties;
    END_IF;

    IF EXISTS(cl.definition[1]\class.its_superclass)
    THEN
        s := s + compute_known_applicable_properties(
            cl.definition[1]\class.its_superclass);
    END_IF;

    RETURN(s);
END_IF;
END_FUNCTION; -- compute_known_applicable_properties
(*)

```

5.12.15 Compute_known_applicable_data_types function

The **compute_known_applicable_data_types** function computes the set of **data_types** that are applicable for a given class. When a definition is not available, it returns only the applicable **data_types** that may be computed.

NOTE When some class **dictionary_definition** is not present in the same exchange context (a PLIB exchange context is never assumed to be complete), the super-class of a class may not be known. Therefore the **data_types** defined as applicable by this super-class cannot be computed by the **compute_known_applicable_data_types** function. Only on the receiving system all the **dictionary_definitions** of the **BSUs** are available. Therefore, on the receiving system, the **compute_known_applicable_data_types** function computes all the **data_types** that are applicable to a class by virtue of being referenced by a **defined_types** attribute or of being imported through an **a_priori_semantic_relationship**.

EXPRESS specification:

```

*)
FUNCTION compute_known_applicable_data_types(cl: class_BSU):
    SET OF data_type_BSU;
LOCAL
    s: SET OF data_type_BSU := [];
END_LOCAL;

IF SIZEOF(cl.definition) = 0
THEN
    RETURN(s);
ELSE
    REPEAT i := 1 TO SIZEOF(cl.definition[1]\class.defined_types);
        s := s + cl.definition[1]\class.defined_types[i];
    END_REPEAT;

    IF (('ISO13584_IEC61360_ITEM_CLASS_CASE_OF_SCHEMA.'
        + 'A_PRIORI_SEMANTIC_RELATIONSHIP')

```

```

        IN TYPEOF (cl.definition[1]))
    THEN
        s := s + cl.definition[1]\a_priori_semantic_relationship.referenc
            ed_data_types;
    END_IF;

    IF EXISTS(cl.definition[1]\class.its_superclass)
    THEN
        s := s + compute_known_applicable_data_types(
            cl.definition[1]\class.its_superclass);
    END_IF;

    RETURN(s);
END_IF;

END_FUNCTION; -- compute_known_applicable_data_types
(*)

```

5.12.16 List_to_set function

The **list_to_set** function creates a SET from a LIST named l, the type of element for the SET will be the same as that in the original LIST.

EXPRESS specification:

```

*)
FUNCTION list_to_set(l: LIST [0:?] OF GENERIC:type_elem):
    SET OF GENERIC: type_elem;

LOCAL
    s: SET OF GENERIC: type_elem := [];
END_LOCAL;

REPEAT i := 1 TO SIZEOF(l);
    s := s + l[i];
END_REPEAT;

RETURN(s);
END_FUNCTION; -- list_to_set
(*)

```

5.12.17 Check_properties_applicability function

The **check_properties_applicability** function checks that only those properties that are not applicable for a class by inheritance may become applicable to this class by virtue of being referenced by the **described_by** attribute.

EXPRESS specification:

```

*)
FUNCTION check_properties_applicability(cl: class): LOGICAL;
LOCAL
    inter: SET OF property_bsu := [];
END_LOCAL;

```

```

IF EXISTS(cl.its_superclass)
THEN
    IF (SIZEOF(cl.its_superclass.definition)=1)
    THEN
        inter := (list_to_set(cl.described_by) *
                    cl.its_superclass.definition[1]\class.
                    known_applicable_properties);
        RETURN(inter = []);
    ELSE
        RETURN(UNKNOWN);
    END_IF;
ELSE
    RETURN(TRUE);
END_IF;

END_FUNCTION; -- check_properties_applicability
(*)

```

5.12.18 Check_datatypes_applicability function

The **check_datatypes_applicability** function checks that only those datatypes that are not applicable for a class by inheritance may become applicable to this class by virtue of being referenced by the **defined_types** attribute.

EXPRESS specification:

```

*)
FUNCTION check_datatypes_applicability(cl: class): LOGICAL;
LOCAL
    inter: SET OF data_type_bsu := [];
END_LOCAL;

IF EXISTS(cl.its_superclass)
THEN
    IF (SIZEOF(cl.its_superclass.definition) = 1)
    THEN
        inter := cl.defined_types *
                    cl.its_superclass.definition[1]\class.
                    known_applicable_data_types;
        RETURN(inter = []);
    ELSE
        RETURN(UNKNOWN);
    END_IF;
ELSE
    RETURN(TRUE);
END_IF;

END_FUNCTION; -- check_datatypes_applicability
(*)

```

5.12.19 One_language_per_translation function

The **one_language_per_translation** function checks that the languages of translation in **administrative_data** are unique.

EXPRESS specification:

```

*)
FUNCTION one_language_per_translation (adm: administrative_data)
    : LOGICAL;

    LOCAL
        count: INTEGER;
        lang: language_code;
    END_LOCAL;

    REPEAT i := 1 TO SIZEOF (adm.translation);
        lang := adm.translation[i].language;
        count := 0;
        REPEAT j := 1 TO SIZEOF (adm.translation);
            IF lang = adm.translation[j].language
            THEN
                count := count+1;
            END_IF;
        END_REPEAT;
        IF count > 1
        THEN RETURN (FALSE);
        END_IF;
    END_REPEAT;
    RETURN (TRUE);

END_FUNCTION; -- one_language_per_translation
(*

```

5.12.20 Allowed_values_integer_types function

The **allowed_values_integer_types** function computes the set of **integer_type** values allowed by a **non_quantitative_int_type**. If the parameter is indeterminate, it returns the indeterminate value.

EXPRESS specification:

```

*)
FUNCTION allowed_values_integer_types (nqit: non_quantitative_int_type)
    : SET OF integer_type;

    LOCAL
        s : SET OF integer_type := [];
    END_LOCAL;

    REPEAT i:=1 TO SIZEOF (nqit.domain.its_values);
        s := s + nqit.domain.its_values[i].value_code;
    END_REPEAT;
    RETURN (s);

END_FUNCTION; -- allowed_values_integer_types
(*

```

5.12.21 **is_class_valued_property** function

The **is_class_valued_property** function returns TRUE if the property **prop** is defined as a class valued property for class **cl** by means of a **sub_class_properties** attribute in class **cl** or in any of its superclasses. If some class **dictionary_definitions** are not available to compute all the superclasses of **cl**, the function returns UNKNOWN.

EXPRESS specification:

```

*)
FUNCTION is_class_valued_property(
    prop: property_BSU; cl: class_BSU): LOGICAL;
    IF (SIZEOF(cl.definition) = 0)
    THEN
        RETURN (UNKNOWN);
    ELSE
        IF NOT (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
            + '.ITEM_CLASS') IN TYPEOF(cl.definition[1]))
        THEN
            RETURN (FALSE);
        END_IF;
        IF prop IN cl.definition[1].sub_class_properties
        THEN RETURN (TRUE);
        END_IF;
        IF NOT EXISTS(cl.definition[1].its_superclass)
        THEN
            (* end of chain reached, didn't meet super so far *)
            RETURN (FALSE);
        END_IF;
        RETURN(is_class_valued_property(prop,
            cl.definition[1].its_superclass));
    END_IF;

END_FUNCTION; -- is_class_valued_property
(*

```

5.12.22 **Class_value_assigned** function

The **class_value_assigned** function returns the set of values of the property **prop** that have been assigned to a class **cl** by means of a **class_constant_values** attribute in class **cl** or in any superclass of class **cl**. If several values are assigned in several superclasses the function returns the set of all assigned values. If some class **dictionary_definitions** are not available to compute all the superclasses of **cl**, only the values computed are returned.

EXPRESS specification:

```

*)
FUNCTION class_value_assigned(prop: property_BSU;
    cl: class_BSU) : SET OF primitive_value;
    LOCAL
        val: SET OF primitive_value := [];
        cva : SET OF class_value_assignment := [];
    END_LOCAL;
    IF (SIZEOF(cl.definition) = 0)
    THEN
        RETURN (val);
    END_IF;

```

```

END_IF;
IF NOT (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
      + '.ITEM_CLASS') IN TYPEOF(cl.definition[1]))
THEN
  RETURN (val);
END_IF;
IF EXISTS(cl.definition[1])
THEN
  cva:= QUERY
    (a <* cl.definition[1].class_constant_values
     | a.super_class_defined_property = prop);
  REPEAT i :=1 TO SIZEOF (cva);
    val := val + cva[i].assigned_value;
  END_REPEAT;
  IF NOT EXISTS(cl.definition[1].its_superclass)
  THEN
    RETURN (val);
  ELSE RETURN (val + class_value_assigned
    (prop,cl.definition[1].its_superclass));
  END_IF;
END_IF;
END_FUNCTION; -- class_value_assigned

END_SCHEMA; -- ISO13584_IEC61360_dictionary_schema
(*)

```

6 ISO13584_IEC61360_language_resource_schema

6.1 Overview

The following schema provides resources for permitting strings in various languages. It has been extracted from the dictionary schema, since it could be used in other schemata. It is largely based on the **support_resource_schema** from ISO 10303-41, and can be seen as an extension to that. It allows for the usage of one specific language throughout an exchange context (physical file) without the overhead introduced when multiple languages are used. See Figure 13 for a graphical depiction.

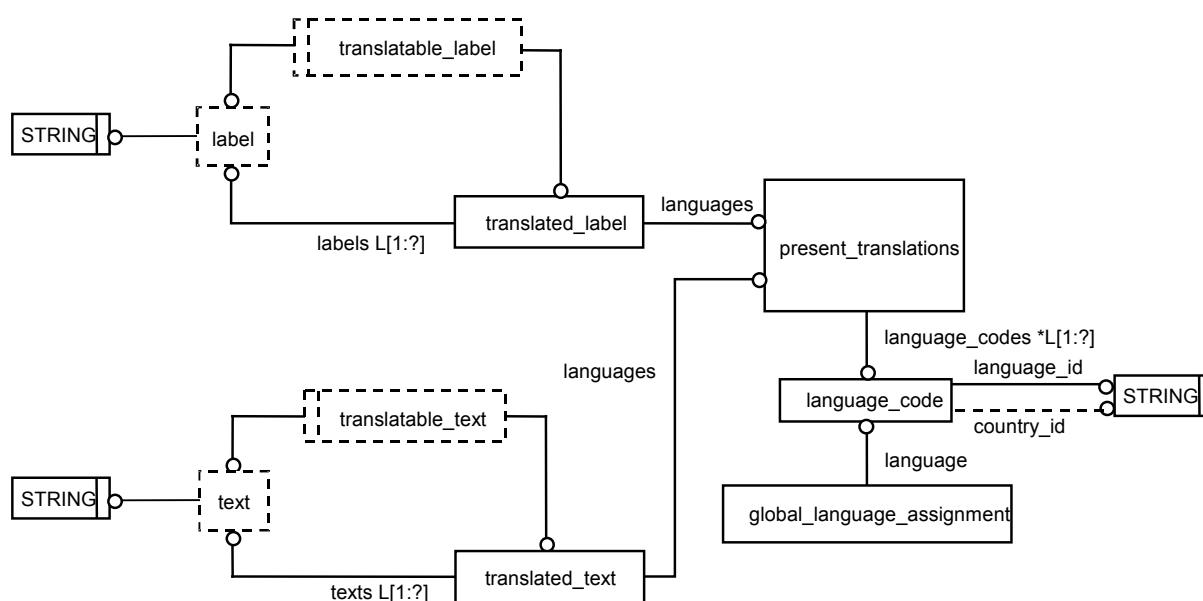


Figure 13 – ISO13584_IEC61360_language_resource_schema and support_resource_schema

EXPRESS specification:

```

*)
SCHEMA ISO13584_IEC61360_language_resource_schema;

REFERENCE FROM support_resource_schema(identifier, label, text);

( *

```

NOTE The **support_resource_schema** schema referenced above can be found in ISO 10303-41.

6.2 ISO13584_IEC61360_language_resource_schema type and entity definitions

6.2.1 general

This subclause contains the EXPRESS type and entity definitions in the **ISO13584_IEC61360_language_resource_schema**.

6.2.2 Language_code

The **language_code** entity enables to identify a language according to ISO 639-1. It contains two codes:

- the language as defined in ISO 639-1 or ISO 639-2, and, optionally
- the country code, as defined in ISO 3166-1, specifying in which country the language is spoken.

EXPRESS specification:

```

*)
ENTITY language_code;
    language_id: identifier;
    country_id: OPTIONAL identifier;

```

```
WHERE
    WR1: (LENGTH (language_id) = 2) OR (LENGTH (language_id) = 3);
    WR2: LENGTH (country_id) = 2;
END_ENTITY; -- language_code
(*)
```

Attribute definitions:

language_id: the code that specifies the language as defined by ISO 639-1 or ISO 639-2.

country_id: the code that specifies the country where the language is spoken as defined by ISO 3166-1.

Formal propositions:

WR1: the length of **language_id** value shall be equal to 2 or 3.

WR2: the length of a **country_id** value shall be equal to 2.

6.2.3 Global_language_assignment

The **global_language_assignment** entity specifies the language for **translatable_label** and **translatable_text**, if **label** and **text** are selected, respectively (i.e., without explicit language indication as is done in **translated_label** and **translated_text**).

EXPRESS specification:

```
*)
ENTITY global_language_assignment;
    language: language_code;
END_ENTITY; -- global_language_assignment
(*)
```

Attribute definitions:

language: the code of the assigned language.

6.2.4 Present_translations

The **present_translations** entity serves to indicate the languages used for **translated_label** and **translated_text**.

EXPRESS specification:

```
*)
ENTITY present_translations;
    language_codes: LIST [1:?] OF UNIQUE language_code;
UNIQUE
    UR1: language_codes;
END_ENTITY; -- present_translations
(*)
```

Attribute definitions:

language_codes: the list of unique language codes corresponding to the language in which a translation is made.

Formal proposition:

UR1: for each list of **language_code** there shall be a unique instance of **present_translations**.

6.2.5 Translatable_label

A **translatable_label** defines a type of values that can be labels or translated_labels.

EXPRESS specification:

```
*)
TYPE translatable_label = SELECT(label, translated_label);
END_TYPE; -- translatable_label
(*
```

6.2.6 Translated_label

The **translated_label** entity defines the labels that are translated and the corresponding languages of translation.

EXPRESS specification:

```
*)
ENTITY translated_label;
    labels: LIST [1:?] OF label;
    languages: present_translations;
WHERE
    WR1: SIZEOF(labels) = SIZEOF(languages.language_codes);
END_ENTITY; -- translated_label
(*
```

Attribute definitions:

labels: the list of **labels** that are translated.

languages: the list of **languages** in which each label is translated.

Formal propositions:

WR1: the number of **labels** contained in the **labels** list shall be equal to the number of languages provided in the **languages.language_codes** attribute.

Informal propositions:

IP1: the content of **labels[i]** is in the language identified by **languages.language_codes[i]**.

6.2.7 Translatable_text

A **translatable_text** defines a type of values that can be **texts** or **translated_texts**.

EXPRESS specification:

```
*)
TYPE translatable_text = SELECT(text, translated_text);
END_TYPE; -- translatable_text
(*
```

6.2.8 Translated_text

The **translated_text** entity defines the **texts** that are translated and the corresponding **languages** of translation.

EXPRESS specification:

```
*)
ENTITY translated_text;
    texts: LIST [1:?] OF text;
    languages: present_translations;
WHERE
    WR1: SIZEOF(texts) = SIZEOF(languages.language_codes);
END_ENTITY; -- translated_text
(*
```

Attribute definitions:

texts: the list of **texts** that are translated.

languages: the list of languages in which each text is translated.

Formal propositions:

WR1: the number of **texts** contained in the **texts** list shall be equal to the number of languages provided in the **languages.language_codes** attribute.

Informal propositions:

IP1: the content of **texts[i]** is in the language identified by **languages.language_codes[i]**.

6.3 ISO13584_IEC61360_language_resource_schema function definitions

6.3.1 General

This subclause contains a function that is referenced in WHERE clauses to assert data consistency.

6.3.2 Check_label_length function

The **check_label_length** function checks that no label in **I** exceeds the length indicated by **l_length**.

EXPRESS specification:

```
*)
FUNCTION check_label_length(l: translatable_label;
    l_length: INTEGER): BOOLEAN;

IF 'ISO13584_IEC61360_LANGUAGE_RESOURCE_SCHEMA.TRANSLATED_LABEL'
```

```

        IN TYPEOF(l)
    THEN
        REPEAT i :=1 TO SIZEOF(l.labels);
            IF LENGTH(l.labels[i]) > l_length
            THEN
                RETURN(FALSE);
            END_IF;
        END_REPEAT;

        RETURN(TRUE);

    ELSE (* the argument l is a single string *)
        RETURN(LENGTH(l) <= l_length);
    END_IF;
END_FUNCTION; -- check_label_length
(*

```

6.4 ISO13584_IEC61360_language_resource_schema rule definition

The rule **single_language_assignment** asserts that only one language may be assigned to be used in **translatable_label** and **translatable_text**.

EXPRESS specification:

```

*)
RULE single_language_assignment FOR(global_language_assignment);
WHERE
    SIZEOF(global_language_assignment) <= 1;
END_RULE; -- single_language_assignment

END_SCHEMA; -- ISO13584_IEC61360_language_resource_schema
(*

```

7 ISO13584_IEC61360_class_constraint_schema

7.1 General

This clause defines the requirements for **class_constraint_schema**. The following EXPRESS declaration introduces the **ISO13584_IEC61360_class_constraint_schema** block and identifies the necessary external references.

EXPRESS specification:

```

*)
SCHEMA ISO13584_IEC61360_class_constraint_schema;

REFERENCE FROM ISO13584_IEC61360_dictionary_schema (
    class_BSU,
    property_BSU,
    definition_available_implies,
    is_subclass,
    data_type,
    simple_type,

```

```
complex_type,
named_type,
allowed_values_integer_types);
```

```
REFERENCE FROM ISO13584_extended_dictionary_schema
(data_type_typeof,
data_type_class_of,
data_type_type_name);
```

```
REFERENCE FROM ISO13584_instance_resource_schema
(Boolean_value,
compatible_class_and_class,
complex_value,
dic_class_instance,
entity_instance_value,
int_level_spec_value,
integer_value,
level_spec_value,
number_value,
primitive_value,
rational_value,
real_level_spec_value,
real_value,
right_values_for_level_spec,
same_translations,
simple_value,
string_value,
translatable_string_value,
translated_string_value,
property_or_data_type_BSU);
```

```
REFERENCE FROM ISO13584_aggregate_value_schema
(aggregate_entity_instance_value,
list_value,
set_value,
bag_value,
array_value,
set_with_subset_constraint_value,
compatible_complete_types_and_value);
```

(*

NOTE The schemata referenced above can be found in the following documents:

ISO13584_IEC61360_dictionary_schema	IEC 61360-2
(which is duplicated for convenience in 4.5 and after).	
ISO13584_extended_dictionary_schema	ISO 13584-24:2003
ISO13584_instance_resource_schema	ISO 13584-24:2003
ISO13584_aggregate_value_schema	ISO 13584-25

7.2 Introduction to the ISO13584_IEC61360_class_constraint_schema

The **ISO13584_IEC61360_class_constraint_schema** provides EXPRESS constructs allowing to redefine, by restriction, the domain of values of a given property when it is applied to a subclass of the characterization class where the property was defined as visible. This constraint shall only make explicit a restriction of the domain of values that already results from the class structure.

EXAMPLE In ISO 13584-511, the class *metric threaded bolt/screw* is a class defined as follows: "headed externally threaded fastener with a cylindrical shank, which may be partly or fully threaded and the head may be furnished with a driving feature". This class has, among other, two properties called *type of head*, and *head properties*. The domain of values of the *type of head* property is a non quantitative data type that includes in particular the following values: *hexagon_head*, *octagonal_head* and *round_head*. The *head properties* property is a feature. It means that it has an *item_class* data type, whose domain is a class *head* that defines any kind of head. The *head* class has several subclasses including: *hexagon head*, associated with all the properties allowing to describe a hexagon head (e. g., *width across flats*), and *round head*, associated with all the properties allowing to describe a round head (e. g., *head diameter*).

The class *metric threaded bolt/screw* has a subclass called *hexagon head screw* defined as follows: "metric externally threaded fastener with a hexagon head threaded up to the head". This class inherits the properties *type of head* and *head*. From the definition of the *hexagon head screw* subclass, it is clear that the *type of head* property could only take the *hexagon_head* value, and that the *head properties* could only be an instance of *hexagon head* feature class. But these constraints are implicit: they are just stated informally in the definition.

Thus these constraints are not computer sensible. The constraints defined in the **ISO13584_IEC61360_class_constraint_schema** would allow to make these two constraints explicit by associating with the *hexagon head screw* class: (1) an **enumeration_constraint** for the *type of head* property (allowing only the *hexagon_head* code) and (2) a **subclass_constraint** for the *head properties* property (allowing only the *hexagon head* feature class).

Constraints are inherited. When a property whose domain of values has already been restricted in a class C through a constraint needs to be further restricted in a subclass of C through another constraint; both constraints apply together. Thus the real domain of values in the C subclass is the intersection of the two domains defined by the two constraints. The proposed mechanism is similar to the type mechanism redefinition operation available in the EXPRESS language.

This schema allows to express constraints that may apply to data types from the type system of the **ISO13584_IEC61360_dictionary_schema**. Rules are used in those entities that reference a constraint to ensure that each constraint may apply to the data type to which it is related.

7.3 ISO13584_IEC61360_class_constraint_schema entity definitions

7.3.1 General

This clause defines the entities in the **ISO13584_IEC61360_class_constraint_schema**.

7.3.2 Constraint

The **constraint** entity allows to define a constraint.

EXPRESS specification:

```
*)
ENTITY constraint
ABSTRACT SUPERTYPE OF ( ONEOF (
    property_constraint,
    class_constraint));
    constraint_id: OPTIONAL constraint_identifier;
END_ENTITY; -- constraint
(*
```

Attribute definitions:

constraint_id: the **constraint_identifier** that identifies the constraint.

7.3.3 Property_constraint

The **property_constraint** entity is a constraint that restricts the allowed set of instances of a class by a single restriction of the domain of values of one of its properties.

EXPRESS specification:

```
*)
ENTITY property_constraint
ABSTRACT SUPERTYPE OF ( ONEOF (
    integrity_constraint,
    context_restriction_constraint))
SUBTYPE OF (constraint);
    constrained_property: property_BSU;
END_ENTITY; -- property_constraint
(*
```

Attribute definitions:

constrained_property: the **property_BSU** for which the constraint applies.

7.3.4 Class_constraint

The **class_constraint** entity is a constraint that restricts the allowed set of instances of a class by constraining several properties or global constraints.

EXPRESS specification:

```
*)
ENTITY class_constraint
ABSTRACT SUPERTYPE OF (configuration_control_constraint)
SUBTYPE OF (constraint);
END_ENTITY; -- class_constraint
(*
```

7.3.5 Configuration_control_constraint

The **configuration_control_constraint** entity allows to restrict the set of instances, called the referenced instances, that a particular instance, called the referencing instance, may reference directly or indirectly by means of a chain of properties. The referencing instance is any instance of a class that references the **configuration_control_constraint** by means of its **constraints** attribute or inherits of a class that does so. The **configuration_control_constraint** entity defines an optional **precondition** that specifies the condition on the referencing instance for the restriction to apply. It defines a **postcondition** that specifies the allowed sets of values for some properties of the referenced instance class.

EXAMPLE A *bolted assembly* consists of the following set of fasteners: one *externally threaded fastener*, any number of *washers* and one or more *nuts*. There exist various kinds of threads, including *tapping screw thread*, *wood screw thread*, *metric external thread*, *metric internal thread*, *imperial internal thread* and *imperial external thread*. Let us assume that one wants to describe a *metric bolted assembly*. One needs to ensure that whatever be the precise structure of the assembly, both the *externally threaded fastener* and all the *nuts* involved in the assembly have metric thread. This can be done by specifying in the *metric bolted assembly* class the **configuration_control_constraint** that ensures that any ISO 13584-511 -described fastener referenced by any instance of this class, or any of its subclass, should belong to classes that either do not have value for the *type of thread* property (e. g., *washer*), or whose values should belong to the set: {*metric external thread*, *metric internal thread*}.

NOTE 1 Both **precondition** and **postcondition** may only restrict properties whose data type is **non_quantitative_code_type**. Such properties may be assigned a value either at the instance level, or at the class level if they are declared as class valued properties, i.e., **sub_class_properties** in a **class**.

NOTE 2 Properties referenced in the **precondition** should be applicable to the class that references the **configuration_control_constraint**.

NOTE 3 In a **configuration_control_constraint**, **filters** are used both to represent precondition on the referencing instance and to express constraints on the referenced instances.

EXPRESS specification:

```
*)
ENTITY configuration_control_constraint
  SUBTYPE OF (class_constraint);
    precondition: SET [0:?] OF filter;
    postcondition: SET [1:?] OF filter;
END_ENTITY; -- configuration_control_constraint
(*
```

Attribute definitions:

precondition: the **filters** that shall hold on the referencing instance for the restriction to apply.

NOTE 4 If the set of filters is empty, the restriction applies on any referencing instance.

postcondition: the **filters** that shall hold on a referenced instance for being allowed for reference.

7.3.6 Filter

The **filter** entity is an **enumeration_constraint** that restricts the allowed domain of a property whose data type is either **non_quantitative_code_type** or **non_quantitative_int_type**.

EXPRESS specification:

```
*)
ENTITY filter;
  referenced_property: property_BSU;
  domain: enumeration_constraint;
WHERE
  WR1: definition_available_implies (
    referenced_property,
    (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
    +'.NON_QUANTITATIVE_CODE_TYPE') IN TYPEOF(
    referenced_property.
    definition[1]\property_DET.domain))
    OR
    (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
    +'.NON_QUANTITATIVE_INT_TYPE') IN TYPEOF(
    referenced_property.
    definition[1]\property_DET.domain)));
  WR2: definition_available_implies (
    referenced_property,
    correct_constraint_type(domain,
    referenced_property.definition[1].domain));
END_ENTITY; -- filter
(*
```

Attribute definitions:

referenced_property: the property whose domain of values is restricted by the **filter**.

domain: the **enumeration_constraint** that restricts the domain of values of the referenced property.

Formal propositions:

WR1: the data type of the **referenced_property** shall be either **non_quantitative_code_type** or **non_quantitative_int_type**.

WR2: the **domain** shall define a domain of values that may restrict the initial domain of values of the property.

7.3.7 Integrity_constraint

The **integrity_constraint** entity is a particular property constraint that allows to make explicit that for some particular class, as a result of the class definition, and all its subclasses, only a restriction of the domain of values specified by a data type is allowed for a property.

EXAMPLE In the reference dictionary defined for fasteners in ISO 13584-511, a *metric threaded bolt/screw* has a *head properties* property that may take, as value, a member of any subclass of the *head* feature class. If the *metric threaded bolt/screw* is also a member of the *hexagon head screw* subclass, the *head properties* may only be a member of the *hexagon head* feature class, else the *metric threaded bolt/screw* cannot be a member of the *hexagon head screw* subclass.

NOTE In the example above, the integrity constraint does not change at all the meaning of the *head properties* property inherited from *metric threaded bolt/screw* into *hexagon head screw*. It just makes explicit the fact that in the context of the *hexagon head screw* subclass, only a subset of the values allowed for this property in the context of the *metric threaded bolt/screw* class remains allowed.

EXPRESS specification:

```
*)
ENTITY integrity_constraint
SUBTYPE OF (property_constraint);
    redefined_domain: domain_constraint;
WHERE
    WR1: definition_available_implies (constrained_property,
        correct_constraint_type(redefined_domain,
            constrained_property.definition[1].domain));
END_ENTITY; -- integrity_constraint
(*)
```

Attribute definitions:

redefined_domain: the constraint that applies on the domain of values of the constrained property.

Formal propositions:

WR1: the **redefined_domain** shall define a domain of values that restricts the initial domain of values of the property.

7.3.8 Context_restriction_constraint

The **context_restriction_constraint** entity is a **property_constraint** that restricts the allowed domain of values for the context parameters on which a context dependent property depends.

EXPRESS specification:

```

*)
ENTITY context_restriction_constraint
SUBTYPE OF (property_constraint);
    context_parameter_constraints: SET [1:?] OF property_constraint;
WHERE
    WR1: definition_available_implies(constrained_property,
        QUERY (cp < *SELF.context_parameter_constraints
            | NOT (cp.constrained_property IN
                constrained_property.definition[1].depends_on))=[]);
    WR2: QUERY (cp < *SELF.context_parameter_constraints
        | NOT (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
            + '.INTEGRITY_CONSTRAINT') IN TYPEOF (cp))) =[];
    WR3: definition_available_implies(constrained_property,
        'ISO13584_IEC61360_DICTIONARY_SCHEMA.DEPENDENT_P_DET'
        IN TYPEOF(constrained_property.definition[1]));
END_ENTITY; -- context_restriction_constraint
(*

```

Attribute definitions:

context_parameter_constraints: the constraint that applies on the domain of values of the context parameters.

Formal propositions:

WR1: the set of properties whose domain is constrained by the **context_parameter_constraints** property shall be context parameters on which the constrained property depends.

WR2: all the **context_parameter_constraints** shall be **integrity_constraints**.

WR3: the **constrained_property** shall be a context dependent **property dependent_P_DET**.

7.3.9 Domain_constraint

A **domain_constraint** defines a constraint that restricts the domain of values of a data type.

EXPRESS specification:

```

*)
ENTITY domain_constraint
ABSTRACT SUPERTYPE OF (ONEOF(
    subclass_constraint,
    entity_subtype_constraint,
    enumeration_constraint,
    range_constraint,
    string_size_constraint,
    string_pattern_constraint,

```

```

        cardinality_constraint
    ));
END_ENTITY; -- domain_constraint
(*)

```

7.3.10 Subclass_constraint

A **subclass_constraint** restricts the domain of values of a **class_reference_type** to one or several subclasses of the class that defines its initial domain.

EXPRESS specification:

```

*)
ENTITY subclass_constraint
SUBTYPE OF(domain_constraint);
    subclasses: SET [1:?] OF class_BSU;
END_ENTITY; -- subclass_constraint
(*)

```

Attribute definitions:

subclasses: the **class_BSUs** which redefine the class to which the value of the **constrained_property** shall belong.

7.3.11 Entity_subtype_constraint

An **entity_subtype_constraint** restricts the domain of values of an **entity_instance_type** to a subtype of the ENTITY that defines its initial domain.

EXPRESS specification:

```

*)
ENTITY entity_subtype_constraint
SUBTYPE OF(domain_constraint);
    subtype_names: SET[1:?] OF STRING;
END_ENTITY; -- entity_subtype_constraint
(*)

```

Attribute definitions:

subtype_names: the set of strings that describe, in the format of the EXPRESS TYPEOF function, the EXPRESS entity data type names that shall belong to the result of the EXPRESS TYPEOF function when it is applied to a value that references the **constrained_property** redefined property.

7.3.12 Enumeration_constraint

An **enumeration_constraint** restricts the domain of values of a data type to a list of values defined in extension. The order defined by the list is the recommended order for presentation purposes. A particular description may optionally be associated with each value of the subset by means of a **non_quantitative_int_type**, of which the i-the value describes the meaning of the i-the value of the subset.

For those subtypes of **number_type** that are associated with a **dic_unit** and alternative units, and possibly with a **dic_unit_identifier** and alternative unit identifiers, the constraint applies

to the value corresponding to the **dic_unit**, or to the single **dic_unit_identifier**. If both exist, they correspond to the same unit.

For those subtypes of **number type** that are associated with a currency, the constraint applies to the currency specified in their data type definition. If no currency is specified in the data type definition, the constraint shall not be used.

For those values that belong to **translatable_string_types**, the constraint applies to the string that is in the source language into which the property domain was defined. This source language may be defined in the **source_language** attribute of the **administrative_data** of the property. If this attribute does not exist, this source language is supposed to be known by the dictionary user.

If another **enumeration_constraint** is applied on a property already associated with an **enumeration_constraint** in some superclass, both constraints apply. Thus the allowed set of values is the intersection of both subsets. Concerning the presentation order, and the possible meaning associated with each value, only those meanings defined in the lower **enumeration_constraint** apply.

EXAMPLE 1 If, in class C1, this property is associated with an **enumeration_constraint** whose **subset** attribute equals {1, 3, 5, 7}, then in class C1, and any of its subclasses, property P1 may only takes one of the four following values: 1 or 3 or 5 or 7.

EXAMPLE 2 If the data type of property P1 is LIST [1:4] OF INTEGER, and if in class C1 this property is associated with an **enumeration_constraint** whose **subset** attribute equals { {1}, {3, 5}, {7}, {1, 3, 7} } then, in class C1 and any of its subclasses, property P1 may only takes one of the four following values: {1} or {3, 5} or {7} or {1, 3, 7}.

EXPRESS specification:

```
*)
ENTITY enumeration_constraint
SUBTYPE OF (domain_constraint);
    subset: LIST [1:?] OF UNIQUE primitive_value;
    value_meaning: OPTIONAL non_quantitative_int_type;
WHERE
    WR1: (NOT (EXISTS (SELF.value_meaning)))
    OR
    (integer_values_in_range(1, SIZEOF(SELF.subset))
    = allowed_values_integer_types(SELF.value_meaning));
END_ENTITY; -- enumeration_constraint
(*)
```

Attribute definitions:

subset: the list describing the subset of values that are allowed as possible values for the property identified by **constrained_property**.

value_meaning: the set of **dic_values** that define the meaning of each value belonging to the **subset**.

Formal propositions:

WR1: if the **value_meaning non_quantitative_int_type** exists, then the set of **value_codes** of its **dic_values** shall be in 1.. SIZE_OF(subset).

7.3.13 Range_constraint

A **range_constraint** entity restricts the domain of values of an ordered type to a subset of values defined by a range.

NOTE 1 Strings are not considered as ordered types and cannot be constrained by a **range_constraint**.

For those subtypes of **number_type** that are associated with a **dic_unit** and alternative units, and possibly with a **dic_unit_identifier** and alternative unit identifiers, the constraint applies to the value corresponding to the **dic_unit**, or to the single **dic_unit_identifier**. If both exists, they correspond to the same unit.

For those subtypes of **number_type** that are associated with a currency, the constraint applies to the currency specified in their data type definition. If no currency is specified in the data type definition, the constraint shall not be used.

NOTE 2 For **non_quantitative_int_type** the constraint applies to the **value_code**.

EXPRESS specification:

```
*)
ENTITY range_constraint
SUBTYPE OF (domain_constraint);
    min_value, max_value: OPTIONAL NUMBER;
    min_inclusive, max_inclusive: OPTIONAL BOOLEAN;
WHERE
    WR1: min_value <= max_value;
    WR2: TYPEOF(min_value) = TYPEOF(max_value);
    WR3: NOT EXISTS (min_value) OR EXISTS (min_inclusive);
    WR4: NOT EXISTS (max_value) OR EXISTS (max_inclusive);
END_ENTITY; -- range_constraint
(*
```

Attribute definitions:

min_value: the number defining the low bound of the range of values; not existing value means no lower bound.

max_value: the number defining the high bound of the range of values; not existing value means no upper bound.

min_inclusive: specifies whether **min_value** belongs to the range; not existing value means that there is no low bound.

max_inclusive: specifies whether **max_value** belongs to the range; not existing value means that there is no high bound.

Formal propositions:

WR1: **min_value** shall be less than or equal to **max_value**.

WR2: **min_value** and **max_value** shall have the same data types.

WR3: if **min_value** has a value, then **min_inclusive** shall also have a value.

WR4: if **max_value** has a value, then **max_inclusive** shall also have a value.

7.3.14 String_size_constraint

A **string_size_constraint** restricts the length of the STRING values allowed for a STRING type, or any of its subtypes.

NOTE 1 A **string_type** property value domain is either a **string_type**, a **non_translatable_string_type**, a **translatable_string_type**, a **URI_type**, a **non_quantitative_code_type**, a **date_data_type**, a **time_data_type** or a **date_time_data_type**.

NOTE 2 For **non_quantitative_code_type** the constraint applies to the code.

For those values that belong to **translatable_string_types**, the constraint applies to the string that is in the source language into which the property domain was defined. This source language may be defined in the **source_language** attribute of the **administrative_data** of the property. If this attribute does not exist, this source language is supposed to be known by the dictionary user.

EXPRESS specification:

```
*)
ENTITY string_size_constraint
SUBTYPE OF (domain_constraint);
    min_length: OPTIONAL INTEGER;
    max_length: OPTIONAL INTEGER;
WHERE
    WR1: (min_length >= 0) AND (max_length >= min_length);
END_ENTITY; -- string_size_constraint
(*
```

Attribute definitions:

min_length: the minimal length for the strings that are allowed as values for the property identified by the **constrained_property** property.

max_length: the maximal length for the strings that are allowed as values for the property identified by the **constrained_property** property.

NOTE 3 If the **min_length** value does not exist, 0 is understood. If the **max_length** value does not exist, unbounded is understood.

Formal propositions:

WR1: **min_length** and **max_length** define correct bounds.

7.3.15 String_pattern_constraint

A **string_pattern_constraint** restricts the domain of values of a **string_type**, or of any of its subtypes, to string values that match a particular pattern. The **pattern** syntax is defined by an XML regular expression and the associated matching algorithms that are defined by the XML Schema Part 2: Datatypes recommendation.

NOTE 1 A **string_type** property value domain is either a **string_type**, a **non_translatable_string_type**, a **translatable_string_type**, an **URI_type**, a **non_quantitative_code_type**, a **date_data_type**, a **time_data_type** or a **date_time_data_type**.

For **string_type**, **non_translatable_string_type**, **URI_type**, **non_quantitative_code_type**, **date_data_type**, **time_data_type** or **date_time_data_type** the constraint applies to the (unique) string that is the value of the data type. For **non_quantitative_code_type** the constraint applies to the code.

For those values that belong to `translatable_string_types`, the constraint applies to the string that is in the source language into which the property domain was defined. This source language may be defined in the `source_language` attribute of the `administrative_data` of the property, if this attribute does not exist, this source language is supposed to be known by the dictionary user.

NOTE 2 For `non_quantitative_code_type`, `date_data_type`, `time_data_type` or `date_time_data_type`, the **pattern** should comply with the informal propositions defined in the corresponding data types.

EXPRESS specification:

```
*)
ENTITY string_pattern_constraint
  SUBTYPE OF (domain_constraint);
    pattern: STRING;
END_ENTITY; -- string_pattern_constraint
(*
```

Attribute definition:

pattern: the pattern of string values that are allowed as values for the property identified by the constrained property.

Informal proposition:

IP1: the **pattern** syntax shall comply with the XML regular expression syntax and the associated matching algorithms that are defined by the XML Schema Part 2: Datatypes recommendation.

EXAMPLE The XML Schema pattern that corresponds to the "[0-9][0-9][0-9][0-9]-[0-9][0-9]-[0-9][0-9]" SQL SIMILAR expression is "[0-9]{4}\-[0-9]{2}\-[0-9]{2}". It allows to match strings as "2009-05-31".

7.3.16 Cardinality_constraint

A **cardinality_constraint** restricts the cardinality of an aggregate data type.

NOTE 1 The resulting cardinality range is the intersection of preexisting cardinality ranges and of the one defined by the **cardinality_constraint**.

NOTE 2 **Cardinality_constraints** are not allowed on **array_type**.

EXPRESS specification:

```
*)
ENTITY cardinality_constraint
  SUBTYPE OF (domain_constraint);
    bound_1: OPTIONAL INTEGER;
    bound_2: OPTIONAL INTEGER;
WHERE
    WR1: (bound_1 >= 0) AND (bound_2 >= bound_1);
END_ENTITY; -- cardinality_constraint
(*
```

Attribute definitions:

bound_1: the lower bound of the cardinality.

bound_2: the upper bound of the cardinality.

NOTE 3 When **bound_1** does not exist, the minimal cardinality is 0. When **bound_2** does not exist, there is no constraint on the maximal cardinality.

Formal propositions:

WR1: **bound_1** and **bound_2** define correct bounds.

7.4 ISO13584_IEC61360_class_constraint_schema type definitions

7.4.1 General

This subclause defines the type in the ISO13584_IEC61360_class_constraint_schema.

7.4.2 Constraint_or_constraint_id

The **constraint_or_constraint_id** is either a constraint or a **constraint_identifier**.

EXPRESS specification:

```
*)
TYPE constraint_or_constraint_id =
    SELECT (constraint, constraint_identifier);
END_TYPE; -- constraint_or_constraint_id
(*
```

7.5 ISO13584_IEC61360_class_constraint_schema function definition

7.5.1 General

This subclause defines the functions in the ISO13584_IEC61360_class_constraint_schema.

7.5.2 Integer_values_in_range function

The **integer_values_in_range** function computes the integer values that belong to an integer range defined by its low bound and its high bound. It returns indeterminate (?) when either bounds are indeterminate.

EXPRESS specification:

```
*)
FUNCTION integer_values_in_range(
    low_bound, high_bound: INTEGER): SET OF INTEGER;
LOCAL
    i: INTEGER;
    result: SET OF INTEGER:= [];
END_LOCAL;
IF EXISTS (low_bound) AND EXISTS (high_bound)
THEN
    REPEAT i := low_bound TO high_bound;
        result := result + [i];
    END_REPEAT;
    RETURN(result);
ELSE
    RETURN(?);
END_IF;
END_FUNCTION; -- integer_values_in_range
(*
```

7.5.3 Correct_precondition function

The **correct_precondition** function checks that the precondition of the **configuration_control_constraint** defined by **cons** uses only properties that are applicable to the **cl** class. It returns a logical that is UNKNOWN when the complete set of applicable properties of the class cannot be computed.

EXPRESS specification:

```

*)
FUNCTION correct_precondition(
    cons: configuration_control_constraint; cl:class): LOGICAL;
LOCAL
    prop: SET OF property_BSU:= [];
END_LOCAL;
REPEAT i := 1 to SIZEOF (cons.precondition);
    prop := prop + cons.precondition[i].referenced_property;
END_REPEAT;

IF prop <= cl.known_applicable_properties
THEN RETURN (TRUE);
ELSE
    IF all_class_descriptions_reachable(cl.identified_by)
    THEN RETURN (FALSE);
    ELSE RETURN (UNKNOWN);
    END_IF;
END_IF;
END_FUNCTION; -- correct_precondition
(*)

```

7.5.4 Correct_constraint_type function

The **correct_constraint_type** function checks that the **domain_constraint** defined by **cons** is compatible with the **data_type** defined by **typ**. It returns a logical that is UNKNOWN when the **domain_constraint** defined by **cons** is not one of the subtypes defined in the **ISO13584_IEC61360_class_constraint_schema**.

EXPRESS specification:

```

(*)
FUNCTION correct_constraint_type(
    cons: domain_constraint; typ:data_type): LOGICAL;

(*case subclass constraint*)

IF ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    + 'SUBCLASS_CONSTRAINT') IN TYPEOF(cons)
THEN
    (*the data type shall be class_reference_type*)
    IF NOT
        ('ISO13584_IEC61360_DICTIONARY_SCHEMA.CLASS_REFERENCE_TYPE'
        IN TYPEOF (typ))
    THEN RETURN(FALSE);
    END_IF;

    (*the cons.subclasses shall consist of subclasses for the class

```

```

        that defined the initial domain of typ.*)
    IF NOT (QUERY (sc <* cons.subclasses |
        definition_available_implies
        (sc,definition_available_implies
        (typ\class_reference_type.domain,is_subclass(sc.definition[1]
        , typ\class_reference_type.domain.definition[1]))= false)
        = []))
    THEN RETURN(FALSE);
    END_IF;

    RETURN (TRUE);
END_IF;

(*case entity subtype constraint*)

IF (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    +'ENTITY_SUBTYPE_CONSTRAINT') IN TYPEOF (CONS))
THEN

    (* the data type is a class_reference_type*)
    IF NOT (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
        +'.ENTITY_INSTANCE_TYPE') IN TYPEOF (typ))
    THEN RETURN(FALSE);
    END_IF;

    (* the subtype_name shall define a subtype for the entity_instance_type
    of the constrained *)
    IF NOT (cons\entity_subtype_constraint.subtype_names
        >= typ\entity_instance_type.type_name)
    THEN RETURN(FALSE);
    END_IF;

    RETURN (TRUE);
END_IF;

(*case enumeration_constraint *)

IF ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    +'.ENUMERATION_CONSTRAINT') IN TYPEOF (CONS)
THEN

    (* all the values belonging to the subset of values shall be compatible
    with the typ data type *)
    IF (QUERY (val<*cons.subset |
        NOT compatible_data_type_and_value ( typ, val))<> [])
    THEN RETURN(FALSE);
    END_IF;

    RETURN (TRUE);
END_IF;

(*case range_constraint *)

IF ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA.RANGE_CONSTRAINT'
    IN TYPEOF (CONS))

```

THEN

(*if the data type is an integer_type then min_value and max_value shall be INTEGERS.*)

```
IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.INTEGER_TYPE'
    IN TYPEOF (typ)) AND
    NOT ('INTEGER' IN TYPEOF (cons.min_value))
THEN RETURN(FALSE);
END_IF;
```

(*if the data type is a rational_type then min_value and max_value shall be rational.*)

```
IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.RATIONAL_TYPE'
    IN TYPEOF (typ)) AND
    NOT ('ISO13584_INSTANCE_RESOURCE_SCHEMA.RATIONAL_VALUE' IN
    TYPEOF (cons.min_value))
THEN RETURN(FALSE);
END_IF;
```

(*if the data type is a real_type then min_value and max_value shall be REALs.*)

```
IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.REAL_TYPE'
    IN TYPEOF (typ)) AND NOT ('REAL' IN TYPEOF (cons.min_value))
THEN RETURN(FALSE);
END_IF;
```

(*all values of the range shall belong to the allowed values defined by the type.*)

```
IF (('ISO13584_IEC61360_DICTIONARY_SCHEMA'
    + '.NON_QUANTITATIVE_INT_TYPE') IN TYPEOF (typ))
AND NOT
    (integer_values_in_range(cons.min_value, cons.max_value)
    <= allowed_values_integer_types (typ))
THEN RETURN(FALSE);
END_IF;
```

RETURN (TRUE);

END_IF;

(*case entity string_size_constraint*)

```
IF ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    + '.STRING_SIZE_CONSTRAINT') IN TYPEOF (CONS)
THEN
```

(* the data type shall be a string_type or any of its subtypes *)

```
IF NOT ('ISO13584_IEC61360_DICTIONARY_SCHEMA.STRING_TYPE'
    IN TYPEOF (typ))
THEN RETURN(FALSE);
END_IF;
```

RETURN (TRUE);

END_IF;

(*case entity string_pattern_constraint *)

```

IF ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    +'.STRING_PATTERN_CONSTRAINT') IN TYPEOF (CONS)
THEN

(* the data type shall be a string_type or any of its subtypes *)
IF NOT ('ISO13584_IEC61360_DICTIONARY_SCHEMA.STRING_TYPE'
    IN TYPEOF (typ))
THEN RETURN(FALSE);
END_IF;
RETURN (TRUE);
END_IF;

(*case entity cardinality_constraint *)

IF ('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    +'.CARDINALITY_CONSTRAINT') IN TYPEOF (CONS)
THEN

(* the data type shall be an aggregate type but not an array*)
IF (NOT(
    ('ISO13584_IEC61360_DICTIONARY_AGGREGATE_EXTENSION_SCHEMA'
    + '.ENTITY_INSTANCE_TYPE_FOR_AGGREGATE')
    IN TYPEOF(typ)))
THEN
    RETURN(FALSE);
END_IF;

IF ('ISO13584_IEC61360_DICTIONARY_AGGREGATE_EXTENSION_SCHEMA'
    + '.ARRAY_TYPE' IN TYPEOF(typ.type_structure))
THEN
    RETURN(FALSE);
END_IF;
RETURN (TRUE);
END_IF;

RETURN (UNKNOWN);

END_FUNCTION; -- correct_constraint_type
(*)

```

7.5.5 Compatible_data_type_and_value function

The function **compatible_data_type_and_value** checks if a value **val** of a **primitive_value** is type compatible with the type defined by a type **dom**. It returns a LOGICAL that is TRUE when they are compatible and FALSE when they are not. This function returns UNKNOWN if the **val** data type is an **uncontrolled_instance_value** (see ISO 13584-24:2003) or when its type is not one of the types defined in the **ISO13584_instance_resource_schema**.

NOTE The value **val** may or may not exist.

EXPRESS specification:

```

*)
FUNCTION compatible_data_type_and_value(dom: data_type;
    val: primitive_value): LOGICAL;

```

```

LOCAL
    temp: class_BSU;
    set_string: SET OF STRING := [];
    set_integer: SET OF INTEGER := [];
    code_type: non_quantitative_code_type;
    int_type: non_quantitative_int_type;
END_LOCAL;

(* The following express statements deal with simple types *)

IF ('ISO13584_INSTANCE_RESOURCE_SCHEMA.INTEGER_VALUE' IN TYPEOF(val))
THEN
    IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.' +
        'NON_QUANTITATIVE_INT_TYPE' IN TYPEOF (dom))
    THEN
        set_integer := [];
        int_type := dom;
        REPEAT j := 1 TO SIZEOF(int_type.domain.its_values);
            set_integer := set_integer +
                int_type.domain.its_values[j].value_code;
        END_REPEAT;

        RETURN(val IN set_integer);

    ELSE
        RETURN(('ISO13584_IEC61360_DICTIONARY_SCHEMA.INT_TYPE'
            IN TYPEOF (dom)) OR
            (('ISO13584_IEC61360_DICTIONARY_SCHEMA.NUMBER_TYPE'
            IN TYPEOF (dom))
            AND NOT(('ISO13584_IEC61360_DICTIONARY_SCHEMA.REAL_TYPE'
            IN TYPEOF (dom))
            OR ('ISO13584_IEC61360_DICTIONARY_SCHEMA.RATIONAL_TYPE'
            IN TYPEOF (dom)))));

    END_IF;
END_IF;

IF ('ISO13584_INSTANCE_RESOURCE_SCHEMA.REAL_VALUE' IN TYPEOF(val))
THEN
    RETURN(('ISO13584_IEC61360_DICTIONARY_SCHEMA.REAL_TYPE'
        IN TYPEOF (dom)) OR
        (('ISO13584_IEC61360_DICTIONARY_SCHEMA.NUMBER_TYPE'
        IN TYPEOF (dom))
        AND NOT(('ISO13584_IEC61360_DICTIONARY_SCHEMA.INT_TYPE'
        IN TYPEOF (dom))
        OR ('ISO13584_IEC61360_DICTIONARY_SCHEMA.RATIONAL_TYPE'
        IN TYPEOF (dom)))));
END_IF;

IF ('ISO13584_INSTANCE_RESOURCE_SCHEMA.RATIONAL_VALUE' IN TYPEOF(val))
THEN
    RETURN(('ISO13584_IEC61360_DICTIONARY_SCHEMA.RATIONAL _TYPE'
        IN TYPEOF (dom)) OR

```

```

        (('ISO13584_IEC61360_DICTIONARY_SCHEMA.NUMBER_TYPE'
        IN TYPEOF (dom))
        AND NOT (('ISO13584_IEC61360_DICTIONARY_SCHEMA.INT_TYPE'
        IN TYPEOF (dom))
        OR ('ISO13584_IEC61360_DICTIONARY_SCHEMA.REAL_TYPE'
        IN TYPEOF (dom)))));
END_IF;

IF ('ISO13584_INSTANCE_RESOURCE_SCHEMA.STRING_VALUE'
    IN TYPEOF(val))
THEN
    IF (('ISO13584_IEC61360_DICTIONARY_SCHEMA' +
        '.NON_QUANTITATIVE_CODE_TYPE') IN TYPEOF (dom))
    THEN
        set_string := [];
        code_type := dom;
        REPEAT j := 1 TO SIZEOF(code_type.domain.its_values);
            set_string := set_string +
                code_type.domain.its_values[j].value_code;
        END_REPEAT;

        RETURN(val IN set_string);

    ELSE
        RETURN('ISO13584_IEC61360_DICTIONARY_SCHEMA' +
            '.STRING_TYPE' IN TYPEOF (dom));
    END_IF;
END_IF;

IF ('ISO13584_INSTANCE_RESOURCE_SCHEMA.TRANSLATED_STRING_VALUE'
    IN TYPEOF(val))
THEN
    RETURN('ISO13584_IEC61360_DICTIONARY_SCHEMA' +
        '.TRANSLATABLE_STRING_TYPE' IN TYPEOF (dom));
END_IF;

(* The following express statements deal with complex types *)

IF 'ISO13584_INSTANCE_RESOURCE_SCHEMA.DIC_CLASS_INSTANCE'
    IN TYPEOF(val)
THEN
    IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.CLASS_REFERENCE_TYPE'
        IN TYPEOF (dom))
    THEN
        temp := dom.domain;
        RETURN(compatible_class_and_class(temp,
            val\dic_class_instance.class_def));
    ELSE
        RETURN(FALSE);
    END_IF;
END_IF;

IF 'ISO13584_INSTANCE_RESOURCE_SCHEMA.LEVEL_SPEC_VALUE' IN TYPEOF(val)
THEN

```

```

IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.LEVEL_TYPE'
    IN TYPEOF (dom))
THEN
    RETURN(compatible_level_type_and_instance(
        dom.levels,
        TYPEOF(dom.value_type),
        val));
ELSE
    RETURN(FALSE);
END_IF;
END_IF;

(* The following express statements deal with aggregate types *)

IF 'ISO13584_AGGREGATE_VALUE_SCHEMA.AGGREGATE_ENTITY_INSTANCE_VALUE' IN
    TYPEOF(val) THEN

    IF (NOT(
        'ISO13584_IEC61360_DICTIONARY_SCHEMA.ENTITY_INSTANCE_TYPE'
        IN TYPEOF(dom)))
    THEN
        RETURN(FALSE);
    END_IF;

    IF (NOT(
        'ISO13584_IEC61360_DICTIONARY_AGGREGATE_EXTENSION_SCHEMA'
        + '.AGGREGATE_TYPE' IN dom.type_name))
    THEN
        RETURN(FALSE);
    END_IF;

    RETURN(compatible_aggregate_type_and_value(dom, val));

END_IF;

IF 'ISO13584_INSTANCE_RESOURCE_SCHEMA.ENTITY_INSTANCE_VALUE'
    IN TYPEOF(val)
THEN
    IF 'ISO13584_INSTANCE_RESOURCE_SCHEMA' +
        '.UNCONTROLLED_ENTITY_INSTANCE_VALUE'
        IN TYPEOF(val)
    THEN
        RETURN(UNKNOWN);
    END_IF;
    IF ('ISO13584_IEC61360_DICTIONARY_SCHEMA.ENTITY_INSTANCE_TYPE'
        IN TYPEOF (dom))
        AND (dom.type_name <= TYPEOF(val))
    THEN
        RETURN(TRUE);
    ELSE
        RETURN(FALSE);
    END_IF;
END_IF;

```

```
RETURN (UNKNOWN) ;
```

```
END_FUNCTION; -- compatible_data_type_and_value
(*
```

7.6 ISO13584_IEC61360_class_constraint_schema rule definition

7.6.1 General

This subclause defines the rule in the ISO13584_IEC61360_class_constraint_schema.

7.6.2 Unique_constraint_id

The **unique_constraint_id** rule asserts that two **constraint_identifiers** associated with two different **constraints** have different values.

EXPRESS specification:

```
*)
RULE unique_constraint_id FOR(constraint);
WHERE
    QUERY(c1 <* constraint |
        SIZEOF(QUERY(c2 <* constraint |
            c1.constraint_id = c2.constraint_id))>1) = [];
END_RULE; -- unique_constraint_id
(*

*)
END_SCHEMA; -- ISO13584_IEC61360_class_constraint_schema

(*
```

8 ISO13584_IEC61360_item_class_case_of_schema

8.1 Overview

This clause defines the requirement for the **ISO13584_IEC61360_item_class_case_of_schema**. The following EXPRESS declaration introduces the **ISO13584_IEC61360_item_class_case_of_schema** block and identifies the necessary external references.

EXPRESS specification:

```
*)
SCHEMA ISO13584_IEC61360_item_class_case_of_schema;

REFERENCE FROM ISO13584_IEC61360_dictionary_schema
    (all_class_descriptions_reachable,
     class,
     class_BSU,
     data_type_BSU,
     item_class,
     property_BSU);

REFERENCE FROM ISO13584_IEC61360_class_constraint_schema
```

```
(constraint,
integrity_constraint,
context_restriction_constraint,
property_constraint,
domain_constraint);
```

```
REFERENCE FROM ISO13584_extended_dictionary_schema
(document_BSU,
table_BSU,
visible_properties,
applicable_properties,
visible_types,
applicable_types,
data_type_named_type);
```

(*

NOTE The schemata referenced above can be found in the following documents:

ISO13584_IEC61360_dictionary_schema	IEC 61360-2
(which is duplicated for convenience in this document).	
ISO13584_IEC61360_class_constraint_schema	IEC 61360-2
(which is duplicated for convenience in this document).	
ISO13584_extended_dictionary_schema	ISO 13584-24:2003

8.2 Introduction to the ISO13584_IEC61360_item_class_case_of_schema

For modularity reasons, the complete common ISO/IEC dictionary model is split among several documents. The kernel model for product ontologies is defined in IEC 61360-2 and duplicated in this part of IEC 61360. Resources for extending this model, including instance representation, document representation, functional model, functional views and table representation are defined in ISO 13584-24:2003. The various standard levels of implementation of the complete ISO/IEC model, called conformance classes, are defined in ISO 13584-25, and duplicated for informative purpose in IEC 61360-5. The first level corresponds precisely to the content of this part of IEC 61360 more the resource for aggregate-structured values, defined in ISO 13584-25. Other conformance classes include more and more resources from ISO 13584-24:2003.

To define an item class as case-of another item class is more and more used by applications based on the common ISO13584/IEC61360 dictionary model. Moreover, this edition of this part of IEC 61360 has required a change of the information model of this concept. Therefore, it has been decided to move the corresponding EXPRESS entity, called **item_class_case_of**, and its superclass, called **a_priori_semantic_relationship**, from ISO 13584-24:2003 to this part of IEC 61360. These entities are included in a new schema, called **ISO13584_IEC61360_item_class_case_of_schema**. ISO 13584-24:2003 and ISO 13584-25 will be updated accordingly by means of a technical corrigendum.

8.3 ISO13584_IEC61360_item_class_case_of_schema entity definitions

8.3.1 A priori semantic relationship

An **a_priori_semantic_relationship** is an abstract **class** that is defined on the basis of other classes, and that can import properties, data types, tables and documents contained in these classes. It also imports all the **constraints** that restricted the domain of the imported properties in the classes from which they are imported. This abstract resource is intended to be subtyped by classes. When a class specializes an **a_priori_semantic_relationship**, the properties, data types, tables or documents whose definitions are imported by inheritance of **a_priori_semantic_relationship** become applicable to the class that imports them. In particular, properties and data types that are so imported are allowed for use for describing

class instances, and the fact, for a product, to have an aspect that corresponds to each imported property is a necessary criteria for being member of the class.

NOTE 1 All imported properties and data types become directly applicable without being visible. Thus, they are not returned by the **compute_known_visible_properties** or **compute_known_visible_data_types** function.

NOTE 2 The inheritance relationship is a well known example of semantic relationship between classes modelled according to the object oriented paradigm. All the properties and other resources defined in a class usually apply implicitly to all its subclasses. This relationship is used in ISO 13584 series where all the properties, data types, tables or documents visible (respectively applicable) to some classes are implicitly visible (respectively applicable) to all its subclasses. As usual, in ISO 13584 this inheritance is implicit (i.e., not declared by means of an **a_priori_semantic_relationship**) and global (i.e., all the properties and data types are inherited by all its subclasses). An **a_priori_semantic_relationship** enables to define other semantic relationships that are useful in the ISO 13584 application domain, and in particular the case-of relationship that allows an explicit and partial importation of properties, and of other resources defined in a class.

EXPRESS specification:

```

*)
ENTITY a_priori_semantic_relationship
ABSTRACT SUPERTYPE
SUBTYPE OF(class);
    referenced_classes: SET [1:?] OF class_BSU;
    referenced_properties: LIST [0:?] OF property_BSU;
    referenced_data_types: SET [0:?] OF data_type_BSU;
    referenced_tables: SET [0:?] OF table_BSU;
    referenced_documents: SET [0:?] OF document_BSU;
    referenced_constraints: SET [0:?] OF constraint_or_constraint_id;
WHERE
    WR1: QUERY (cons <* SELF.referenced_constraints
        | NOT (('ISO13584_IEC61360_DICTIONARY_SCHEMA' +
            '.ISO_29002_IRDI_type') IN TYPEOF(cons))
        AND NOT (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
            + '.PROPERTY_CONSTRAINT') IN TYPEOF (cons)))
        = [];
    WR2: QUERY (cons <* SELF.referenced_constraints
        | (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
            + '.PROPERTY_CONSTRAINT') IN TYPEOF (cons))
        AND NOT (cons\property_constraint.constrained_property
            IN SELF.referenced_properties))
        = [];
    WR3: compute_known_referenced_property_constraints(SELF)
        <= SELF.referenced_constraints;
    WR4: QUERY(prop <* SELF.referenced_properties
        | QUERY(cl <* SELF.referenced_classes
            | visible_properties(cl, [prop])
            OR applicable_properties(cl, [prop])))
        = []) = [];
    WR5: QUERY(typ <* SELF.referenced_data_types
        | QUERY(cl <* SELF.referenced_classes
            | visible_types(cl, [typ])
            OR applicable_types(cl, [typ])))
        = []) = [];
END_ENTITY; -- a_priori_semantic_relationship
(*)

```

Attribute definitions:

referenced_classes: the class(es) from where the properties, data types, tables or documents are imported.

NOTE 3 The class from which properties, data types, tables or documents are imported cannot be deduced from the identification of the imported properties, data types, tables or documents because they may be imported from a class where they are inherited or imported. For instance, in IEC 61360-DB, "input-voltage" is a property visible at the root level of the IEC classification. If a supplier class imports the "input-voltage" property from the IEC "transistor" class, this means that (1) the supplier class defines a transistor, and (2) these transistors are described by means of an "input voltage" property.

referenced_properties: the properties whose definitions are imported through the **a_priori_semantic_relationship** entity.

NOTE 4 The list order defines the default order for displaying imported properties during user access to the various subtypes of **a_priori_semantic_relationship**.

referenced_data_types: the data types whose definitions are imported through the **a_priori_semantic_relationship** entity.

referenced_tables: the tables whose definitions are imported through the **a_priori_semantic_relationship** entity.

NOTE 5 Detailed resources and rules on the use of tables are defined in ISO 13584-24:2003. They are not used in this part of IEC 61360, nor in the integrated models documented in ISO 13584-32 (OntoML) and ISO 13584-25.

referenced_documents: the documents whose definitions are imported through the **a_priori_semantic_relationship** entity.

NOTE 6 Detailed resources and rules on the use of documents are defined in ISO 13584-24:2003. They are used in the integrated models documented in ISO 13584-32 (OntoML) or ISO 13584-25.

referenced_constraints: the **property_constraints** that apply to the various imported properties.

NOTE 7 Unlike other referenced entities, the **referenced_constraints** constraints cannot be selected when the **a_priori_semantic_relationship** is designed. These constraints are all the constraints that affect any of the properties defined in the **referenced_properties** attribute in any class of the **referenced_classes** attribute.

Formal propositions:

WR1: all the **referenced_constraints** that are not IRDI shall be **property_constraints**.

WR2: all the **referenced_constraints** shall constrain properties that are imported through the **referenced_properties** attribute.

WR3: all the **property_constraints** that constrain one of the **referenced_properties** property in any of the **referenced_classes** class shall be imported through the **referenced_constraints** attribute.

WR4: the imported properties defined by the **referenced_properties** attribute shall be visible or applicable for one of the classes belonging to the **referenced_classes** attribute.

WR5: the imported types defined by the **referenced_data_types** attribute shall be visible or applicable for one of the classes belonging to the **referenced_classes** attribute.

Informal propositions:

IP1: all the **constraints** that are represented by **constraint_identifiers** in the **referenced_constraints** set shall correspond to **property_constraints** that constrain one of

the **referenced_properties** properties in one of the **referenced_classes** class. Such constraint shall not be represented, in the same **referenced_constraints** set, both as a **property_constraint** and as a **constraint_identifier**.

NOTE 8 A constraint represented as a **property_constraint** in one of the **referenced_classes** class may be represented in the **referenced_constraints** set either as a **property_constraint** or as a **constraint_identifier**

IP2: all the constraints that are represented by a **constraint_identifier** in one of the **referenced_classes** classes but whose corresponding **constraint** is a **property_constraint** that constrains one of the properties imported through the **referenced_properties** attribute shall be represented by their **constraint_identifier** in the **referenced_constraints** set.

NOTE 9 These two informal rules ensure that the **referenced_constraints** set of constraints is the union of the sets of **property_constraints** defined in the various **referenced_classes** classes whose **constrained_property** belongs to the **referenced_properties** set, even when the exchange context does not contain the definitions of all the classes involved in the **a_priori_semantic_relationship** and when some **constraints** are only represented by their **constraint_identifiers**.

8.3.2 Item_class_case_of

An **item_class_case_of** is the description of an item class that is defined as a is-case-of of some other item class(es).

NOTE 1 An **item_class_case_of** defines an a priori semantic relationship.

EXPRESS specification:

```

*)
ENTITY item_class_case_of
SUBTYPE OF (item_class, a_priori_semantic_relationship);
    is_case_of: SET [1:?] OF class_BSU;
    imported_properties: LIST [0:?] OF property_BSU;
    imported_types: SET [0:?] OF data_type_BSU;
    imported_tables: SET [0:?] OF table_BSU;
    imported_documents: SET [0:?] OF document_BSU;
    imported_constraints: SET [0:?] OF constraint_or_constraint_id;
DERIVE
    SELF\ a_priori_semantic_relationship.referenced_classes:
        SET [1:?] OF class_BSU := SELF.is_case_of;
    SELF\ a_priori_semantic_relationship.referenced_properties:
        LIST [0:?] OF property_BSU := SELF.imported_properties;
    SELF\ a_priori_semantic_relationship.referenced_data_types:
        SET [0:?] OF data_type_BSU := SELF.imported_types;
    SELF\ a_priori_semantic_relationship.referenced_tables:
        SET [0:?] OF table_BSU := SELF.imported_tables;
    SELF\ a_priori_semantic_relationship.referenced_documents:
        SET [0:?] OF document_BSU := SELF.imported_documents;
    SELF\ a_priori_semantic_relationship.referenced_constraints:
        SET [0:?] OF property_constraint
        := SELF.imported_constraints;
WHERE
    WR1: superclass_of_item_is_item(SELF);
    WR2: check_is_case_of_referenced_classes_definition(SELF);
    WR3: QUERY(p <* SELF\class.sub_class_properties
        | NOT((p IN SELF.described_by)
        OR (p IN SELF.imported_properties))) = [];
    WR4: QUERY(p <* SELF\class.sub_class_properties
        | (p IN SELF.imported_properties)
        AND (QUERY(c1<*SELF.is_case_of

```

```

| all_class_descriptions_reachable(cl) AND
(p IN compute_known_applicable_properties(cl)) AND
(NOT is_class_valued_property(p, cl))<>[]))
=[];
WR5: QUERY(ccv <* SELF\class.class_constant_values
| (ccv.super_class_defined_property
IN SELF.imported_properties)
AND (QUERY(cl<*SELF.is_case_of
| all_class_descriptions_reachable(cl) AND
(ccv.super_class_defined_property
IN compute_known_applicable_properties(cl)) AND
(QUERY (v<*class_value_assigned(
ccv.super_class_defined_property, cl)
|v<> ccv.assigned_value) <> []))<>[]))
=[];
WR6: QUERY(prop <* imported_properties
| (QUERY(cl<*SELF.is_case_of
| is_class_valued_property(prop, cl)) <>[]))
AND NOT is_class_valued_property(prop, SELF.identified_by))
=[];
WR7: QUERY(ccv <* SELF\class.class_constant_values
| QUERY(cl<*SELF.is_case_of
| (class_value_assigned
(ccv.super_class_defined_property, cl) <> []))
AND (QUERY(v <* class_value_assigned
(ccv.super_class_defined_property, cl)
| v <> ccv.assigned_value)<>[])) <> []))
=[];
END_ENTITY; -- item_class_case_of
(*)

```

Attribute definitions:

is_case_of: the **item_class(es)** of which the present **item_class** is-case-of.

imported_properties: the list of properties that are imported from the **item_class(es)** the defined **item_class** is-case-of.

imported_types: the set of data types that are imported from the **item_class(es)** the defined **item_class** is-case-of.

imported_tables: the set of **table_BSU**s that are imported from the **item_class(es)** the defined **item_class** is-case-of.

imported_documents: the set of **document_BSU**s that are imported from the **item_class(es)** the defined **item_class** is-case-of.

imported_constraints: the set of **property_constraints** or **constraint_id** that are imported from the **item_class(es)** the defined **item_class** is-case-of.

NOTE 2 Unlike other imported entities, the **imported_constraints** constraints cannot be selected when an **item_class_case_of** is designed. These constraints are all the constraints that restrict the domains of any of the properties defined in the **imported_properties** in the classes of the **is_case_of** attribute from which they are imported. This is specified in a where rule of **a_priori_semantic_relationship**.

Formal propositions:

WR1: the superclass of an **item_class_case_of** shall be an **item_class**.

WR2: an **item_class_case_of** shall be case-of **item_class(es)**.

WR3: the **sub_class_properties** shall belong either to the **described_by** list, or to the **imported_properties** list.

WR4: all the class valued properties declared by means of the **sub_class_properties** that are **imported_properties** shall be class valued properties in all the **is-case-of** classes where they are applicable.

WR5: the values assigned to an imported property by means of the **class_constant_value** assignment shall not be different than the possible value assigned to the same property in the referenced classes.

WR6: all the **imported_properties** that are class valued properties in a class from the **is_case_of** set of classes, shall be class valued properties in the current class.

WR7: all the **imported_properties** that are assigned a **class_constant_value** in a class from the **is_case_of** set of classes, shall be assigned the same **class_constant_value** class value in the current class.

8.4 ISO13584_IEC61360_item_class_case_of_schema function definitions

8.4.1 General

This subclause contains functions that are referenced in WHERE clauses to assert data consistency or that provide resources for application development.

8.4.2 Compute_known_property_constraints function

The **compute_known_property_constraints** function computes the set of **property_constraints** that applies for the properties of a set of classes. Constraints represented by their identifiers are not returned. When the definition of some classes is not available, the function returns only the **property_constraints** that may be computed.

NOTE When the **dictionary_definition** of a class is not available in the same exchange context (a PLIB exchange context is never assumed to be complete), its own superclass may not be known. Therefore the constraints defined by this superclass cannot be computed by the **compute_known_property_constraints** function. On the contrary, when all the is-a superclasses of a class are available in the same exchange context, all the constraints that apply to this class may be computed by a single traversal of its is-a inheritance tree, even when some of these superclasses import properties by means of **a_priori_semantic_relationships** such as **item_class_case_of**.

EXPRESS specification:

```

*)
FUNCTION compute_known_property_constraints(classes: SET OF class_BSU) :
    SET OF property_constraint;

LOCAL
    s: SET OF property_constraint := [];
END_LOCAL;

REPEAT nb := 1 TO SIZEOF (classes);
    IF SIZEOF(classes[nb].definition)=1
    THEN
        REPEAT i := 1 TO
            SIZEOF(classes[nb].definition[1]\class.constraints);

```

```

    IF (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
    + '.PROPERTY_CONSTRAINT')
    IN TYPEOF
        (classes[nb].definition[1]\class.constraints[i]))
    THEN
        s := s + classes[nb].definition[1]\class.constraints[i];
    END_IF;
END_REPEAT;

IF (('ISO13584_IEC61360_ITEM_CLASS_CASE_OF_SCHEMA.'
+ 'A_PRIORI_SEMANTIC_RELATIONSHIP')
IN TYPEOF (classes[nb].definition[1]))
THEN
    REPEAT i := 1 TO
        SIZEOF(classes[nb].definition[1]
        \a_priori_semantic_relationship
        .referenced_constraints);

        IF (('ISO13584_IEC61360_CLASS_CONSTRAINT_SCHEMA'
        + '.PROPERTY_CONSTRAINT') IN TYPEOF
            (classes[nb].definition[1]
            \a_priori_semantic_relationship
            .referenced_constraints[i]))
        THEN
            s := s + classes[nb].definition[1]
            \a_priori_semantic_relationship
            .referenced_constraints [i];
        END_IF;
    END_REPEAT;
END_IF;

IF EXISTS(classes[nb].definition[1]\class.its_superclass)
THEN
    s := s + compute_known_property_constraints(
        [classes[nb].definition[1]\class.its_superclass]);
END_IF;

END_IF;
END_REPEAT;
RETURN (s);

END_FUNCTION; -- compute_known_property_constraints
(*

```

8.4.3 Compute_known_referenced_property_constraints function

The **compute_known_referenced_property_constraints** function computes all the **property_constraints** that should be imported by an **ap a_priori_semantic_relationship** by computing all the constraints that apply to a property that is imported through the **referenced_properties** attribute of **ap**, and that are defined or inherited in any **ap referenced_classes** class whose **class dictionary_definition** is available in the same exchange context.

NOTE 1 In an **a_priori_semantic_relationship** all the **property_constraints** defined in or inherited by the classes referenced by the **referenced_classes** attribute that apply to a property that is imported through the

referenced_properties attribute of the **a_priori_semantic_relationship** should be imported through its **referenced_constraints** attribute.

NOTE 2 When the **dictionary_definition** of a class belonging to the **referenced_classes** attribute of **ap** is not available in the same exchange context as **ap** (a PLIB exchange context is never assumed to be complete), the constraints belonging to this class cannot be computed. Thus the result of the function **compute_known_referenced_property_constraints** may be only a subset of the constraints that should be imported by **ap**.

NOTE 3 When the **dictionary_definitions** of all the **referenced_classes** classes of **ap** are available in the same exchange context, and when no constraint is represented by a single **constraint_identifier**, the function **compute_known_referenced_property_constraints** returns exactly all the constraints that should be imported by **ap**.

EXPRESS specification:

```

*)
FUNCTION compute_known_referenced_property_constraints(
    ap: a_priori_semantic_relationship):
    SET OF property_constraint;

LOCAL
    s: SET OF property_constraint := []; -- result
    prop: SET OF property_BSU :=
        list_to_set(ap.referenced_properties); --imported properties
    cl: SET OF class_BSU :=
        ap.referenced_classes; --source of importation
    cons: SET OF property_constraint
        := compute_known_property_constraints(cl);
        -- all those property_constraints existing in the various
        -- classes from cl that may be computed in the current
        -- exchange context.
END_LOCAL;

    REPEAT n_cons := 1 TO SIZEOF(cons);
        IF cons[n_cons].constrained_property IN prop
        THEN
            s := s + cons[n_cons];
        END_IF;
    END_REPEAT;
RETURN (s);

END_FUNCTION; -- compute_known_referenced_property_constraints
(*)

```

8.4.4 Superclass_of_item_is_item function

The **superclass_of_item_is_item** function checks that the superclass of an **item_class** **cl**, if it exists, is an **item_class**.

If the **class** associated with a **class_BSU** cannot be computed, the function returns UNKNOWN.

EXPRESS specification:

```

*)
FUNCTION superclass_of_item_is_item(cl: item_class): LOGICAL;

IF NOT EXISTS(cl\class.its_superclass)
THEN
    RETURN(TRUE);
END_IF;

```

```

IF SIZEOF(cl\class.its_superclass.definition) = 0
THEN
    RETURN (UNKNOWN);
END_IF;

RETURN (('ISO13584_IEC61360_DICTIONARY_SCHEMA.ITEM_CLASS')
    IN TYPEOF(cl\class.its_superclass.definition[1]));

END_FUNCTION; -- superclass_of_item_is_item
(*)

```

8.4.5 Check_is_case_of_referenced_classes_definition function

The **check_is_case_of_referenced_classes_definition** returns TRUE if the **item_class_case_of is_case_of** set of referenced class dictionary definition(s) is type compatible with the given **cl item_class_case_of** instance. Otherwise, it returns FALSE.

EXPRESS specification:

```

*)
FUNCTION check_is_case_of_referenced_classes_definition(
    cl: item_class_case_of): BOOLEAN;
LOCAL
    class_def_ok: BOOLEAN := TRUE;
END_LOCAL;

REPEAT i := 1 TO SIZEOF(cl.is_case_of);
    IF (SIZEOF(cl.is_case_of[i].definition) = 1)
    THEN
        IF (NOT('ISO13584_IEC61360_DICTIONARY_SCHEMA' +
            '.ITEM_CLASS'
            IN TYPEOF(cl.is_case_of[i].definition[1])))
        THEN
            class_def_ok := FALSE;
        END_IF;
    END_IF;
END_REPEAT;

RETURN(class_def_ok);

END_FUNCTION; -- check_is_case_of_referenced_classes_definition
(*)

```

8.5 ISO13584_IEC61360_item_class_case_of_schema rule definitions

8.5.1 General

This subclause defines the rule in the **ISO13584_IEC61360_item_class_case_of_schema**.

8.5.2 Imported_properties_are_visible_or_applicable_rule rule

The **imported_properties_are_visible_or_applicable_rule** rule checks that when a property is imported by a class by means of an **a_priori_semantic_relationship**, this property is visible or applicable for the class it is imported from.

NOTE Applicable properties include the properties imported through a semantic relationship. This rule enables to import properties from a class where they were already imported.

EXPRESS specification:

```
*)
RULE imported_properties_are_visible_or_applicable_rule FOR(
    a_priori_semantic_relationship, property_DET);
WHERE
    WR1: QUERY(rel <* a_priori_semantic_relationship
        | QUERY(prop <* rel.referenced_properties
        | QUERY(cl <* rel.referenced_classes
        | NOT visible_properties(cl, [prop])
        AND NOT applicable_properties(cl, [prop]))
        = rel.referenced_classes) = [])
        = a_priori_semantic_relationship;
END_RULE; -- imported_properties_are_visible_or_applicable_rule
(*
```

8.5.3 Imported_data_types_are_visible_or_applicable_rule rule

The **imported_data_types_are_visible_or_applicable_rule** rule checks that when a data type is imported by a class by means of an **a_priori_semantic_relationship**, this data type is visible or applicable for the class it is imported from.

NOTE Applicable data types include the data types imported through a semantic relationship. This rule enables to import data types from a class where they were already imported.

EXPRESS specification:

```
*)
RULE imported_data_types_are_visible_or_applicable_rule FOR(
    a_priori_semantic_relationship, data_type_element);
WHERE
    WR1: QUERY(rel <* a_priori_semantic_relationship
        | QUERY(typ <* rel.referenced_data_types
        | QUERY(cl <* rel.referenced_classes
        | NOT visible_types(cl, [typ])
        AND NOT applicable_types(cl, [typ]))
        = rel.referenced_classes) = [])
        = a_priori_semantic_relationship;
END_RULE; -- imported_data_types_are_visible_or_applicable_rule
(*
```

8.5.4 Allowed_named_type_usage_rule rule

The **allowed_named_type_usage_rule** rule is related to the usage of a named type. It states that only types that are applicable to a class may be used to specify the domain of the properties declared by a class, through its **described_by** attribute.

EXPRESS specification:

```
*)
RULE allowed_named_type_usage_rule FOR(class);
LOCAL
    named_type_usage_allowed: LOGICAL := TRUE;
    is_app: LOGICAL;
```

```

    prop: property_bsu;
    cl: class;
    dtnt: SET[0:1] OF data_type_bsu := [];
END_LOCAL;

REPEAT i := 1 TO SIZEOF(class);
    cl := class[i];
    REPEAT j := 1 TO SIZEOF(class[i].described_by);
        prop := cl.described_by[j];
        dtnt := data_type_named_type(prop);

        IF (SIZEOF(dtnt) = 1) THEN
            is_app := applicable_types(cl.identified_by, dtnt);
            IF (NOT is_app) THEN
                named_type_usage_allowed := FALSE;
            END_IF;
        END_IF;
    END_REPEAT;
END_REPEAT;

WHERE
    WR1: named_type_usage_allowed;
END_RULE; -- allowed_named_type_usage_rule
(*)

*)
END_SCHEMA; -- ISO13584_IEC61360_item_class_case_of_schema
(*)

```

Annex A (informative)

Example physical file

A.1 Objective

This annex gives some fragments of a physical file for exchanging the data of IEC 61360-DB. It is intended to show the use of the EXPRESS model in Clause 5 "ISO13584_IEC61360_dictionary_schema" together with ISO 10303-21 to exchange corresponding data.

A.2 File Header

```
*/
ISO-10303-21;
HEADER;
FILE_DESCRIPTION(('Example physical file'), '2;1');
FILE_NAME('example.spf', '2007-07-18', ('IEC SC3D WG2'), (),
'Version 1', '', '');
FILE_SCHEMA(('example_schema'));
ENDSEC;
DATA;
/*
```

A.3 Supplier data

```
*/
#1=SUPPLIER_BSU('112/2///61360_4_1', *); /*according to ISO 13584-26*/
#2=SUPPLIER_ELEMENT(#1, #3, '01', $, $, $, #4, #5);
#3=DATES('1994-09-16', '1994-09-16', $);
#4=ORGANIZATION('IEC', 'IEC Maintenance Agency', 'The IEC Maintenance
Agency');
#5=ADDRESS('to be determined', $, $, $, $, $, $, $, $, $, $, $);
#10=SUPPLIER_BSU('112/3///_00', *); /* ISO/IEC ICS */
/*
```

A.4 Root class data

The AAA000 IEC root class provides a name scope corresponding IEC 61360-DB. It covers two trees, one for materials, one for components. It is defined as an **item_class**.

```
*/
#90=CLASS_BSU('00', '001', #10);
#100=CLASS_BSU('AAA000', '001', #1);
#101=ITEM_CLASS(#100, #3, '01', $, $, $, #102, TEXT('IEC root class that
provides a name scope corresponding to IEC 61360-DB. It covers two trees,
one for materials, one for components'), $, $, $, #90, (#110), (), (), $,
(), (#110), (), $, $, $);
#102=ITEM_NAMES(LABEL('IEC root class'), (), LABEL('IEC root'), $, $);
#110=PROPERTY_BSU('AAE000', '001', #100);
```

```
#111=NON_DEPENDENT_P_DET(#110, #3, '01', $, $,$, #112, TEXT('the type of
tree: material or component'), $, $, $, $, (), $, $, #113, $);
#112=ITEM_NAMES(LABEL('type of tree'), (), LABEL('tree type'), $, $);
#113=NON_QUANTITATIVE_CODE_TYPE((), 'A..8', #114);
#114=VALUE_DOMAIN((#120,#122), $, $, (), $, $);
#120=DIC_VALUE(VALUE_CODE_TYPE('MATERIAL'), #121, $, $, $, $, $, $);
#121=ITEM_NAMES(LABEL('material tree'), (), LABEL('mat tree'), $, $);
#122=DIC_VALUE(VALUE_CODE_TYPE('COMPONS'), #123, $, $, $, $, $, $);
#123=ITEM_NAMES(LABEL('component tree'), (), LABEL('comp tree'), $, $);
/*
```

A.5 Material data

```
*/
#200=CLASS_BSU('AAA218', '001', #1);
#201=ITEM_CLASS(#200, #3, '01', $, $, $, #202, TEXT('root class of the
materials tree'), $, $, $, #100, (#210,#230), (), (), $, (), (#210),
(#205), $, 'MATERIAL', $);
#202=ITEM_NAMES(LABEL('materials root class'), (), LABEL('materials root'),
$, $);
#205=CLASS_VALUE_ASSIGNMENT(#110, STRING_VALUE('MATERIAL'));
#210=PROPERTY_BSU('AAF311', '005', #100);
#211=NON_DEPENDENT_P_DET(#210, #3, '01', $, $, $, #212, TEXT('code of the
type of material'), $, $, $, $, (), $, 'A57', #213, $);
#212=ITEM_NAMES(LABEL('material type'), (), LABEL('material type'), $, $);
#213=NON_QUANTITATIVE_CODE_TYPE((), 'M..3', #214);
#214=VALUE_DOMAIN((#220,#222,#224,#226), $, $, (), $, $);
#220=DIC_VALUE(VALUE_CODE_TYPE('ACO'), #221, $, $, $, $, $, $);
#221=ITEM_NAMES(LABEL('acoustical'), (), LABEL('acoustical'), $, $);
#222=DIC_VALUE(VALUE_CODE_TYPE('MG'), #223, $, $, $, $, $, $);
#223=ITEM_NAMES(LABEL('magnetic'), (), LABEL('magnetical'), $, $);
#224=DIC_VALUE(VALUE_CODE_TYPE('OP'), #225, $, $, $, $, $, $);
#225=ITEM_NAMES(LABEL('optical'), (), LABEL('optical'), $, $);
#226=DIC_VALUE(VALUE_CODE_TYPE('TH'), #227, $, $, $, $, $, $);
#227=ITEM_NAMES(LABEL('thermal-electric'), (), LABEL('th-electric'), $, $);
#230=PROPERTY_BSU('AAF286', '005', #100);
#231=NON_DEPENDENT_P_DET(#230, #3, '01', $, $,$, #232, TEXT('The nominal
density (in kg/m**3) of a material.'), $, $, $, #233, (), $, 'K02', #234,
$);
#232=ITEM_NAMES(LABEL('density'), (), LABEL('density'), $, $);
#233=MATHEMATICAL_STRING('$r_d', '&rho;<sub>d</sub>');
#234=REAL_MEASURE_TYPE((), 'NR3..3.3ES2', #235, $, $, $);
#235=DIC_UNIT(#236, $);
#236=DERIVED_UNIT((#239,#237));
#237=DERIVED_UNIT_ELEMENT(#238, 1.);
#238=SI_UNIT(*, .KILO., .GRAM.);
#239=DERIVED_UNIT_ELEMENT(#240, -3.);
#240=SI_UNIT(*, $, .METRE.);
/*
```

A.6 Component data

```

*/
#300=CLASS_BSU('EEE000', '001', #1);
#301=ITEM_CLASS(#300, #3, '01', $, $, $, #302, TEXT('root class of the
components tree'), $, $, $, #100, (#310,#330,#350), (), (), $, (), (#310),
(#305), $, 'COMPONS', $);
#302=ITEM_NAMES(LABEL('components root class'), (), LABEL('components
root'), $, $);
#305=CLASS_VALUE_ASSIGNMENT(#110, STRING_VALUE('COMPONS'));
#310=PROPERTY_BSU('AAE001', '005', #100);
#311=NON_DEPENDENT_P_DET(#310, #3, '01', $, $, $, #312, TEXT('Code of the
main functional class to which a component belongs'), $, $, $, $, (), $,
'A52', #313, $);
#312=ITEM_NAMES(LABEL('main class of component'), (), LABEL('main class'),
$, $);
#313=NON_QUANTITATIVE_CODE_TYPE((), 'M..3', #314);
#314=VALUE_DOMAIN((#320,#322,#324,#326), $, $, (), $, $);
#320=DIC_VALUE(VALUE_CODE_TYPE('EE'), #321, $, $, $, $, $, $);
#321=ITEM_NAMES(LABEL('EE (electric / electronic)'), (), LABEL('EE'), $,
$);
#322=DIC_VALUE(VALUE_CODE_TYPE('EM'), #323, $, $, $, $, $, $);
#323=ITEM_NAMES(LABEL('electromechanical'), (), LABEL('electromech'), $,
$);
#324=DIC_VALUE(VALUE_CODE_TYPE('ME'), #325, $, $, $, $, $, $);
#325=ITEM_NAMES(LABEL('mechanical'), (), LABEL('mechanical'), $, $);
#326=DIC_VALUE(VALUE_CODE_TYPE('MP'), #327, $, $, $, $, $, $);
#327=ITEM_NAMES(LABEL('magnetic part'), (), LABEL('magnetic'), $, $);
#330=PROPERTY_BSU('AAF267', '005', #100);
#331=NON_DEPENDENT_P_DET(#330, #3, '01', $, $, $, #332, TEXT('The nominal
distance (in m) between the inside of the two tapes used for taped products
with axial leads'), $, $, $, #333, (), $, 'T03', #334, $);
#332=ITEM_NAMES(LABEL('inner tape spacing'), (), LABEL('inner tape spac'),
$, $);
#333=MATHEMATICAL_STRING('b_tape', 'b<sub>tape</sub>');
#334=LEVEL_TYPE((), (.NOM.), #335);
#335=REAL_MEASURE_TYPE((), 'NR3..3.3ES2', #336, $, $, $);
#336=DIC_UNIT(#337, $);
#337=SI_UNIT(*, $, .METRE.);
#350=PROPERTY_BSU('AAE022', '005', #100);
#351=NON_DEPENDENT_P_DET(#350, #3, '01', $, $, $, #352, TEXT('The value as
specified by level (miNoMax) of the outside diameter (in m) of a component
with a body of circular cross-section'), $, $, $, #353, (), $, 'T03', #354,
$);
#352=ITEM_NAMES(LABEL('outside diameter'), (), LABEL('outside diam'), $,
$);
#353=MATHEMATICAL_STRING('d_out', 'd<sub>out</sub>');
#354=LEVEL_TYPE((), (.MIN., .NOM., .MAX.), #355);
#355=REAL_MEASURE_TYPE((), 'NR3..3.3ES2', #356, $, $, $);
#356=DIC_UNIT(#357, #358);

```

```
#357=SI_UNIT(*, $,.METRE.);
#358=MATHEMATICAL_STRING('m', 'm');
/*
```

A.7 Electric / electronic component data

```
*/
#400=CLASS_BSU('EEE001', '001', #1);
#401=ITEM_CLASS(#400, #3, '01', $, $,$, #402, TEXT('electric / electronic
components'), $, $, $, #300, (#410,#470), (), (), $, (), (#410), (#405), $,
'EE', $);
#402=ITEM_NAMES(LABEL('EE components'), (), LABEL('EE components'), $, $);
#405=CLASS_VALUE_ASSIGNMENT(#310, STRING_VALUE('EE'));
#410=PROPERTY_BSU('AAE002', '005', #100);
#411=NON_DEPENDENT_P_DET(#410, #3, '01', $, $,$, #412, TEXT('Code of the
category to which an electric/electronic component belongs.'), $, $, $, $,
(), $, 'A52', #413, $);
#412=ITEM_NAMES(LABEL('category EE component'), (), LABEL('categ EE comp'),
$, $);
#413=NON_QUANTITATIVE_CODE_TYPE((), 'M..3', #414);
#414=VALUE_DOMAIN((#420,#422,#424,#426,#428
,#430,#432,#434,#436,#438
,#440), $, $, (), $, $);
#420=DIC_VALUE(VALUE_CODE_TYPE('AMP'), #421, $, $, $, $, $, $);
#421=ITEM_NAMES(LABEL('amplifier'), (), LABEL('amplifier'), $, $);
#422=DIC_VALUE(VALUE_CODE_TYPE('ANT'), #423, $, $, $, $, $, $);
#423=ITEM_NAMES(LABEL('antenna (aerial)'), (), LABEL('antenna (aer)'), $,
$);
#424=DIC_VALUE(VALUE_CODE_TYPE('BAT'), #425, $, $, $, $, $, $);
#425=ITEM_NAMES(LABEL('battery'), (), LABEL('battery'), $, $);
#426=DIC_VALUE(VALUE_CODE_TYPE('CAP'), #427, $, $, $, $, $, $);
#427=ITEM_NAMES(LABEL('capacitor'), (), LABEL('capacitor'), $, $);
#428=DIC_VALUE(VALUE_CODE_TYPE('CND'), #429, $, $, $, $, $, $);
#429=ITEM_NAMES(LABEL('conductor'), (), LABEL('conductor'), $, $);
#430=DIC_VALUE(VALUE_CODE_TYPE('DEL'), #431, $, $, $, $, $, $);
#431=ITEM_NAMES(LABEL('delay line'), (), LABEL('delay line'), $, $);
#432=DIC_VALUE(VALUE_CODE_TYPE('DID'), #433, $, $, $, $, $, $);
#433=ITEM_NAMES(LABEL('diode device'), (), LABEL('diode device'), $, $);
#434=DIC_VALUE(VALUE_CODE_TYPE('FIL'), #435, $, $, $, $, $, $);
#435=ITEM_NAMES(LABEL('filter'), (), LABEL('filter'), $, $);
#436=DIC_VALUE(VALUE_CODE_TYPE('IC'), #437, $, $, $, $, $, $);
#437=ITEM_NAMES(LABEL('integrated circuit'), (), LABEL('IC'), $, $);
#438=DIC_VALUE(VALUE_CODE_TYPE('IND'), #439, $, $, $, $, $, $);
#439=ITEM_NAMES(LABEL('inductor'), (), LABEL('inductor'), $, $);
#440=DIC_VALUE(VALUE_CODE_TYPE('LAM'), #441, $, $, $, $, $, $);
#441=ITEM_NAMES(LABEL('lamp'), (), LABEL('lamp'), $, $);
#470=PROPERTY_BSU('AAE754', '005', #100);
```

```
#471=NON_DEPENDENT_P_DET(#470, #3, '01', $, $, $, #472, TEXT('The number of
electrical terminals of an electric/electronic or electromechanical
component'), $, $, $, #473, (), $, 'Q56', #474, $);
#472=ITEM_NAMES(LABEL('number of terminals'), (LABEL('number of pins')),
LABEL('nr of terminals'), $, $);
#473=MATHEMATICAL_STRING('N_term', 'N<sub>term</sub>');
#474=INT_TYPE((), 'NR1..4');

ENDSEC;
END-ISO-10303-21;
```

Annex B
(informative)

EXPRESS-G Diagram

This annex contains the EXPRESS-G diagrams for the Clauses 6 through 8. EXPRESS-G is defined in Annex A of ISO 10303-11:2004.

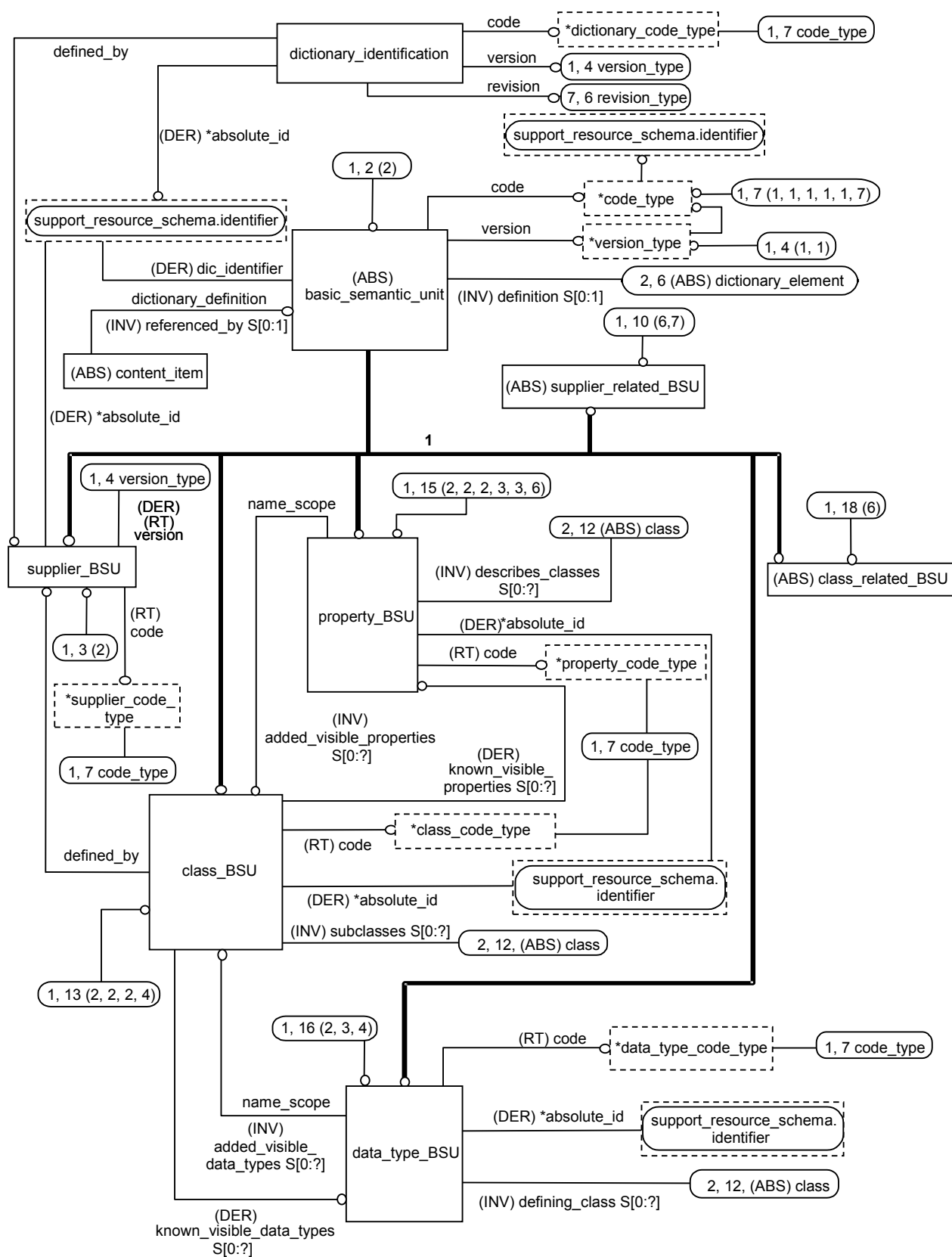


Figure B.1 – ISO13584_IEC61360_dictionary_schema – EXPRESS-G diagram 1 of 7

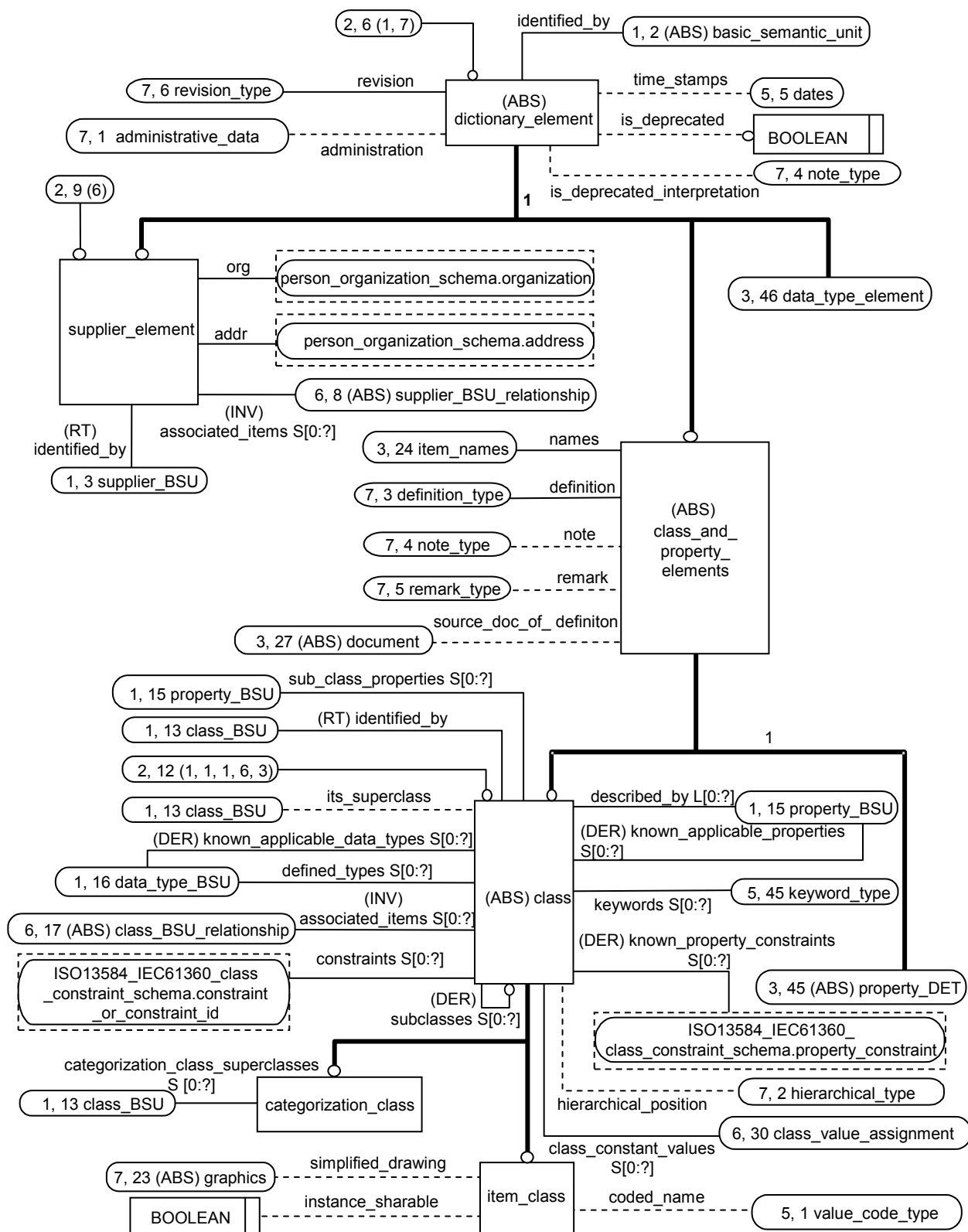


Figure B.2 – ISO13584_IEC61360_dictionary_schema – EXPRESS-G diagram 2 of 7

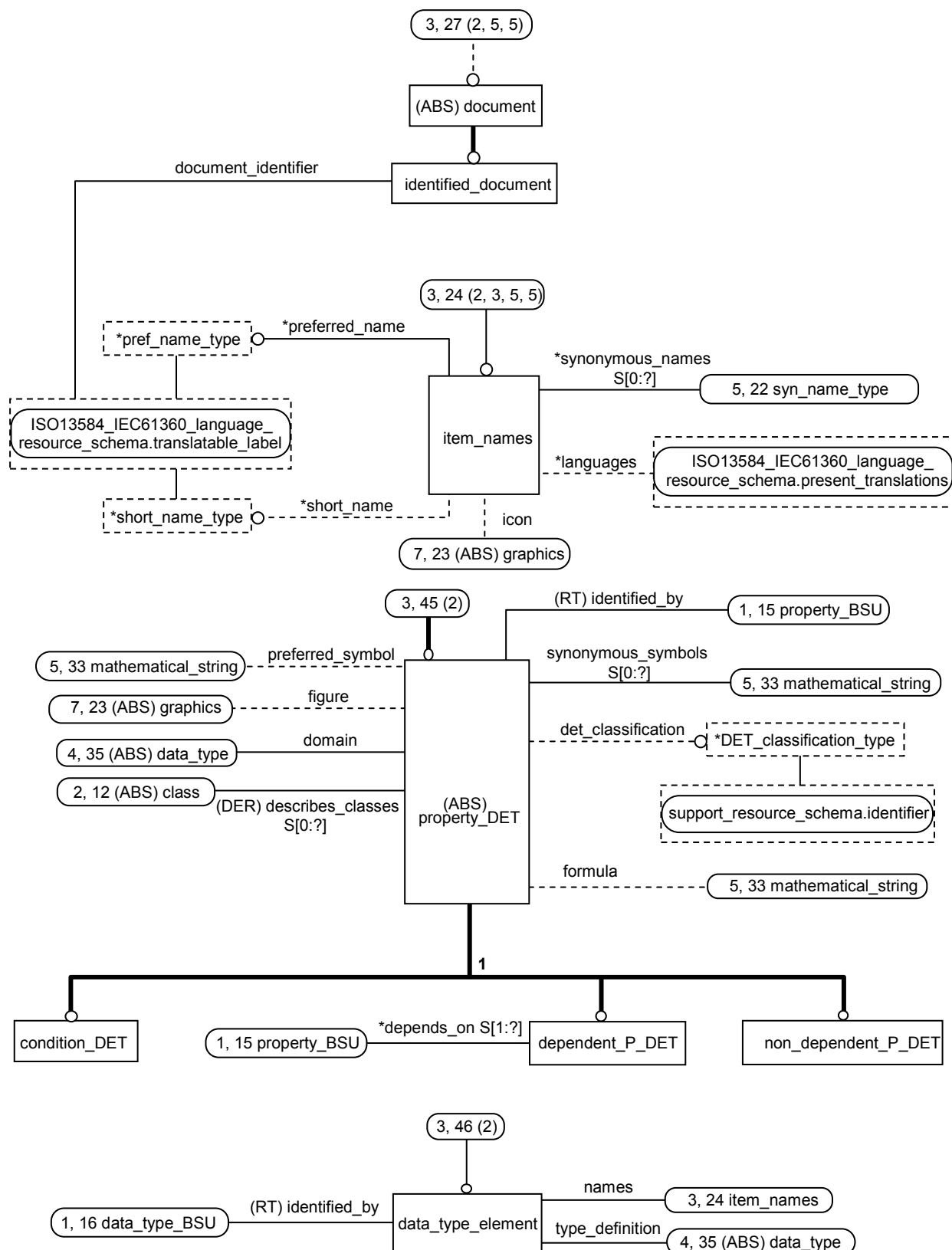


Figure B.3 – ISO13584_IEC61360_dictionary_schema – EXPRESS-G diagram 3 of 7



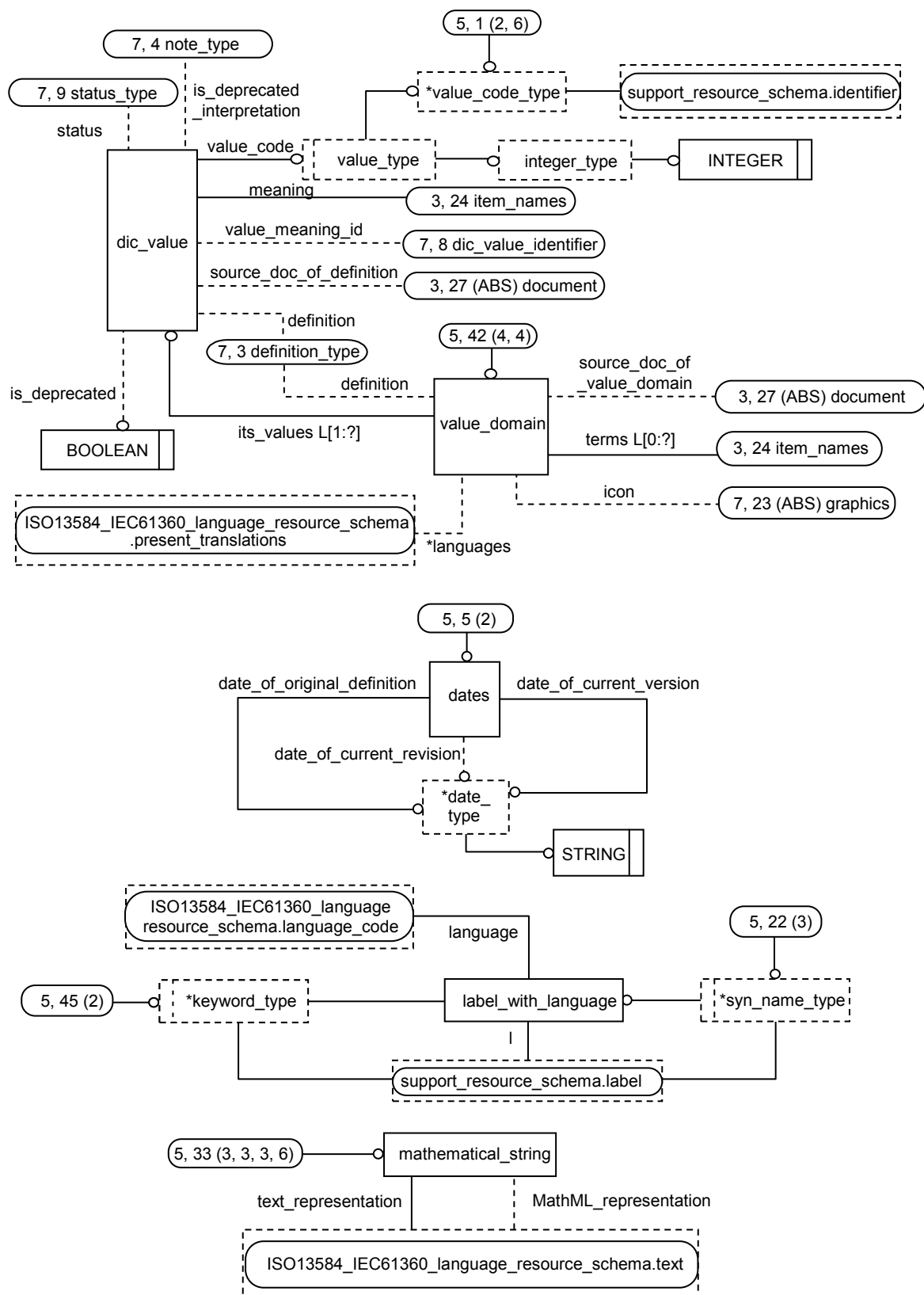


Figure B.5 – ISO13584_IEC61360_dictionary_schema – EXPRESS-G diagram 5 of 7

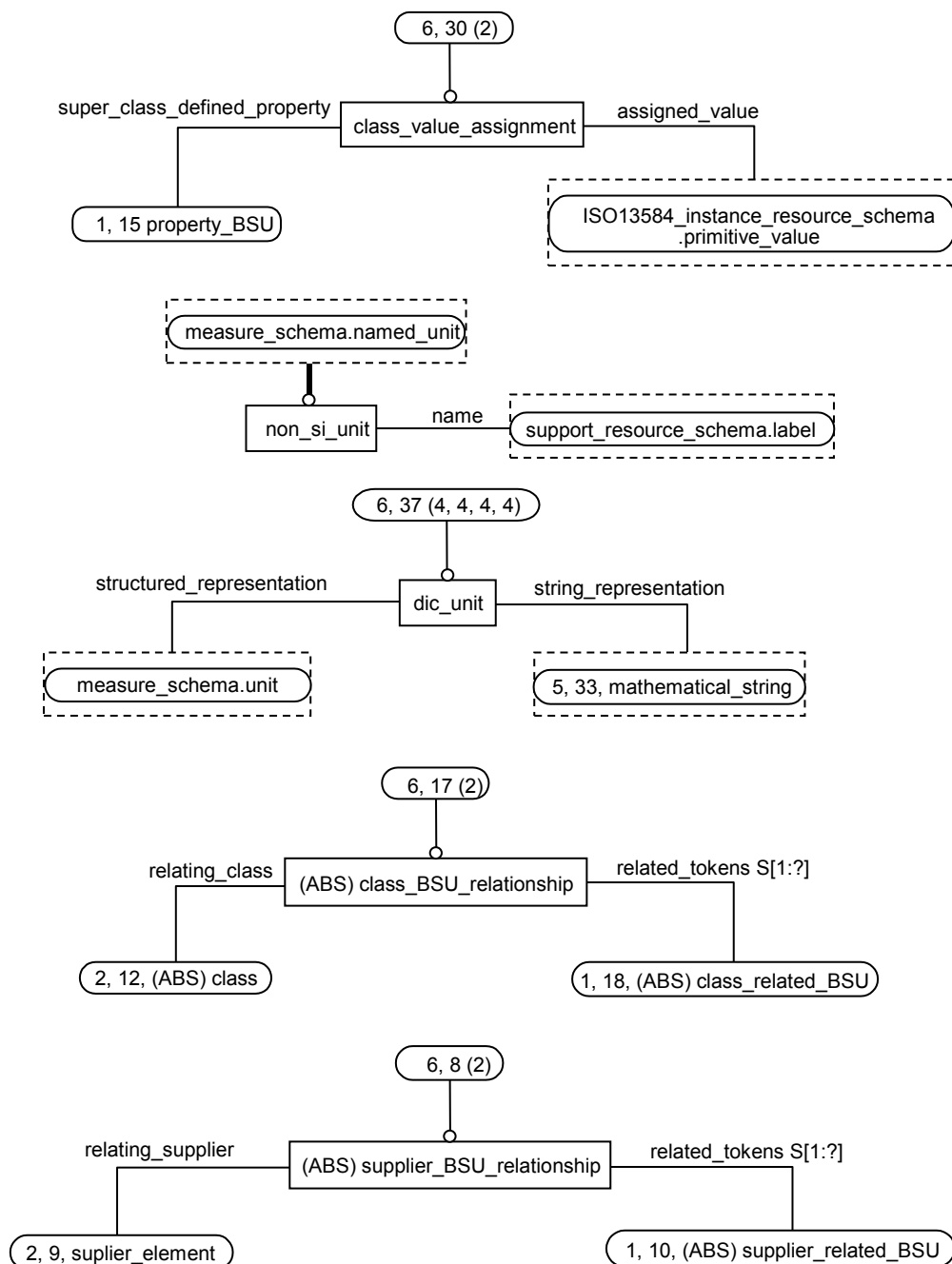


Figure B.6 – ISO13584_IEC61360_dictionary_schema – EXPRESS-G diagram 6 of 7

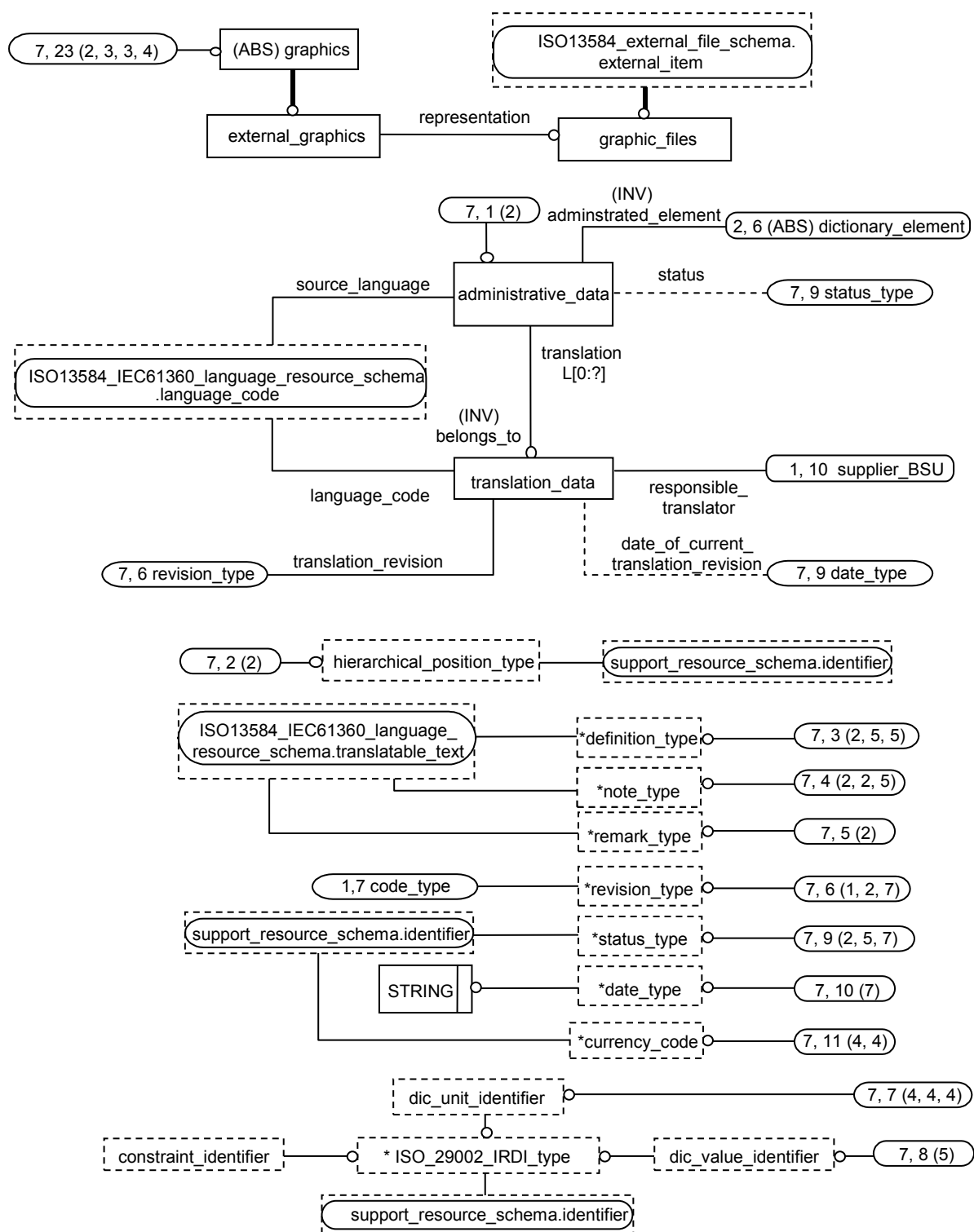


Figure B.7 – ISO13584_IEC61360_dictionary_schema – EXPRESS-G diagram 7 of 7

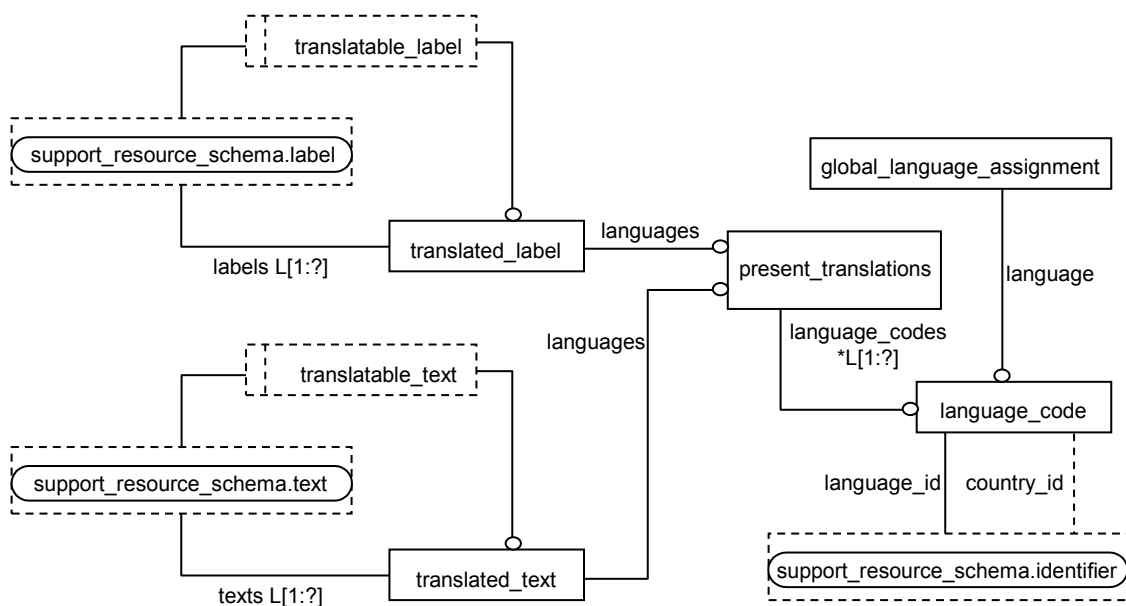


Figure B.8 – ISO13584_IEC61360_language_resource_schema –
EXPRESS-G diagram 1 of 1

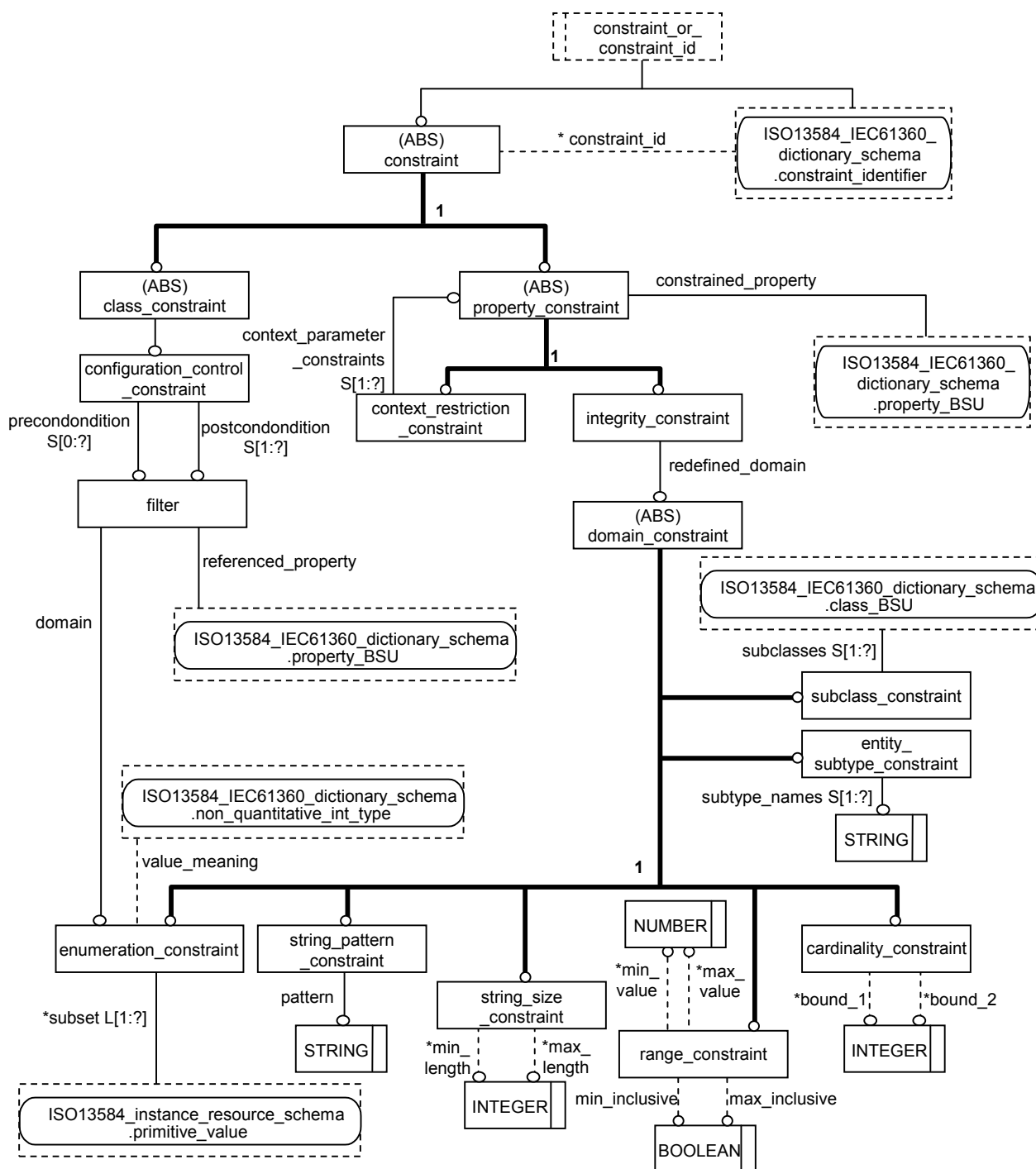


Figure B.9 – ISO13584_IEC61360_constraint_schema – EXPRESS-G diagram 1 of 1

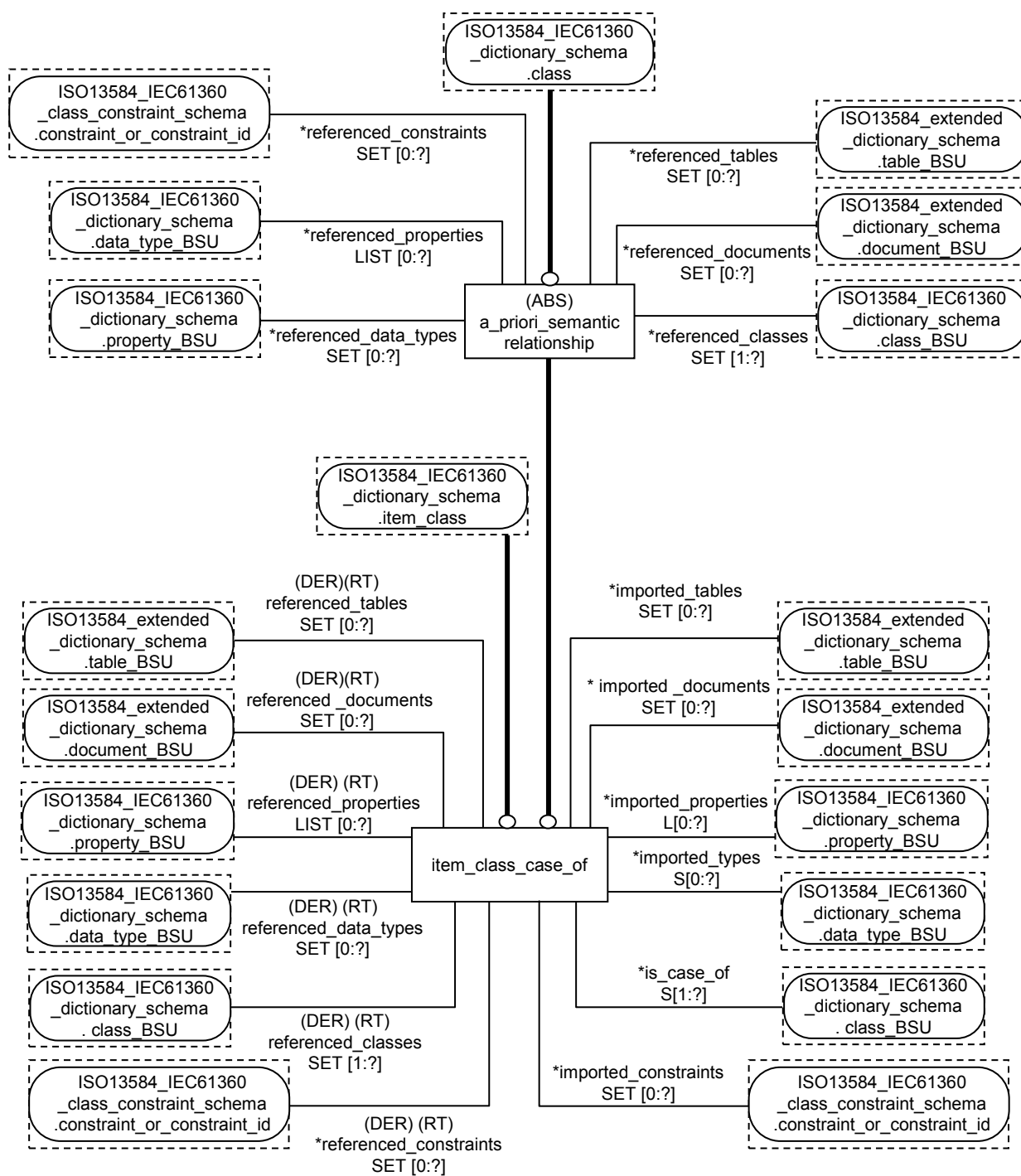


Figure B.10 – ISO13584_IEC61360_item_class_case_of_schema – EXPRESS-G diagram 1 of 1

Annex C (informative)

Partial dictionaries

The EXPRESS data model published in this part of IEC 61360 and duplicated for convenience in ISO 13584-42 allows to describe dictionaries composed of classes, properties and data types and it provides for their unique identification through the Basic Semantic Unit (BSU) mechanism. This data model allows to describe hierarchies of classes structured according to a tree structure using a simple inheritance mechanism. With this data model, only single and self-contained dictionaries were addressed.

The use of this data model leads to several different dictionaries. During a dictionary building process it may happen that designers require to reference a particular class, property or data type already defined in another dictionary. The possibility to import properties and data types in the dictionary under design is offered by the case-of relationship that may be used with the complete common ISO13584/IEC61360 dictionary model, documented both in ISO 13584-25 and in IEC 61360-5. Indeed, this relationship allows to import externally defined properties and data types providing the possibility to support partial dictionaries and to avoid duplication between dictionaries, each dictionary defining its own class structure.

In the complete common ISO13584/IEC61360 dictionary model, two mechanisms are offered:

- the **a_priori_case_of_semantic_relationship** EXPRESS entity allows to directly use properties or data types defined in external dictionary or dictionaries without describing them again;
- the **a_posteriori_case_of_relationship** EXPRESS entity allows, once properties or data types have been already defined in a dictionary under design, to map them onto corresponding properties or data types defined in an external dictionary.

These mechanisms allow to design dictionaries that refers to data elements that are defined in other dictionaries without affecting their semantic meaning. Moreover, the case-of relationships are recorded in the dictionaries that use this capability, providing for automatic integration of dictionaries based on the same standard dictionaries.

This mechanism is also recommended when designing an end-user dictionary. As a rule, an end-user dictionary does not need the whole class structure defined in standard dictionaries, while wishing to be able to exchange information with other users whose dictionaries are based on the same standard dictionary or dictionaries. If the end-user defines its own hierarchy but maps each of his/her classes through case-of onto the corresponding standard class, and if he/she imports all the existing standard properties that prove useful in his/her context, while adding user-specific properties, the user may customise its own dictionary while being able to exchange standard information with other users. This approach for designing end-user dictionaries is the approach recommended in this standard.

Annex D (normative)

Value format specification

D.1 General

This part of IEC 61360 and ISO 13584-42 provide a particular syntax to specify the allowed formats for the string and numeric values that may be associated with a property.

EXAMPLE 1 The format NR1 3 allows to specify that only integer values consisting of exactly three digits are allowed.

NOTE 1 No value format is defined for any other **data_type**, including **boolean_type**.

NOTE 2 In this part of IEC 61360, to define the format of property values is not mandatory.

The syntax of the allowed formats is defined in this Annex using a subset of the Extended Backus-Naur Form (EBNF) defined in ISO/IEC 14977.

EXAMPLE 2 The syntax of the format NR1 3 are the letters 'NR1' ' ' '3'.

The meaning of each syntax, that is the characters that may be used to represent a value, cannot be defined using the EBNF. Thus the meaning of each part of the format concerning the characters allowed to represent the value is specified separately for each part of the format.

EXAMPLE 3 The syntax of the format NR1 3 has the following meaning: *NR1* means that only an integer value may be represented. Space means that a fixed number of characters is specified by the format. 3 means that exactly three digits are required.

D.2 Notation

Table D.1 summarizes the subset of the ISO/IEC 14977 EBNF syntactic metalanguage used by this part of IEC 61360 to specify value format of properties.

Using these notations, the syntax of the subset of the EBNF metalanguage used by this part of IEC 61360 to specify value format of properties is summarized by the following grammar (the meta-identifier character, letter and digit are not detailed):

```
syntax = syntaxrule, { syntaxrule };
syntaxrule = metaidentifier, '=', definitionslist, ';';
definitionslist = singledefinition, { '|', singledefinition };
singledefinition = term, { ',', term };
term = primary, [ '-', primary ];
primary = optionalsequence | repeatedsequence | groupedsequence |
        metaidentifier | terminal | empty;
optionalsequence = '[' definitionslist ']';
repeatedsequence = '{' definitionslist '}';
groupedsequence = '(' definitionslist ')';
metaidentifier = letter, { letter };
terminal = '"', (character - '"'), {character - '"'}, "'"
        | "'", (character - "'"), {character - "'"}, '"';
empty =;
```

The equal sign '=' indicates a syntax rule. The meta-identifier on the left may be re-written by the combination of the elements on the right. Any spaces appearing between the elements are meaningless unless they appear within a `terminal`. A syntax rule is terminated by a semicolon ';'.
 Copyright © IEC. Not for commercial use or reproduction

Table D.1 – ISO/IEC 14977 EBNF syntactic metalanguage

Representation	ISO/IEC 10646-1 Character names	Metalanguage symbol and role
' '	apostrophe	First quote symbol: represents language terminals. Terminal shall not contain apostrophe. Example: 'Hello'
" "	quotation mark	Second quote symbol: represents language terminals. Terminal shall not contain quotation mark. Example: "John's car"
()	left parenthesis, right parenthesis	Start / end group symbols. The content is considered as a single symbol.
[]	left square bracket, right square bracket	Start / end option symbols. The content may or not be present.
{ }	left curly bracket, right curly bracket	Start/ end repeat symbols. The content may be present 0 to n times.
-	hyphen-minus	Except symbol.
,	comma	concatenate symbol.
=	equals sign	Defining symbol. Syntax rule: defines the symbol of the left by the formula on the right.
	vertical line	Alternative separator symbol.
;	semicolon	Terminator symbol. End of a syntax rule.

The use of a meta-identifier within a definition-list denotes a non-terminal symbol which appears on the left side of another syntax rule. A meta-identifier is composed of letters or digits, the first character being a letter. If a term contains both a `primary` preceding a minus sign, and a `primary` that follows the minus sign, only the sequence of symbols that are represented by the first `primary` and that are not represented by the second `primary` are represented by the term.

EXAMPLE 1 Notation:

''', character – ''', '''

means any character but the apostrophe character, inserted between two apostrophe characters.

The `terminal` denotes a symbol which cannot be expanded further by a syntax rule, and which will appear in the final result. Two ways are allowed to represent a `terminal`: either a set of characters without apostrophe, inserted between two apostrophes, or a set of characters without quotation marks, inserted between two quotation marks.

EXAMPLE 2 Assume that we want to describe, by such a grammar, the price of a product in €. Such a price is a positive number with no more than 2 digits in the cents part. We introduce three meta-identifiers associated with three syntax rules:

```
digit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9';
cents = [ '.', digit [, digit ] ];
euros = digit {, digit } cents;
```

With these syntax rules: 012, 4323.3, 3.56 are examples of licit representations of Euros. 12.,.10 are examples of non licit representation of Euros.

D.3 Data value format types

The grammar defined in this annex defines eight different types of value formats: four quantitative and five non-quantitative value formats.

In the next clause, we define the meta-identifiers that are used to specify these formats. In Clause D.5 we define the syntax rule for the four meta-identifiers that represent the four quantitative value formats, together with their meaning at the value level. In Clause D.6 we define the meta-identifiers for the five non-quantitative value formats, together with their meaning at the value level.

D.4 Meta-identifier used to define the formats

The meta-identifiers used in the grammar that define the various value formats are the following:

dot = '.';

decimalMark = '.';

exponentIndicator = 'E';

numeratorIndicator = 'N';

denominatorIndicator = 'D';

leadingDigit = '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9';

lengthOfExponent = leadingDigit, {trailingDigit};

lengthOfIntegerPart = (leadingDigit, {trailingDigit});

lengthOfNumerator = leadingDigit, {trailingDigit};

lengthOfDenominator = leadingDigit, {trailingDigit};

lengthOfFractionalPart = (leadingDigit, {trailingDigit}) | '0';

lengthOfIntegralPart = (leadingDigit, {trailingDigit}) | '0';

lengthOfNumber = leadingDigit, {trailingDigit};

trailingDigit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9';

signedExponent = 'S';

signedNumber = space, 'S';

space = ' ';

variableLengthIndicator = '..';

decimalMark: separator between integral and fractional part of numbers of format NR2 or NR3.

leadingDigit: first cipher of a number comprising one or more ciphers.

trailingDigit: one of the ciphers that combines to form numbers, except the first one.

NOTE If a number comprises only one digit, no `trailingDigit` is present.

D.5 Quantitative value formats

D.5.1 General

The four quantitative value format syntax rules and their meanings for value representation are defined in the following four subclauses. They are allowed for use for properties having the following data types:

- **number_type** or any of its subtype;
- **level_type** whose **value_type** are either **real_measure_type** or **int_measure_type**;
- **list_type**, **set_type**, **bag_type**, **array_type** or **set_with_subset_constraint_type** whose **value_type** are **number_type** or any of its subtype.

NOTE 1 **list_type**, **set_type**, **bag_type**, **array_type** or **set_with_subset_constraint_type** are defined in ISO 13584-25.

NOTE 2 For **non_quantitative_int_type** the value format applies to the code.

NOTE 3 The value of this attribute should be compatible with the data type of the property: it should not change this data type, else it should be ignored.

EXAMPLE The value format NR2 is not compatible with **int_type**, since integer values should not have a fractional part.

D.5.2 NR1-value format

The NR1-value syntax specifies the format of an integer property value.

Syntax rule:

```
NR1Value = 'NR1', ((signedNumber, variableLengthIndicator) |
(signedNumber, space) | variableLengthIndicator | space),
lengthOfNumber;
```

The meaning of NR1-value format components for value representation is as follows:

- 'NR1 ': the value shall be an integer.

NOTE 1 NR1 number values should not contain any spaces.

- `lengthOfNumber`: number of digits of the value.

NOTE 2 If preceded by a `variableLengthIndicator` the actual number of digits may be less.

- `signedNumber`: if `signedNumber` is present, the related number shall have either a positive, negative, or zero value. In case of positive values a '+' sign may be present. Negative values shall be preceded by a '-' sign. The value zero shall not be preceded by a '-' sign.

- `variableLengthIndicator`: if `variableLengthIndicator` is present, the related number shall contain a number of digits that is less or equal to its length specification, i.e., to `lengthOfNumber`.

D.5.3 NR2-value format

The NR2-value syntax specifies the format of a real property value that does not need an exponent.

Syntax rule:

```
NR2Value = 'NR2', ((signedNumber, variableLengthIndicator) |
(signedNumber, space) | variableLengthIndicator | space),
lengthOfIntegralPart, decimalMark, lengthOfFractionalPart;
```

The meaning of NR2-value format components for value representation is as follows:

- 'NR2 ': the value shall be a real.

NOTE 1 NR2 number values should not contain any spaces.

- `lengthOfFractionalPart`: number of digits of the fractional part of the number.

NOTE 2 If preceded by a `variableLengthIndicator` the actual number of digits of the fractional part may be less.

NOTE 3 `lengthOfFractionalPart` implicitly specifies the recommended accuracy of the value. The actual accuracy of the number from which this value was derived may have been greater than the value expressed here.

- `lengthOfIntegralPart`: number of digits of the integral part of the number.

NOTE 4 If preceded by a `variableLengthIndicator` the actual number of digits of the integral part may be less.

- `signedNumber`: if `signedNumber` is present, the related number shall have either a positive, negative, or zero value. In case of positive values a '+' sign may be present. Negative values shall be preceded by a '-' sign. The value zero shall not be preceded by a '-' sign.
- `variableLengthIndicator`: if `variableLengthIndicator` is present, either integral part or fractional part of the number or both parts shall contain a number of digits that is less or equal to its related length specification, i.e., to `lengthOfIntegralPart` or `lengthOfFractionalPart`. At least one cipher shall be present in the number.

D.5.4 NR3-value format

The NR3-value syntax specifies the format of a real property value that is represented with an exponent.

Syntax rule:

```
NR3Value = 'NR3', ((signedNumber, variableLengthIndicator) |
(signedNumber, space) | variableLengthIndicator | space),
lengthOfIntegralPart, decimalMark, lengthOfFractionalPart,
exponentIndicator, [signedExponent], lengthOfExponent;
```

The meaning of NR3-value format components for value representation is as follows:

- 'NR3 ': the value shall be a real with an exponent of base 10.

NOTE 1 There should be at least one digit and the decimal mark in the mantissa. The exponent shall contain at least one digit, too.

NOTE 2 NR3 number values shall not contain any spaces.

- `exponentIndicator`: separator between mantissa and exponent in numbers of format NR3.
- `lengthOfExponent`: number of digits of the exponent.

NOTE 3 If preceded by a `variableLengthIndicator` the actual number of digits of the exponent may be less.

NOTE 4 Eventually existing signs or a decimal mark are not counted by `lengthOfNumber`, `lengthOfIntegralPart`, `lengthOfFractionalPart` or `lengthOfExponent`.

- `lengthOfFractionalPart`: number of digits of the fractional part of the mantissa.

NOTE 5 If preceded by a `variableLengthIndicator` the actual number of digits of the fractional part may be less.

NOTE 6 `lengthOfFractionalPart` implicitly specifies the recommended accuracy of the value. The actual accuracy of the number from which this value was derived may have been greater than the value expressed here.

- `lengthOfIntegralPart`: number of digits of the integral part of the mantissa.

NOTE 7 If preceded by a `variableLengthIndicator` the actual number of digits of the integral part may be less.

- `signedExponent`: if `signedExponent` is present, the related exponent shall have either a positive, negative, or zero value. In case of positive values a '+' sign may be present. Negative values shall be preceded by a '-' sign. The value zero shall not be preceded by a '-' sign.
- `variableLengthIndicator`: if `variableLengthIndicator` is present, the related number or exponent shall contain a number of digits that is less or equal to its related length specification, i.e., to `lengthOfIntegralPart`, `lengthOfFractionalPart`, or `lengthOfExponent`. At least one cipher shall be present in the mantissa and in the exponent.

D.5.5 NR4-value format

The NR4-value syntax specifies the format of a rational property value that is represented with an integer part, and possibly a fraction part with a denominator and a numerator.

Syntax rule:

NR4Value = 'NR4', ((signedNumber, variableLengthIndicator) | (signedNumber, space) | variableLengthIndicator | space), lengthOfIntegerPart, numeratorIndicator, lengthOfNumerator, denominatorIndicator, lengthOfDenominator;

The meaning of NR4-value format components for value representation is as follows:

- 'NR4': the value shall be a rational number represented either as an integer, or as a fraction consisting of a numerator and a denominator, or as an integer and a fraction.

EXAMPLE 12 ½ and 12 ¾ are values that may be represented in the NR4 format.

NOTE 1 There shall be at least one digit either in the integer part, or both in the numerator and in the denominator part. If one part of the fraction contains a digit, the other part shall also contain some digits. All three parts may also contain digits.

NOTE 2 NR4 number values shall not contain any spaces.

- `numeratorIndicator`: separator between the integer part description and the fraction part description in formats NR4.
- `lengthOfNumerator`: number of digits of the numerator.

NOTE 3 If preceded by a `variableLengthIndicator` the actual number of digits of the numerator may be less.

NOTE 4 If the value of the rational number is completely represented by its integer part, neither the numerator of the fraction nor its denominator shall be represented.

- `denominatorIndicator`: separator between the numerator part description and the denominator part description in formats NR4.
- `lengthOfDenominator`: number of digits of the denominator.

NOTE 5 If preceded by a `variableLengthIndicator` the actual number of digits of the denominator may be less.

NOTE 6 If the value of the rational number is completely represented by its integer part, neither the numerator of the fraction nor its denominator shall be represented.

- `lengthOfIntegerPart`: number of digits of the integer part of the rational number.

NOTE 7 If preceded by a `variableLengthIndicator` the actual number of digits of the integer part may be less.

- `variableLengthIndicator`: if `variableLengthIndicator` is present, the three parts of the rational number shall contain a number of digits that is less or equal to its related length specification, i.e., to `lengthOfIntegralPart`, `lengthOfNumerator`, or `lengthOfDenominator`. At least one cipher shall be present either in the integral part, or in the two parts of the fraction.

D.6 Non-quantitative value formats

D.6.1 General

The five non-quantitative value format syntax rules and their meanings are defined in the following five sub-clauses. They are allowed for use for properties having the following data types:

- **string_type** or any of its subtype;
- **list_type**, **set_type**, **bag_type**, **array_type** or **set_with_subset_constraint_type** whose **value_type** are **string_type** or any of its subtype.

NOTE 1 **list_type**, **set_type**, **bag_type**, **array_type** or **set_with_subset_constraint_type** are defined in ISO 13584-25.

NOTE 2 For **translatable_string_type** the value format applies to any language-specific representation of the string.

NOTE 3 For **non_quantitative_code_type**, the value format applies to the code.

Non-quantitative values are represented by strings which comprise characters. The length of a string may be either specified by directly specifying the upper limit of the number of contained characters or by specifying that the total number of characters may be any integral multiple of the length specified.

Syntax rule:

`factor` = `leadingDigit`, {`trailingDigit`};

`numberOfCharacters` = (`leadingDigit`, {`trailingDigit`})[(`'(nx'`, `factor`,`')`);

The meaning of the factor components is as follows

- **factor**: when **factor** is present, then **numberOfCharacters** shall be any integral multiple of the value given in **factor**. **factor** shall not contain the value zero.
- **numberOfCharacters**: determines the maximum amount of characters contained in the string.

D.6.2 Alphabetic Value Format

An “Alphabetic Value Format (A)” defines the value format of a string that contains alphabetic letters. Thus, the content shall be taken from the characters of row 00, cell 20, cell 40 to 7E, or cell C0 to FF, of the Basic Multilingual Plane (BMP) (Plane 00 of Group 00) of ISO/IEC 10646-1.

NOTE 4 Due to potential interpretation problems of value content within components of one system or of multiple systems, it is recommended that, where possible, the characters used should be restricted to the G0 set of ISO/IEC 10646-1 and/or row 00 columns 002 to 007 of ISO/IEC 10646-1.

NOTE 5 For alternative languages, as supported by translated data, the relevant characters or ideographs of the related language specific character set are available as defined by the Unicode Standard. In most cases, there will be no 1:1 relation between the characters of the source language to the characters of the target language.

EXAMPLE CJK (Chinese-Japanese-Korean) ideographs.

Syntax rule:

AValue = 'A', (space | variableLengthIndicator), numberOfCharacters;

The meaning of A-value format components for value representation is as follows:

- 'A': the value shall be a string, or several substrings, of alphabetic letters.
- variableLengthIndicator: if variableLengthIndicator is present, the string may contain fewer characters than indicated by numberOfCharacters. The string shall contain at least one character.

D.6.3 Mixed characters value format

A “Mixed value format (M)” format defines the value format of a string that may contain any character specified in Clause D.7.

NOTE For alternative languages as supported by translated data, the relevant characters or ideographs of the related language specific character set are available as defined by the Unicode Standard.

EXAMPLE CJK (Chinese-Japanese-Korean) characters.

Syntax rule:

MValue = 'M', (space | variableLengthIndicator), numberOfCharacters;

The meaning of M-value format components for value representation is as follows:

- 'M': the value shall be a string, or several substrings.
- variableLengthIndicator: if variableLengthIndicator is present, the string may contain fewer characters than indicated by numberOfCharacters. The string shall contain at least one character.

D.6.4 Number value format

A “Number value format (N)” defines the value format of a string that contains numeric digits only. Thus, the content shall be taken from the characters of row 00, cell 2B, cell 2D, cell 30 to 39, or cell 45 of the Basic Multilingual Plane (BMP) (Plane 00 of Group 00) of ISO/IEC 10646-1.

NOTE For alternative languages as supported by translated data, the relevant characters or ideographs of the related language specific character set are available as defined by the Unicode Standard.

EXAMPLE Table D.2 shows the transposition of the European digits “0” to “9” into Arabic digits.

Table D.2 – Transposing European style digits into Arabic digits

European digits	9	8	7	6	5	4	3	2	1	0
Arabic digits	٩	٨	٧	٦	٥	٤	٣	٢	١	٠

Syntax rule:

NValue = 'N', (space | variableLengthIndicator), numberOfCharacters;

The meaning of N-value format components for value representation is as follows:

- 'N': the value shall be a string, or several substrings, of numeric digits.
- variableLengthIndicator: if variableLengthIndicator is present, the string may contain fewer characters than indicated by numberOfCharacters. The string shall contain at least one character.

D.6.5 Mixed alphabetic or numeric characters value format

A “Mixed alphabetic or numeric characters value format (X)” defines the value format of a string that contains alphanumeric characters, i.e., any combination of characters from A-value format or N-value format.

NOTE For alternative languages as supported by translated data, the relevant characters or ideographs of the related language specific character set are available as defined by the Unicode Standard.

Syntax rule:

XValue = 'X', (space | variableLengthIndicator), numberOfCharacters;

The meaning of X-value format components for value representation is as follows:

- 'X': the value shall be a string, or several substrings, of alphanumeric, i.e., any combination of alphabetic and numeric characters;
- variableLengthIndicator: if variableLengthIndicator is present, the string may contain fewer characters than indicated by numberOfCharacters. The string shall contain at least one character.

D.6.6 Binary value format

A “Binary value format (B)” defines the value format of a string that contains binary characters, i.e., “0” or “1”. Thus the content shall be taken from the characters of row 00, cell 30 or 31, of the Basic Multilingual Plane (BMP) (Plane 00 of Group 00) of ISO/IEC 10646-1.

NOTE For alternative languages as supported by translated data, the relevant characters or ideographs of the related language specific character set are available as defined by the Unicode Standard.

Syntax rule:

BValue = 'B', (space | variableLengthIndicator), numberOfCharacters;

The meaning of B-value format components for value representation is as follows:

- 'B': the value shall be a string, or several substrings, consisting of binary values, i.e., the characters “0” or “1” or sequences thereof.
- variableLengthIndicator: if variableLengthIndicator is present, the string may contain fewer characters than indicated by numberOfCharacters. The string shall contain at least one character.

D.7 Value examples

Table D.3 below contains some examples for values that may be contained in numbers and strings characterized by the above value format scheme.

Table D.3 – Number value examples

Value format	Examples of possible values
NR1 3	123; 001; 000;
NR1..3	123; 87; 5
NR1S 3	+123; +000;
NR1S..3	-123; +1; 0; -12;
NR2 3.3	123.300; 000.400; 000.420;
NR2..3.3	321.233; 1.234; 23.56; 9.783; .72; 324.
NR2S 3.3	-123.123; +123.300;
NR2S..3.3	-123.123; +12.3; 0.1; +.4; -3.; 0.; .0;
NR3 3.3E4	123.123E0004; 003.000E1000;
NR3 3.3ES4	123.123E+0004; 123.123E0004; 123.000E-0001
NR3..3.3E4	123.123E0004; .123E0001; 5.E1234
NR3S 3.3ES4	+123.123E+0004; 123.000E-0001;
NR3S..3.3ES4	-123.123E+0004; +1.00E-01;.0E0; +3.E-1; -.2E-1000;
NR4 3N2D2	001 02/03; 012 00/01; 123 03/04; 000 01/04
NR4..3N2D2	1 ½; 12; 123 ¾; ¼;
NR4S 3N2D2	+001 02/03; -012 00/01; 123 03/04; -000 01/04
NR4S..3N2D2	-1 ½; 12; +123 ¾; ¼;
A 19	My name is Reinhard, abcdefghijklmnopqrs
A..3	Abc; de; G
X..5	A23RN1; B1; ca
M..10	A23RN1; B1; ca. 256 µm;
N (nx5)	12345; 1234512345; 222223333344444;
N..(nx5)	1234; 12345; 34512345; 1234512345; 23333344444; 222223333344444; -3; 5E2;
B 1	0; 1;
B 3	011; 101;

Characters from ISO/IEC 10646-1

The following characters shall be used for the purpose of Mixed value format (M) (see D.5.3):

- all characters from row 00 of the Basic Multilingual Plane (BMP) (Plane 00 of Group 00) of ISO/IEC 10646-1;
- characters from other rows of the Basic Multilingual Plane of ISO/IEC 10646-1 as listed in Table D.4;
- for alternative languages as supported by translated data, the full character set is available as defined by the Unicode Standard.

NOTE 1 Due to potential interpretation problems of value content within components of one system or of multiple systems it is recommended that, where possible, the characters used should be restricted to the G0 set of ISO/IEC 10646-1 and/or row 00 columns 002 to 007 of ISO/IEC 10646-1.

NOTE 2 The Unicode Standard is published by The Unicode Consortium, P.O. Box 391476, Mountain View, CA 94039-1476, U.S.A., www.unicode.org.

Table D.4 – Characters from other rows of the Basic Multilingual Plane of ISO/IEC 10646-1

Character	Name	Row	Cell
	CARON	02	C7
≡	IDENTICAL TO	22	61
∧	LOGICAL AND	22	27
∨	LOGICAL OR	22	28
∩	INTERSECTION	22	29
∪	UNION	22	2A
⊂	SUBSET OF (IS CONTAINED)	22	82
⊃	SUBSET OF (CONTAINS)	22	83
⇐	LEFTWARDS DOUBLE ARROW (IS IMPLIED BY)	21	D0
⇒	RIGHTWARDS DOUBLE ARROW (IMPLIES)	21	D2
∴	THEREFORE	22	34
∵	BECAUSE	22	35
∈	ELEMENT OF	22	08
⊋	CONTAINS AS MEMBER (HAS AS AN ELEMENT)	22	0B
⊆	SUBSET OR EQUAL TO (CONTAINED AS SUB-CLASS)	22	86
⊇	SUPERSET OR EQUAL TO (CONTAINS AS SUB-CLASS)	22	87
∫	INTEGRAL	22	2B
∮	CONTOUR INTEGRAL	22	2E
∞	INFINITY	22	1E

Table D.4 (continued)

Character	Name	Row	Cell
∇	NABLA	22	07
∂	PARTIAL DIFFERENTIAL	22	02
$\sim$	TILDE OPERATOR (DIFFERENCE BETWEEN)	22	3C
$\approx$	ALMOST EQUAL TO	22	48
$\asymp$	ASYMPTOTICALLY EQUAL TO	22	43
$\approx$	APPROXIMATELY EQUAL TO (SIMILAR TO)	22	45
$\leq$	LESS THAN OR EQUAL TO	22	64
$\neq$	NOT EQUAL TO	22	60
$\geq$	GREATER THAN OR EQUAL TO	22	65
$\Leftrightarrow$	LEFT RIGHT DOUBLE ARROW (IF AND ONLY IF)	21	D4
$\neg$	NOT SIGN	00	AC
$\forall$	FOR ALL	22	00
$\exists$	THERE EXISTS	22	03
$\aleph$	HEBREW LETTER ALEF	05	D0
$\square$	WHITE SQUARE (D'ALEMBERTIAN OPERATOR)	25	A1
$\parallel$	PARALLEL TO	22	25
Γ	GREEK CAPITAL LETTER GAMMA	03	93
Δ	GREEK CAPITAL LETTER DELTA	03	94
$\perp$	UPTACK (ORTHOGONAL TO)	22	A5
$\angle$	ANGLE	22	20

Table D.4 (continued)

Character	Name	Row	Cell
⊥	RIGHT ANGLE WITH ARC	22	BE
Θ	GREEK CAPITAL LETTER THETA	03	98
⟨	LEFT POINTING ANGLE BRACKET (BRA)	23	29
⟩	RIGHT POINTING ANGLE BRACKET (KET)	23	2A
Λ	GREEK CAPITAL LETTER LAMBDA	03	9B
'	PRIME	20	32
''	DOUBLE PRIME	20	33
Ξ	GREEK CAPITAL LETTER XI	03	9E
±	MINUS –OR– PLUS SIGN	22	13
Π	GREEK CAPITAL LETTER PI	03	A0
²	SUPERSCRIPIT TWO	00	B2
Σ	GREEK CAPITAL LETTER SIGMA	03	A3
×	MULTIPLICATION SIGN	00	D7
³	SUPERSCRIPIT THREE	00	B3
Υ	GREEK CAPITAL LETTER UPSILON	03	A5
Φ	GREEK CAPITAL LETTER PHI	03	A6
.	MIDDLE DOT	00	B7
Ψ	GREEK CAPITAL LETTER PSI	03	A8
Ω	GREEK CAPITAL LETTER OMEGA	03	A9
∅	EMPTY SET	22	05

Table D.4 (continued)

Character	Name	Row	Cell
↗	RIGHTWARDS HARPOON WITH BARB UPWARDS (VECTOR OVERBAR)	21	C0
√	SQUARE ROOT (RADICAL)	22	1A
f	LATIN SMALL LETTER F WITH HOOK (FUNCTION OF)	01	92
∝	PROPORTIONAL TO	22	1D
±	PLUS – MINUS SIGN	00	B1
°	DEGREE SIGN	00	B0
α	GREEK SMALL LETTER ALPHA	03	B1
β	GREEK SMALL LETTER BETA	03	B2
γ	GREEK SMALL LETTER GAMMA	03	B3
δ	GREEK SMALL LETTER DELTA	03	B4
ε	GREEK SMALL LETTER EPSILON	03	B5
ζ	GREEK SMALL LETTER ZETA	03	B6
η	GREEK SMALL LETTER ETA	03	B7
θ	GREEK SMALL LETTER THETA	03	B8
ι	GREEK SMALL LETTER IOTA	03	B9
κ	GREEK SMALL LETTER KAPPA	03	BA
λ	GREEK SMALL LETTER LAMBDA	03	BB
μ	GREEK SMALL LETTER MU	03	BC
ν	GREEK SMALL LETTER NU	03	BD
ξ	GREEK SMALL LETTER XI	03	BE

Table D.4 (continued)

Character	Name	Row	Cell
‰	PER MILLE SIGN	20	30
π	GREEK SMALL LETTER PI	03	C0
ρ	GREEK SMALL LETTER RHO	03	C1
σ	GREEK SMALL LETTER SIGMA	03	C3
÷	DIVISION SIGN	03	F7
τ	GREEK SMALL LETTER TAU	03	C4
υ	GREEK SMALL LETTER UPSILON	03	C5
φ	GREEK SMALL LETTER PHI	03	C6
χ	GREEK SMALL LETTER CHI	03	C7
ψ	GREEK SMALL LETTER PSI	03	C8
ω	GREEK SMALL LETTER OMEGA	03	C9
†	DAGGER	20	20
←	LEFTWARDS ARROW	21	90
↑	UPWARDS ARROW	21	91
→	RIGHTWARDS ARROW	21	92
↓	DOWNWARDS ARROW	21	93
–	OVERLINE	20	3E

Bibliography

- [1] IEC 60027 (all parts), *Letter symbols to be used in electrical technology*
- [2] IEC 60748 (all parts), *Semiconductor devices – Integrated circuits*
- [3] IEC 61360-5, *Standard data element types with associated classification scheme for electric components – Part 5: Extensions to the EXPRESS dictionary schema*
- [4] IEC 80000 (all parts)³, *Quantities and units*
- [5] ISO/IEC 8824-1, *Information technology – Abstract Syntax Notation One (ASN.1) – Part 1: Specification of basic notation*
- [6] ISO/IEC 11179-5, *Information technology – Metadata registries (MDR) – Part 5: Naming and identification principles*
- [7] ISO/IEC, *International Classification of Standard (ICS)*
- [8] ISO 31 (all parts), *Quantities and units*
- [9] ISO 639-1, *Codes for the representation of names of languages – Part 1: Alpha-2 code*
- [10] ISO 639-2, *Codes for the representation of names of languages – Part 2: Alpha-3 code*
- [11] ISO 704, *Terminology work – Principles and methods*
- [12] ISO 843, *Information and documentation – Conversion of Greek characters into Latin characters*
- [13] ISO 1087-1, *Terminology work – Vocabulary – Part 1: Theory and application*
- [14] ISO 6523, *Data interchange – Structure for the identification of organizations*
- [15] ISO 10303-1:1994, *Industrial automation systems and integration – Product data representation and exchange – Part 1: Overview and fundamental principles*
- [16] ISO 10303-21, *Industrial automation systems and integration – Product data representation and exchange – Part 21: Implementation methods: Clear text encoding of the exchange structure*
- [17] ISO 10303-42:2000, *Industrial automation systems and integration – Product data representation and exchange – Part 42: Integrated generic resource: Geometric and topological representation²*
- [18] ISO 13584-1:2001, *Industrial automation systems and integration – Parts library – Part 1: Overview and fundamental principles*
- [19] ISO 13584-24:2003, *Industrial automation systems and integration – Parts library – Part 24: Logical resource: Logical model of supplier library*

² A new edition of ISO 10303-41 was published in 2005.

- [20] ISO 13584-25, *Industrial automation systems and integration – Parts library – Part 25: Logical resource: Logical model of supplier library with aggregate values and explicit content*
- [21] ISO 13584-32, *Industrial automation systems and integration – Parts library – Part 32: Implementation resources: OntoML: Product ontology markup language*
- [22] ISO 13584-511, *Industrial automation systems and integration – Parts library – Part 511: Mechanical systems and components for general use – Reference dictionary for fasteners*
- [23] ISO/TS 23768-1³, *Rolling bearings – Parts library – Part 1: Reference dictionary for rolling bearings*
- [34] ISO/TS 29002-5, *Industrial automation systems and integration – Exchange of characteristic data – Part 5: Identification scheme*
- [25] ISO/TS 29002-20, *Industrial automation systems and integration – Exchange of characteristic data – Part 20: Concept dictionary resolution services*
- [26] ISO 80000 (all parts)⁴, *Quantities and units*
- [27] XML Schema Part 2: Datatypes. Second Edition. World Wide Web Consortium Recommendation 28-October-2004
Available from <<http://www.w3.org/TR/2004/REC-xmlschema-2-20041028>>
- [28] Mathematical Markup Language (MathML). Second Edition. World Wide Web Consortium Recommendation 21-October-2003
Available from <<http://www.w3.org/TR/MathML2/>>

³ To be published.

⁴ The ISO 31 series of parts has been cancelled and replaced by the ISO 80000/IEC 80000 series of parts.

SOMMAIRE

AVANT-PROPOS	181
INTRODUCTION.....	183
1 Domaine d'application	184
2 Références normatives.....	185
3 Termes et définitions	186
4 Vue d'ensemble du schéma de dictionnaire commun et de la compatibilité avec l'ISO13584_IEC61360_dictionary_schema	195
4.1 Généralités.....	195
4.2 Utilisation du schéma de dictionnaire commun pour échanger des données conformes à la CEI 61360-1	195
4.3 Compatibilité avec l'ISO 13584-42.....	196
4.4 Correspondance de dénomination entre la CEI 61360-1 et la CEI 61360-2	196
4.5 Structure principale du schéma de dictionnaire commun	197
5 ISO13584_IEC61360_dictionary_schema	198
5.1 Généralités.....	198
5.2 Schéma de dictionnaire	198
5.3 Références à d'autres schémas conceptuels	198
5.4 Définitions de constantes	199
5.5 Identification d'un dictionnaire	199
5.6 Basic Semantic Units (Unités sémantiques de base): définition et utilisation du dictionnaire.....	201
5.6.1 Exigences pour l'échange	201
5.6.2 Architecture à trois niveaux des données du dictionnaire.....	201
5.6.3 Vue d'ensemble d'unités sémantiques de base et d'éléments de dictionnaire.....	206
5.6.4 Identification d'éléments de dictionnaire: structure à trois niveaux	206
5.6.5 Possibilités d'extension pour d'autres types de données	207
5.7 Données fournisseur	208
5.7.1 Généralités.....	208
5.7.2 Supplier_BSU	209
5.7.3 Supplier_element.....	209
5.8 Données de classe	210
5.8.1 Généralités.....	210
5.8.2 Détail de structure	212
5.8.3 Item_class	219
5.8.4 Categorization_class	220
5.9 Type de données d'un élément / propriétés des données	223
5.9.1 Généralités.....	223
5.9.2 Property_BSU	223
5.9.3 Property_DET.....	223
5.9.4 Types de données d'un élément, conditionnels, dépendants et indépendants.....	225
5.9.5 Détail de structure	225
5.9.6 Class_value_assignment	227
5.10 Données de domaine: le système de types.....	228
5.10.1 Généralités.....	228
5.10.2 Détail de structure	228

5.10.3	Le système de types.....	230
5.10.4	Valeurs.....	247
5.10.5	Détail de structure	248
5.10.6	Extension pour les définitions d'unités de l'ISO 10303-41	252
5.11	Types de base et définitions des entités	254
5.11.1	Définitions des types de base	254
5.11.2	Détail de structure	254
5.11.3	Définitions de base des entités	264
5.12	Définitions des fonctions	268
5.12.1	Généralités.....	268
5.12.2	Fonction <code>acyclic_superclass_relationship</code>	269
5.12.3	Fonction <code>check_syn_length</code>	269
5.12.4	Fonction <code>codes_are_unique</code>	270
5.12.5	Fonction <code>definition_available_implies</code>	270
5.12.6	Fonction <code>is_subclass</code>	271
5.12.7	Fonction <code>string_for_derived_unit</code>	271
5.12.8	Fonction <code>string_for_named_unit</code>	273
5.12.9	Fonction <code>string_for_SI_unit</code>	274
5.12.10	Fonction <code>string_for_unit</code>	275
5.12.11	Fonction <code>all_class_descriptions_reachable</code>	276
5.12.12	Fonction <code>compute_known_visible_properties</code>	276
5.12.13	Fonction <code>compute_known_visible_data_types</code>	277
5.12.14	Fonction <code>compute_known_applicable_properties</code>	278
5.12.15	Fonction <code>compute_known_applicable_data_types</code>	279
5.12.16	Fonction <code>list_to_set</code>	280
5.12.17	Fonction <code>check_properties_applicability</code>	280
5.12.18	Fonction <code>check_datatypes_applicability</code>	281
5.12.19	Fonction <code>one_language_per_translation</code>	281
5.12.20	Fonction <code>allowed_values_integer_types</code>	282
5.12.21	Fonction <code>is_class_valued_property</code>	282
5.12.22	Fonction <code>class_value_assigned</code>	283
6	ISO13584_IEC61360_language_resource_schema	284
6.1	Vue d'ensemble.....	284
6.2	Type ISO13584_IEC61360_language_resource_schema et définitions d'entités	285
6.2.1	Généralités.....	285
6.2.2	<code>Language_code</code>	285
6.2.3	<code>Global_language_assignment</code>	286
6.2.4	<code>Present_translations</code>	286
6.2.5	<code>Translatable_label</code>	286
6.2.6	<code>Translated_label</code>	287
6.2.7	<code>Translatable_text</code>	287
6.2.8	<code>Translated_text</code>	287
6.3	Définitions des fonctions de l'ISO13584_IEC61360_language_resource_schema.....	288
6.3.1	Généralités.....	288
6.3.2	Fonction <code>check_label_length</code>	288
6.4	Définition des règles de l'ISO13584_IEC61360_language_resource_schema	289
7	ISO13584_IEC61360_class_constraint_schema	289

7.1	Généralités.....	289
7.2	Introduction à l'ISO13584_IEC61360_class_constraint_schema type	290
7.3	Définitions d'entités de l'ISO13584_IEC61360_class_constraint_schema	291
7.3.1	Généralités.....	291
7.3.2	Constraint.....	291
7.3.3	Property_constraint	291
7.3.4	Class_constraint.....	292
7.3.5	Configuration_control_constraint	292
7.3.6	Filter.....	293
7.3.7	Integrity_constraint.....	294
7.3.8	Context_restriction_constraint	294
7.3.9	Domain_constraint.....	295
7.3.10	Subclass_constraint	295
7.3.11	Entity_subtype_constraint.....	296
7.3.12	Enumeration_constraint.....	296
7.3.13	Range_constraint	297
7.3.14	String_size_constraint	298
7.3.15	String_pattern_constraint.....	299
7.3.16	Cardinality_constraint.....	300
7.4	Définitions des types de l'ISO13584_IEC61360_class_constraint_schema	301
7.4.1	Généralités.....	301
7.4.2	Constraint_or_constraint_id.....	301
7.5	Définition de fonctions de l'ISO13584_IEC61360_class_constraint_schema	301
7.5.1	Généralités.....	301
7.5.2	Fonction integer_values_in_range	301
7.5.3	Fonction correct_precondition.....	301
7.5.4	Fonction correct_constraint_type	302
7.5.5	Fonction compatible_data_type_and_value	305
7.6	Définition de règles de l'ISO13584_IEC61360_class_constraint_schema	309
7.6.1	Généralités.....	309
7.6.2	Unique_constraint_id.....	309
8	ISO13584_IEC61360_item_class_case_of_schema	309
8.1	Vue d'ensemble.....	309
8.2	Introduction à l'ISO13584_IEC61360_item_class_case_of_schema.....	310
8.3	Définitions d'entités de l'ISO13584_IEC61360_item_class_case_of_schema.....	311
8.3.1	Relation sémantique <i>a priori</i>	311
8.3.2	Item_class_case_of.....	313
8.4	Définitions de fonctions de l'ISO13584_IEC61360_item_class_case_of_schema.....	316
8.4.1	Généralités.....	316
8.4.2	Fonction compute_known_property_constraints.....	316
8.4.3	Fonction compute_known_referenced_property_constraints	317
8.4.4	Fonction superclass_of_item_is_item	318
8.4.5	Fonction check_is_case_of_referenced_classes_definition.....	319
8.5	Définitions de règle de l'ISO13584_IEC61360_item_class_case_of_schema	319
8.5.1	Généralités.....	319
8.5.2	Règle imported_properties_are_visible_or_applicable_rule.....	319
8.5.3	Règle imported_data_types_are_visible_or_applicable_rule	320
8.5.4	Règle allowed_named_type_usage_rule.....	320

Annexe A (informative) Exemple de fichier physique	322
Annexe B (informative) Diagramme EXPRESS-G	327
Annexe C (informative) Dictionnaires partiels.....	338
Annexe D (normative) Spécification du format des valeurs.....	339
Bibliographie.....	354
Figure 1 – Vue d'ensemble du schéma de dictionnaire	198
Figure 2 – Éléments de données avec relations	201
Figure 3 – Mise en œuvre de relations "entre éléments" à l'aide d'unités sémantiques de base	202
Figure 4 – Relation entre unité sémantique de base et élément de dictionnaire	205
Figure 5 – BSU et éléments de dictionnaire courants	206
Figure 6 – Vue d'ensemble des données fournisseur et des relations.....	209
Figure 7 – Vue d'ensemble des données de classe et de leurs relations	211
Figure 8 – Exemple d'ontologie fournisseur.....	221
Figure 9 – Vue d'ensemble des attributs des données d'un élément et de leurs relations.....	225
Figure 10 – Genres des types de données d'un élément	225
Figure 11 – Hiérarchie d'entités pour le système de types.....	228
Figure 12 – Vue d'ensemble des types d'élément de données non quantitatif	248
Figure 13 – ISO13584_IEC61360_language_resource_schema et support_resource_schema	284
Figure B.1 – ISO13584_IEC61360_dictionary_schema – Diagramme EXPRESS-G 1 sur 7	328
Figure B.2 – ISO13584_IEC61360_dictionary_schema – Diagramme EXPRESS-G 2 sur 7	329
Figure B.3 – ISO13584_IEC61360_dictionary_schema – Diagramme EXPRESS-G 3 sur 7	330
Figure B.4 – ISO13584_IEC61360_dictionary_schema – Diagramme EXPRESS-G 4 sur 7	331
Figure B.5 – ISO13584_IEC61360_dictionary_schema – Diagramme EXPRESS-G 5 sur 7	332
Figure B.6 – ISO13584_IEC61360_dictionary_schema – Diagramme EXPRESS-G 6 sur 7	333
Figure B.7 – ISO13584_IEC61360_dictionary_schema – Diagramme EXPRESS-G 7 sur 7	334
Figure B.8 – ISO13584_IEC61360_language_resource_schema – Diagramme EXPRESS-G 1 sur 1	335
Figure B.9 – ISO13584_IEC61360_constraint_schema – Diagramme EXPRESS-G 1 sur 1	336
Figure B.10 – ISO13584_IEC61360_item_class_case_of_schema – Diagramme EXPRESS-G 1 sur 1	337
Tableau 1 – Table de références croisées.....	196
Tableau D.1 – Métalangage syntaxique EBNF de l'ISO/CEI 14977.....	340
Tableau D.2 – Transposition des chiffres de style européen en chiffres arabes.....	347
Tableau D.3 – Exemples de valeurs numériques.....	348

Tableau D.4 – Caractères issus d'autres rangées du Basic Multilingual Plane (plan multilingue de base) de l'ISO/CEI 10646-1	349
---	-----

COMMISSION ÉLECTROTECHNIQUE INTERNATIONALE

TYPES NORMALISÉS D'ÉLÉMENTS DE DONNÉES AVEC PLAN DE CLASSIFICATION POUR COMPOSANTS ÉLECTRIQUES –

Partie 2: Schéma d'un dictionnaire EXPRESS

AVANT-PROPOS

- 1) La Commission Electrotechnique Internationale (CEI) est une organisation mondiale de normalisation composée de l'ensemble des comités électrotechniques nationaux (Comités nationaux de la CEI). La CEI a pour objet de favoriser la coopération internationale pour toutes les questions de normalisation dans les domaines de l'électricité et de l'électronique. A cet effet, la CEI – entre autres activités – publie des Normes internationales, des Spécifications techniques, des Rapports techniques, des Spécifications accessibles au public (PAS) et des Guides (ci-après dénommés "Publication(s) de la CEI"). Leur élaboration est confiée à des comités d'études, aux travaux desquels tout Comité national intéressé par le sujet traité peut participer. Les organisations internationales, gouvernementales et non gouvernementales, en liaison avec la CEI, participent également aux travaux. La CEI collabore étroitement avec l'Organisation Internationale de Normalisation (ISO), selon des conditions fixées par accord entre les deux organisations.
- 2) Les décisions ou accords officiels de la CEI concernant les questions techniques représentent, dans la mesure du possible, un accord international sur les sujets étudiés, étant donné que les Comités nationaux de la CEI intéressés sont représentés dans chaque comité d'études.
- 3) Les Publications de la CEI se présentent sous la forme de recommandations internationales et sont agréées comme telles par les Comités nationaux de la CEI. Tous les efforts raisonnables sont entrepris afin que la CEI s'assure de l'exactitude du contenu technique de ses publications; la CEI ne peut pas être tenue responsable de l'éventuelle mauvaise utilisation ou interprétation qui en est faite par un quelconque utilisateur final.
- 4) Dans le but d'encourager l'uniformité internationale, les Comités nationaux de la CEI s'engagent, dans toute la mesure possible, à appliquer de façon transparente les Publications de la CEI dans leurs publications nationales et régionales. Toutes divergences entre toutes Publications de la CEI et toutes publications nationales ou régionales correspondantes doivent être indiquées en termes clairs dans ces dernières.
- 5) La CEI elle-même ne fournit aucune attestation de conformité. Des organismes de certification indépendants fournissent des services d'évaluation de conformité et, dans certains secteurs, accèdent aux marques de conformité de la CEI. La CEI n'est responsable d'aucun des services effectués par les organismes de certification indépendants.
- 6) Tous les utilisateurs doivent s'assurer qu'ils sont en possession de la dernière édition de cette publication.
- 7) Aucune responsabilité ne doit être imputée à la CEI, à ses administrateurs, employés, auxiliaires ou mandataires, y compris ses experts particuliers et les membres de ses comités d'études et des Comités nationaux de la CEI, pour tout préjudice causé en cas de dommages corporels et matériels, ou de tout autre dommage de quelque nature que ce soit, directe ou indirecte, ou pour supporter les coûts (y compris les frais de justice) et les dépenses découlant de la publication ou de l'utilisation de cette Publication de la CEI ou de toute autre Publication de la CEI, ou au crédit qui lui est accordé.
- 8) L'attention est attirée sur les références normatives citées dans cette publication. L'utilisation de publications référencées est obligatoire pour une application correcte de la présente publication.
- 9) L'attention est attirée sur le fait que certains des éléments de la présente Publication de la CEI peuvent faire l'objet de droits de brevet. La CEI ne saurait être tenue pour responsable de ne pas avoir identifié de tels droits de brevets et de ne pas avoir signalé leur existence.

La Norme internationale CEI 61360-2 a été établie par le sous-comité 3D: Propriétés et classes des produits et leur identification, du comité d'études 3 de la CEI: Structures d'informations, documentation et symboles graphiques.

Cette troisième édition annule et remplace la deuxième édition parue en 2002 et son Amendement 1 (2003). Elle constitue une révision technique.

Cette édition inclut les modifications techniques majeures suivantes par rapport à l'édition précédente :

- séparation des concepts entre la classe de catégorisation et la classe de caractérisation;
- introduction de contraintes de valeurs pour les classes et les propriétés;

- ajout de nouveaux sous-types différents pour les types de données, y compris l'entité `rational_type` ;
- amélioration de la représentation de l'unité de mesure.

Le texte de cette norme est issu des documents suivants:

FDIS	Rapport de vote
3D/196/FDIS	3D/204/RVD

Le rapport de vote indiqué dans le tableau ci-dessus donne toute information sur le vote ayant abouti à l'approbation de cette norme.

Cette publication a été rédigée selon les Directives ISO/CEI, Partie 2.

Une liste de toutes les parties de la série CEI 61360, présentées sous le titre général *Types normalisés d'éléments de données avec plan de classification pour composants électriques*, peut être consultée sur le site web de la CEI.

Le comité a décidé que le contenu de cette publication ne sera pas modifié avant la date de stabilité indiquée sur le site web de la CEI sous "<http://webstore.iec.ch>" dans les données relatives à la publication recherchée. A cette date, la publication sera

- reconduite,
- supprimée,
- remplacée par une édition révisée, ou
- amendée.

INTRODUCTION

Le schéma commun de dictionnaire ISO/CEI présenté ici est basé sur l'intersection des domaines d'application des normes suivantes:

- CEI 61360-1;
- ISO 13584-42.

Les parties pertinentes des articles du domaine d'application de ces normes comprennent ce qui suit:

CEI 61360-1:2009

«La présente partie de la CEI 61360 donne une base solide pour la définition claire et non ambiguë des propriétés caractéristiques (types d'éléments de données) de tous les éléments des systèmes électrotechniques depuis les composants de base jusqu'aux sous-ensembles et aux systèmes complets. Bien qu'ils aient été conçus à l'origine dans l'optique de fournir une base pour l'échange d'information sur les composants électriques/électroniques, il est admis d'utiliser les principes et les méthodes contenus dans la présente norme dans des domaines autres que ceux de la conception d'origine comme les ensembles de composants et les systèmes et les sous-systèmes électrotechniques.»

ISO 13584-42:2010 (disponible en anglais seulement)

“Cette partie de l'ISO 13584 spécifie les principes à utiliser pour définir les classes de caractérisation des parties et les propriétés des parties qui caractérisent une partie indépendamment de toute identification particulière définie par un fournisseur.

Les règles et lignes directrices données dans cette partie de l'ISO 13584 sont obligatoires pour les comités de normalisation responsables de la création de caractérisation hiérarchisée normalisée.

L'utilisation de ces règles par les fournisseurs et utilisateurs est une méthodologie recommandée pour construire leur propre hiérarchie. ”

Le SC 3D de la CEI et le TC184/SC4 de l'ISO ont convenu de NE PAS changer/et ou modifier le modèle EXPRESS proposé de manière indépendante l'un de l'autre afin de garantir l'harmonisation et la réutilisabilité du travail fourni par les deux comités. Par conséquent, il convient d'envoyer les demandes d'amendements aux deux comités. Il convient que ces demandes soient adoptées par les deux comités avant de modifier le modèle d'informations EXPRESS.

TYPES NORMALISÉS D'ÉLÉMENTS DE DONNÉES AVEC PLAN DE CLASSIFICATION POUR COMPOSANTS ÉLECTRIQUES –

Partie 2: Schéma d'un dictionnaire EXPRESS

1 Domaine d'application

La présente partie de la série CEI 61360 fournit un modèle formel pour les données conformément au domaine d'application donné dans la CEI 61360-1 et l'ISO 13584-42 et, donc, fournit un moyen pour la représentation et l'échange des données interprétables par un ordinateur.

L'intention est de fournir un modèle commun d'information pour le travail du SC 3D de la CEI et du TC184/SC4 de l'ISO, permettant ainsi la mise en œuvre de dictionnaires de systèmes traitant des données livrées conformément à l'une ou l'autre des normes établies par les deux comités.

Le domaine d'application de la présente partie de la CEI 61360 est le schéma de dictionnaire commun ISO/CEI basé sur l'intersection des deux domaines d'application des deux normes de base CEI 61360-1 et ISO 13584-42.

Le modèle EXPRESS proposé représente un modèle formel commun pour les deux normes et facilite leur harmonisation.

La CEI 61360-2 constitue le document maître. L'ISO 13584-42 contient une copie du modèle EXPRESS de la CEI 61360-2 dans une annexe informative

Dans un certain d'articles, où le modèle EXPRESS commun donne plus de liberté, la CI a défini un plus grand nombre de restrictions qui peuvent être consultées dans la partie méthodologie de la CEI 61360-1.

Deux schémas sont fournis dans la présente partie de la CEI 61360 définissant deux options qui peuvent être sélectionnées pour une mise en œuvre. Chacune de ces options est appelée "classe de conformité".

- L'ISO13584_IEC61360_dictionary_schema2 permet de prendre en charge la modélisation et l'échange de types d'éléments de données techniques avec le schéma de classification associé utilisé dans les définitions des types d'élément de données. Elle constitue la classe de conformité 1 de la présente partie de la CEI 61360.
- L'ISO13584_IEC61360_language_resource_schema fournit des ressources si les chaînes le permettent dans des langues diverses. Elle a été extraite du schéma de dictionnaire, car elle pourrait être utilisée dans d'autres schémas conceptuels. Elle repose largement sur le support_resource_schema issu de l'ISO 10303-41:2000 et peut être vue comme une extension à cela. Elle permet l'utilisation d'une langue spécifique dans tout un contexte d'échange (Fichier physique) sans la surcharge introduite lorsque plusieurs langues sont utilisées.

Lorsqu'il est utilisé avec l'ISO 10303-21, chaque schéma définit un seul format d'échange. Le format d'échange défini par la classe de conformité 1 est totalement compatible avec la série ISO 13584.

2 Références normatives

Les documents de référence suivants sont indispensables pour l'application du présent document. Pour les références datées, seule l'édition citée s'applique. Pour les références non datées, la dernière édition du document de référence s'applique (y compris les éventuels amendements).

CEI 61360-1:2009, *Types normalisés d'éléments de données avec plan de classification pour composants électriques – Partie 1: Définitions – Principes et méthodes*

CEI 61360-DB, *Standard data element types with associated classification scheme for electric components – Part 4: IEC reference collection of standard data element types and component classes* (disponible en anglais seulement)

ISO/CEI 8859-1:1998, *Technologies de l'information – Jeux de caractères graphiques codés sur un seul octet – Partie 1 : Alphabet latin n°1*

ISO/CEI 10646-1, *Technologies de l'information – Jeu universel de caractères codés sur plusieurs octets (JUC) – Partie 1:Architecture et plan multilingue de base*

ISO/CEI 14977, *Technologies de l'information – Métalangage syntaxique – BNF étendu*

ISO 639 (toutes les parties), *Codes pour la représentation des noms de langue*

ISO 843:1997, *Information et documentation – Conversion des caractères grecs en caractères latins*

ISO 3166-1, *Codes pour la représentation des noms de pays et de leurs subdivisions – Partie 1: Codes de pays*

ISO 4217:2008, *Codes pour la représentation des monnaies et types de fonds*

ISO 8601:2004, *Éléments de données et formats d'échange – Échange d'information – Représentation de la date et de l'heure*

ISO 10303-11:2004, *Systèmes d'automatisation industrielle et intégration – Représentation et échange de données de produits – Partie 11: Méthodes de description: Manuel de référence du langage EXPRESS*

ISO 10303-21:2002, *Systèmes d'automatisation industrielle et intégration – Représentation et échange de données de produits – Partie 21: Méthodes de mise en application: Encodage en texte clair des fichiers d'échange*

ISO 10303-41:2000, *Systèmes d'automatisation industrielle et intégration – Représentation et échange de données de produits – Partie 41: Ressources génériques intégrées: Principes de description et de support de produits¹*

ISO 13584-26, *Systèmes d'automatisation industrielle et intégration – Bibliothèque de composants – Partie 26: Ressource logique: Identification des fournisseurs d'information*

ISO 13584-42:2010, *Systèmes d'automatisation industrielle et intégration – Bibliothèque de composants – Partie 42: Méthodologie descriptive: Méthodologie appliquée à la structuration des familles de pièces*

¹ Une nouvelle édition de l'ISO 10303-41 a été publiée en 2005.

3 Termes et définitions

Pour les besoins du présent document, les termes et définitions suivants s'appliquent.

3.1

classe abstraite

classe dont les membres sont également des membres d'une de ses sous-classes

Note 1 à l'article: Les classes abstraites sont utilisées lorsqu'il est nécessaire de regrouper différentes sortes d'objets en une classe d'une hiérarchie d'inclusion de classes.

Note 2 à l'article: Dans le modèle de dictionnaire commun ISO13584/CEI61360, il est possible de définir aussi bien des classes de catégorisation abstraites que des classes de caractérisation abstraites. Le fait qu'elle soit abstraite est seulement une caractéristique conceptuelle d'une classe. Cette caractéristique n'est pas représentée de manière explicite dans le modèle.

Note 3 à l'article: Par la relation d'héritage, une classe de caractérisation abstraite permet de partager, par exemple, certaines propriétés visibles entre différentes sous-classes qui correspondent à différentes sortes d'éléments.

3.2

propriété applicable d'une classe

propriété applicable possédée nécessairement par chaque partie qui est membre d'une classe de caractérisation

Note 1 à l'article: Chaque partie qui est membre d'une classe de caractérisation possède un aspect correspondant à chaque propriété applicable de cette classe de caractérisation.

Note 2 à l'article: La définition ci-dessus est conceptuelle, il n'y a pas d'exigence demandant que toutes les propriétés applicables d'une classe soient utilisées pour décrire chaque partie de cette classe au niveau du modèle de données.

Note 3 à l'article: Toutes les propriétés applicables d'une superclasse sont aussi des propriétés applicables pour les sous-classes de cette superclasse.

Note 4 à l'article: Seules les propriétés définies ou héritées comme étant des propriétés visibles et importées d'une classe peuvent être des propriétés applicables.

Note 5 à l'article: Pour faciliter l'intégration des bibliothèques de composants et des catalogues électroniques selon l'ISO 13584-24:2003 et l'ISO 13584-25, ces parties de l'ISO 13584 demandent que seules les propriétés qui sont applicables à une classe soient utilisées pour caractériser leurs instances dans les bibliothèques de composants et les catalogues électroniques.

3.3

attribut

élément de données interprétable par un ordinateur pour la description d'une propriété, d'une relation ou d'une classe

Note 1 à l'article: Un attribut décrit un seul détail d'une propriété, d'une classe ou d'une relation.

EXEMPLE Le nom d'une propriété, le code d'une classe, l'unité de mesure dans laquelle sont fournies les valeurs d'une propriété.

3.4

unité sémantique de base

entité qui fournit une identification absolue et universellement unique d'un certain objet du domaine d'application qui est représenté comme un élément du dictionnaire

EXEMPLE 1 Un dictionnaire conforme à la présente partie de la CEI 61360 permet la prise en charge de l'identification de classes, de propriétés, de sources d'informations et de types de données (datatypes).

EXEMPLE 2 Un dictionnaire conforme à l'ISO 13584-24:2003 permet la prise en charge de l'identification de classes, de propriétés, de sources d'informations, de types de données (datatypes), de tableaux, de documents et des bibliothèques de programmes.

EXEMPLE 3 Dans l'ISO 13584-511, la classe des boulons à tête hexagonale est identifiée par une BSU, la propriété "qualité de tolérance de filetage" est également identifiée par une BSU.

Note 1 à l'article: Le contenu d'une unité sémantique de base peut également être représenté par un IRDI.²

3.5

caractéristique d'un produit

caractéristique produit

propriété invariable, caractéristique d'un produit, dont la valeur est fixée une fois le produit défini

Note 1 à l'article: Changer la valeur d'une caractéristique d'un produit signifierait changer le produit.

EXEMPLE Pour un roulement à billes, le diamètre intérieur et le diamètre extérieur sont des caractéristiques de produit.

Note 2 à l'article: Adaptée de l'ISO 13584-24:2003, définition 3.12.

3.6

classe

abstraction d'un ensemble de produits semblables

Note 1 à l'article: Un produit qui se conforme à l'abstraction définie par une classe est appelé un "membre de classe".

Note 2 à l'article: Une classe est un concept intentionnel qui peut prendre de nombreuses significations différentes dans des contextes différents.

EXEMPLE L'ensemble des produits utilisés par une entreprise particulière et l'ensemble des produits normalisés par l'ISO sont deux exemples de contextes. Dans ces deux contextes (l'entreprise particulière et l'ISO), l'ensemble des produits qui sont considérés comme étant des membres de la classe *roulements avec une rangée de billes* peut être différent, en particulier parce que les employés de chaque entreprise ignorent l'existence d'un certain nombre de produits roulements avec une rangée de billes.

Note 3 à l'article: Les classes sont structurées par des relations d'inclusion de classes.

Note 4 à l'article: Une classe de produits est un concept général tel que défini dans l'ISO 1087-1. Ainsi, il est conseillé d'utiliser les règles définies dans l'ISO 704 pour définir les attributs de dénomination et de définition des classes de produits.

Note 5 à l'article: Dans le contexte de la série ISO 13584, une classe est soit une classe de caractérisation, associée à des propriétés et utilisable pour caractériser des produits, soit une classe de catégorisation, non associée à des propriétés et non utilisable pour caractériser des produits.

3.7

relation d'inclusion de classes

relation entre classes qui signifie l'inclusion des membres de classes: si A est une superclasse de A1, cela signifie que, dans n'importe quel contexte, tout membre de A1 est également membre de A

EXEMPLE 1 L'ensemble de produits utilisés par une entreprise particulière et l'ensemble de tous les produits normalisés par l'ISO sont deux exemples de contextes.

EXEMPLE 2 Dans n'importe quel contexte, la classe *condensateur* inclut la classe *condensateur électrolytique*.

Note 1 à l'article: L'inclusion de classes définit une structure hiérarchique entre les classes.

Note 2 à l'article: L'inclusion de classes est une relation conceptuelle qui ne prescrit rien au niveau de la représentation des données. En conséquence, elle ne prescrit aucun schéma particulier de base des données ou modèle particulier de données.

Note 3 à l'article: Dans le modèle défini dans la présente partie de la CEI 61360, la relation "is-a" (c'est-à-dire: "est un") assure l'inclusion de classes. La présente partie de la CEI 61360 recommande que la relation "case-of" assure aussi l'inclusion de classe.

Note 4 à l'article: La relation d'inclusion de classe s'appelle également "subsomption" (ou "subsumption").

² IRDI = *International Registration Data Identifier*.

3.8

membre de classe

produit qui se conforme à l'abstraction définie par une classe

3.9

propriété de valeur d'une classe

propriété qui a une seule valeur pour toute une classe de caractérisation de produits

Note 1 à l'article: La valeur d'une propriété de valeur d'une classe n'est pas définie de façon individuelle pour chaque produit d'une caractérisation de classe, mais de façon globale pour la classe elle-même.

Note 2 à l'article: Lorsque tous les produits d'une classe de produits ont la même valeur pour une propriété particulière, le fait de définir la propriété comme étant une propriété de valeur d'une classe permet d'éviter la duplication de la valeur pour chaque instance.

Note 3 à l'article: Les propriétés de valeur d'une classe peuvent également être utilisées pour capturer une certaine identité entre différentes caractérisations de classes lorsque cette identité n'est pas capturée par la structure hiérarchique.

3.10

modèle de dictionnaire commun ISO13584/CEI61360

modèle de données pour ontologie de produits, utilisant le langage de modélisation d'informations EXPRESS, résultant d'un effort conjoint entre le TC 184/SC 4/WG 2 de l'ISO et le SC 3D de la CEI

Note 1 à l'article: Plusieurs niveaux de mises en œuvre autorisées, appelées classes de conformité, sont définis pour le modèle de dictionnaire commun ISO13584/CEI61360. La classe de conformité 1 est constituée des divers schémas documentés dans la présente partie de la CEI 61360 (qui dupliquent des informations contenue dans la présente norme), plus l'ISO13584_IEC61360_dictionary_aggregate_extension_schema documenté dans l'ISO 13584-25 (dupliqué dans la CEI 61360-5). D'autres classes de conformité sont documentées dans l'ISO 13584-25 (classes de conformité 2, 3 et 4).

Note 2 à l'article: Dans la série de normes ISO 13584, chaque ontologie particulière de produits traitant d'un domaine particulier de produits et basée sur le modèle de dictionnaire commun ISO13584/CEI61360 est appelée "dictionnaire de référence" pour le domaine en question.

3.11

caractéristique de produit dépendante du contexte

propriété d'un *produit* dont la valeur dépend de certains *paramètres de contexte*

Note 1 à l'article: Pour un produit donné, une caractéristique dépendant du contexte est définie mathématiquement comme étant une fonction dont le domaine est défini par un certain nombre de paramètres du contexte qui définissent l'environnement du produit.

EXEMPLE Pour un *roulement à billes*, la *durée de vie* est une caractéristique dépendant du contexte qui est fonction de la *charge radiale*, de la *charge axiale* et de la *vitesse de rotation*.

Note 2 à l'article: Adaptée de l'ISO 13584-24:2003, définition 3.22.

3.12

paramètre de contexte

variable dont la valeur caractérise le contexte dans lequel un *produit* est placé

EXEMPLE 1 La *charge dynamique* appliquée à un *palier* est un paramètre de contexte pour ce *palier*.

EXEMPLE 2 La *température ambiante* à laquelle la *valeur de résistance* d'une *résistance* est mesurée est un paramètre de contexte pour cette *résistance*.

Note 1 à l'article: Cette définition annule et remplace la définition donnée dans l'ISO 13584-24:2003, qui était la suivante: "variable dont la valeur caractérise le contexte dans lequel il est prévu de placer un *produit*".

Note 2 à l'article: Dans la série de normes ISO 13584, une valeur de propriété est représentée comme étant un type d'élément de données.

3.13

type d'élément de données

unité de données pour laquelle l'identification, la description et la représentation de la valeur ont été spécifiées

Note 1 à l'article: Dans la série de normes ISO 13584, une valeur de propriété est représentée comme étant un type d'élément de données.

3.14

données de dictionnaire

ensemble des données qui représente des ontologies de produits éventuellement associées à des catégorisations de produits

Note 1 à l'article: Il est conseillé d'échanger les données du dictionnaire en utilisant une certaine classe de conformité du modèle de dictionnaire commun ISO/CEI.

Note 2 à l'article: Cette définition de "données de dictionnaire" annule et remplace la définition antérieure issue de la première édition de la CEI 61360-2 qui était la suivante: "ensemble de données qui décrit des hiérarchies des classes de caractérisation des produits et des propriétés de ces produits".

3.15

élément de dictionnaire

ensemble d'attributs qui constitue la description du dictionnaire de certains objets du domaine d'application

EXEMPLE 1 Un dictionnaire conforme à la présente partie de la CEI 61360 permet la prise en charge de la description des classes, des propriétés, des sources d'informations et des types de données (datatypes).

EXEMPLE 2 Un dictionnaire conforme à l'ISO 13584-24:2003 permet la prise en charge de la description des classes, des propriétés, des sources d'informations, des types de données (datatypes), des tableaux, des documents et des bibliothèques de programmes.

3.16

famille de produits

ensemble de produits représentés par la même caractérisation de classe

Note 1 à l'article: Cette définition annule et remplace la définition donnée dans l'ISO 13584-24:2003, qui était la suivante: "famille simple ou générique de pièces".

3.17

caractéristique intrinsèque

aspect d'un produit qui peut être décrit par une classe de caractérisation et un ensemble de paires propriété-valeur

Note 1 à l'article: Dans le monde réel, une instance de caractéristique intrinsèque n'existe qu'intégrée au sein du produit dont elle est un aspect.

EXEMPLE 1 La tête d'une vis est une caractéristique intrinsèque décrite par une classe de têtes et un certain nombre de propriétés de la tête, qui dépend de la classe de têtes. Une tête de vis existe seulement lorsqu'elle appartient à une vis.

Note 2 à l'article: Les caractéristiques intrinsèques sont représentées au moyen d'**item_class** dont l'attribut **instance_sharable** est égal à *false*.

Note 3 à l'article: L'attribut **instance_sharable** permet de spécifier le statut conceptuel d'un article: soit un article autonome (**instance_sharable** = *true*), soit une caractéristique intrinsèque (**instance_sharable** = *false*). Cela n'implique aucune contrainte au niveau de la représentation des données. Dans le modèle de dictionnaire commun ISO13584/CEI61360, le fait de représenter plusieurs instances du monde réel qui partagent la même représentation EXPRESS par une seule entité EXPRESS ou par plusieurs entités EXPRESS est considéré comme étant dépendant de la mise en œuvre. Il n'existe aucun mécanisme pour spécifier si une valeur de données d'une instance de caractéristique intrinsèque peut ou peut ne pas être partagée.

EXEMPLE 2 La même instance d'une classe de têtes de vis peut être référencée par plusieurs instances d'une classe de vis. Cela signifie qu'il existe plusieurs têtes de vis, mais toutes les têtes de vis ont la classe de caractérisation et le même ensemble de valeurs de propriétés. L'attribut **instance_sharable** permet de spécifier que le fait de changer cette instance de la classe têtes de vis changerait plusieurs instances de la classe vis.

3.18

propriété importée

propriété définie dans une classe qui est sélectionnée par une autre classe du même dictionnaire de référence ou d'un dictionnaire de référence différent, au moyen de la relation "case-of", pour devenir applicable à cette dernière classe

Note 1 à l'article: Seules des propriétés qui sont visibles et/ou applicables dans une classe peuvent être importées de cette classe.

Note 2 à l'article: L'importation entre classes de dictionnaires de référence différentes permet de réutiliser des propriétés, définies par exemple dans un dictionnaire de référence normalisé, sans les redéfinir.

Note 3 à l'article: L'importation entre classes du même dictionnaire de référence reconnaît le fait que certains produits puissent accomplir plusieurs fonctions, ce qui exige la capacité d'importer une propriété de plusieurs classes de niveau supérieur.

Note 4 à l'article: Lorsqu'elle est importée dans une nouvelle classe, une propriété conserve son identificateur initial et, de ce fait, tous les attributs peuvent ne pas être dupliqués.

Note 5 à l'article: Une propriété importée est applicable pour la classe dans laquelle elle est importée.

3.19

information

fait, conception ou instruction

[SOURCE: ISO 10303-1:1994, définition 3.2.20]

3.20

modèle d'information

modèle formel d'un ensemble borné de faits, de concepts ou d'instructions pour satisfaire à une exigence spécifiée

[SOURCE: ISO 10303-1:1994, définition 3.2.21]

3.21

fournisseur d'information

fournisseur

organisation qui délivre une ontologie, c'est-à-dire un dictionnaire de données ou une bibliothèque fournisseur qui est responsable de son contenu

Note 1 à l'article: Cette définition de "fournisseur" annule et remplace la définition de "fournisseur d'information" issue de l'ISO 13584-1:2001 qui était la suivante: "organisation qui délivre une bibliothèque fournisseur dans le format normalisé défini la présente Norme Internationale et qui est responsable de son contenu".

3.22

identificateur international de données d'enregistrement

identificateur unique au niveau international pour un certain objet du domaine d'application tel que défini dans l'ISO/CEI 11179-5

Note 1 à l'article: Seuls les identificateurs internationaux de données d'enregistrement conformes à l'ISO/TS 29002-5 sont utilisés dans le contexte de la série de normes de l'ISO 13584.

Note 2 à l'article: Un identificateur international de données d'enregistrement peut être utilisé pour représenter le contexte d'une unité sémantique de base qui identifie un élément de dictionnaire comme étant une chaîne.

Note 3 à l'article: Un identificateur international de données d'enregistrement peut aussi être utilisé pour identifier le contenu d'un attribut d'un élément du dictionnaire.

EXEMPLE L'unité de mesure d'une propriété, d'une valeur d'une propriété ou d'une contrainte sur une propriété peut être identifiée par un identificateur international de données d'enregistrement (IRDI).³

3.23

relation "is-a"

relation d'inclusion de classes associée à l'héritage: si A1 *is-a* A, alors chaque produit appartenant à A1 appartient à A et tout ce qui est décrit dans ce contexte de A est automatiquement dupliqué dans le contexte de A1

Note 1 à l'article: Ce mécanisme est habituellement appelé "héritage".

³ IRDI = International Registration Data Identifier.

Note 2 à l'article: Dans le modèle de dictionnaire commun ISO13584/CEI61360, la relation *is-a* peut seulement être définie entre des classes de caractérisation. Il est conseillé qu'elle ne définisse qu'une seule hiérarchie et elle assure que les propriétés tant visibles qu'applicables sont héritées.

3.24

relation "is-case-of" case-of

mécanisme d'importation de propriétés: si A1 *is case-of* A, alors la définition des produits de A couvre également les produits de A1 et, donc, A1 peut importer n'importe quelle propriété provenant de A

Note 1 à l'article: Le but de la relation *case-of* est de permettre de connecter ensemble plusieurs hiérarchies d'inclusion de classes tout en assurant que les hiérarchies référencées peuvent être mises à jour de façon indépendante.

Note 2 à l'article: Il n'existe pas de contrainte pour que la relation "case-of" soit destinée à définir des hiérarchies simples.

Note 3 à l'article: Dans le modèle de dictionnaire commun ISO13584/CEI61360, la relation "case-of" peut être utilisée en particulier dans quatre cas: (1) pour lier une classe de caractérisation à une classe de catégorisation, (2) pour importer, dans le contexte de certains dictionnaires de référence normalisés, un certain nombre de propriétés définies dans d'autres dictionnaires de référence normalisés, (3) pour relier un dictionnaire utilisateur de référence à un ou plusieurs dictionnaires de référence normalisés, (4) pour décrire un produit en utilisant les propriétés de différentes classes: lorsque des produits de classe A1 remplissent deux fonctions différentes et sont donc logiquement décrits par des propriétés associées à deux classes différentes, A et B, A1 peut être reliée par une relation "is-a", par exemple, à A et par une relation "case-of" à B.

Note 4 à l'article: Les constructions de ressources EXPRESS pour modéliser les relations "case-of" sont définies en 4.5 et après.

3.25

article

chose qui peut être caractérisée au moyen d'une classe de caractérisation à laquelle elle appartient et d'un ensemble de paires propriété-valeur

Note 1 à l'article: Cette définition remplace la définition donnée dans l'ISO 13584-24:2003, qui était la suivante: "un objet dont la description peut être capturée par une structure de classe et un ensemble de propriétés".

Note 2 à l'article: Dans la série de normes ISO 13584, des produits et des caractéristiques intrinsèques de produits qui correspondent à des propriétés composées sont des articles.

3.26

classe de caractérisation feuille

classe de caractérisation qui n'est pas encore spécialisée en classes de caractérisation plus précises

EXEMPLE Vis à tête fraisée plate à empreinte cruciforme (type Y) et vis à tête cylindrique à six pans creux avec filetage métrique à pas fin sont des classes de caractérisation feuilles définies dans l'ISO 13584-511.

3.27

classe de caractérisation non feuille

classe de caractérisation qui est encore spécialisée en classes de caractérisation plus précises

EXEMPLE Composant à filetage externe et boulon/vis à filetage métrique sont des classes de caractérisation non feuilles définies dans l'ISO 13584-511.

3.28

type d'élément de données non quantitatif

type d'élément de données qui identifie ou décrit un objet au moyen de codes, d'abréviations, de noms, de références ou de descriptions

3.29

composant

élément matériel ou fonctionnel conçu pour constituer un composant de différents produits

[SOURCE: ISO 13584-1:2001, définition 3.1.16]

3.30

bibliothèque de composants

ontologie de produits interprétable par un ordinateur et description interprétable par un ordinateur d'un ensemble de produits au moyen des références à cette ontologie

Note 1 à l'article: Cette définition remplace la définition donnée dans la première édition de la présente partie de la CEI 61360, qui était la suivante: "ensemble identifié de données et éventuellement de programmes qui peuvent générer des informations relatives à un ensemble de composants".

3.31

produit

chose ou substance produite par un processus naturel ou artificiel

Note 1 à l'article: Dans la présente partie de la CEI 61360, le terme "produit" est pris dans son sens le plus large afin d'inclure les dispositifs, les systèmes et les installations ainsi que les matériels, les processus, les logiciels et les services.

3.32

catégorisation de produits

catégorisation de composants

catégorisation

partitionnement récursif d'un ensemble de produits en sous-ensembles dans un but spécifique

Note 1 à l'article: Les sous-ensembles qui apparaissent dans une catégorisation de produit sont appelés "classes de catégorisation de produits" ou "catégories de produits".

Note 2 à l'article: Une catégorisation de produits n'est pas une ontologie de produits. Elle ne peut pas être utilisée pour caractériser des produits.

Note 3 à l'article: Aucune propriété n'est associée à des catégorisations.

Note 4 à l'article: Plusieurs catégorisations du même ensemble de produits sont possibles en fonction de leur usage visé.

EXEMPLE La classification UNSPSC (United Nations Standard Products and Service Code, définie par l'Organisation des Nations Unies) est un exemple de catégorisation de produits qui a été développée pour l'analyse des dépenses.

Note 5 à l'article: En utilisant la relation *is-case-of*, plusieurs hiérarchies de classes de caractérisation de produits peuvent être reliées à une hiérarchie de catégorisation pour créer une seule structure.

3.33

classe de catégorisation de produits

classe de catégorisation de composants

classe de catégorisation

classe de produits qui constitue un élément d'une catégorisation

EXEMPLE *Composants de fabrication et Fournitures*, et *Optiques industrielles* sont des exemples d'une classe de catégorisation de produits définie dans l'UNSPSC.

Note 1 à l'article: Aucune règle n'est donnée dans la présente partie de la CEI 61360 quant à la manière de sélectionner des classes de catégorisation. Ce concept est introduit (1) pour clarifier sa différence avec "classe de caractérisation", et (2) pour expliquer que la même classe de caractérisation peut être reliée à un nombre quelconque de classes de catégorisation.

Note 2 à l'article: Il n'y a aucune propriété associée à une classe de catégorisation.

3.34

caractérisation de produits

caractérisation de composants

description d'un produit au moyen d'une classe de caractérisation de produits à laquelle il appartient et d'un ensemble de paires propriété-valeur

EXEMPLE *Hexagon_head_bolts_ISO_4014* (Qualités de produits = A, thread_type=M, longueur= 50, Diamètre = 8) est un exemple de caractérisation de produits.

3.35
classe de caractérisation de produits
classe de caractérisation de composants
classe de caractérisation

classe de produits qui remplissent la même fonction et qui partagent des propriétés communes

Note 1 à l'article: Des classes de caractérisation de produits peuvent être définies à divers niveaux de détails, définissant ainsi une hiérarchie d'inclusion de classes.

EXEMPLE *Boulon/vis à filetage métrique* et *boulon six pans* sont des exemples de classes de caractérisation de produits définies dans l'ISO 13584-511. La première classe de caractérisation est incluse dans la seconde. *Transistor* et *transistor de puissance bipolaire* sont des exemples de classes de caractérisation de produits définies dans la CEI 61360-DB. La seconde est incluse dans la première.

3.36
ontologie de produits
ontologie de composants
ontologie

modèle de connaissances des produits, réalisé par une représentation formelle et consensuelle des concepts d'un domaine de produits en termes de classes de caractérisation identifiées, de relations de classes et de propriétés identifiées

Note 1 à l'article: Les ontologies de produits reposent sur un modèle classe-instance qui permet de reconnaître et de dénommer les ensembles de produits, appelés "classes de caractérisation", qui ont une fonction semblable (par exemple, *roulement à billes*, *condensateur*), mais aussi de discerner au sein d'une classe les divers sous-ensembles de produits, appelés "instances", qui sont considérés comme étant identiques. Il est conseillé d'utiliser les règles définies dans l'ISO 1087-1 pour formuler la dénomination et les définitions des classes de caractérisation. Les instances n'ont pas de définition. Elles sont désignées par la classe à laquelle elles appartiennent et un ensemble de paires propriété-valeur.

Note 2 à l'article: Les ontologies ne sont pas concernées par des mots, mais par des concepts indépendants de tout langage particulier.

Note 3 à l'article: "Consensuel" signifie que la conceptualisation est obtenue à la suite d'un accord commun dans une certaine communauté.

Note 4 à l'article: "Formel" signifie que l'ontologie est censée être interprétable par une machine. Un certain niveau de raisonnement machine est logiquement possible sur l'ontologie, par exemple vérification de cohérence, en utilisant des déductions.

Note 5 à l'article: "Identifié" signifie que chaque classe de caractérisation d'ontologie et des propriétés sont associées à un identificateur unique au niveau global permettant de référencer ce concept à partir de n'importe quel contexte.

Note 6 à l'article: Le modèle de données pour ontologie recommandé dans la présente partie de la CEI 61360 est le modèle de dictionnaire commun ISO13584/CEI61360, dont la version la plus simple est documentée dans la présente partie de la CEI 61360. Des versions plus complètes sont documentées dans l'ISO 13584-25 et la CEI 61360-5 (classes de conformité 1, 2, 3 et 4 de ces deux documents).

Note 7 à l'article: Dans la présente partie de la CEI 61360, chaque ontologie de produits traitant d'un domaine particulier de produits conforme au modèle de dictionnaire commun ISO13584/CEI61360 est appelée "dictionnaire de référence" pour le domaine en question.

EXEMPLE Le dictionnaire de référence pour composants électriques, qui est défini dans la CEI 61360-DB, est une ontologie de produits pour composants électriques conforme au modèle de dictionnaire commun ISO13584/CEI61360. Il est obtenu à la suite d'un commun accord entre tous les organismes membres du SC 3D de la CEI. Un dictionnaire de référence d'entreprise est obtenu à la suite d'un accord commun entre des experts désignés par la direction, et ce, au nom de la société.

3.37**propriété**

paramètre défini adapté pour la description et la différenciation de produits

Note 1 à l'article: Une propriété décrit un seul aspect d'un objet donné.

Note 2 à l'article: Une propriété est définie par la totalité de ses attributs associés. Les types et nombre d'attributs qui décrivent une propriété avec une exactitude élevée sont documentés dans la présente partie de la CEI 61360.

Note 3 à l'article: La présente partie de la CEI 61360 a identifié trois sortes différentes de propriétés: caractéristiques de produits, paramètres de contexte et caractéristiques de produits dépendantes du contexte.

Note 4 à l'article: Cette définition de "propriété" remplace la définition antérieure issue de la première édition de la présente partie de la CEI 61360 qui était la suivante: "information pouvant être représentée par un type d'élément de données".

Note 5 à l'article: Dans la série de normes ISO 13584, une valeur de propriété est représentée comme étant un type d'élément de données.

3.38

type de données de propriété

ensemble autorisé de valeurs d'une propriété

3.39

classe de définition de propriété

classe de caractérisation de produits dans le contexte duquel une propriété de produit est définie

Note 1 à l'article: Dans le modèle de dictionnaire commun ISO13584/CEI61360, chaque propriété de produit a une seule classe de définitions de propriété qui définit son domaine d'application. La propriété a un sens seulement pour cette classe, et toutes ses sous-classes, et elle est dite *visible* sur ce domaine.

EXEMPLE Dans l'ISO 13584-511, *wrenching height* («hauteur de prise de clé») possède *nut* («écrou») comme sa classe de définition de propriétés et *major diameter of external thread* («grand diamètre du filetage externe») possède *metric external thread* («filetage externe métrique») comme sa classe de définition de produits.

3.40

type d'élément de données quantitatif

type d'élément de données doté d'une valeur numérique représentant une grandeur physique, une somme d'informations ou un total d'objets

3.41

dictionnaire de référence

ontologie de produits conforme au modèle de dictionnaire commun ISO 13584/CEI 61360

Note 1 à l'article: Dans la série de normes ISO 13584, une ontologie de produits qui traite d'un domaine particulier de produits, basée sur le modèle de dictionnaire commun ISO13584/CEI61360, est appelée "dictionnaire de référence" pour le domaine en question.

3.42

construction de ressource

collection d'entités, types, fonctions, règles et références du langage EXPRESS, définissant ensemble une description valide de données

Note 1 à l'article: Cette définition est adaptée de la définition de "construction de ressource" donnée dans l'ISO 10303-1:1994, à savoir "collection d'entités, types, fonctions, règles et références du langage EXPRESS, définissant ensemble une description valide de données de produit".

[SOURCE: ISO 13584-1:2001, définition 3.1.21, modifiée]

3.43

sous-classe

classe inférieure d'un niveau à une autre classe dans une hiérarchie d'inclusion de classes

Note 1 à l'article: Dans le modèle de dictionnaire commun ISO13584/CEI61360, les hiérarchies d'inclusion de classes sont définies par la relation *is-a*. Elles peuvent également être établies par les relations *case-of*.

3.44

superclasse

classe supérieure d'un niveau à une autre classe dans une hiérarchie d'inclusion de classes

Note 1 à l'article: Dans le modèle de dictionnaire commun ISO13584/CEI61360, les hiérarchies d'inclusion de classes sont définies par la relation *is-a*. Elles peuvent également être établies par les relations *case-of*.

Note 2 à l'article: Dans le modèle de dictionnaire commun ISO13584/CEI61360, une classe à tout au plus une superclasse spécifiée au moyen d'une relation *is-a*.

3.45

bibliothèque fournisseur

bibliothèque de composants dont le fournisseur d'informations est différent de l'utilisateur de la bibliothèque

Note 1 à l'article: Cette définition remplace la définition donnée dans l'ISO 13584-1:2001, qui était la suivante: "ensemble de données et éventuellement de programmes, pour lesquels le fournisseur est défini et décrivant dans le format normalisé défini dans la présente Norme internationale un ensemble de produits et/ou un ensemble de représentations de produits".

3.46

propriété visible

propriété qui a une définition ayant un sens dans le domaine d'application d'une classe de caractérisation donnée, mais qui ne s'applique pas nécessairement aux divers produits appartenant à cette classe

Note 1 à l'article: "Ayant un sens dans le domaine d'application d'une classe de caractérisation donnée" signifie qu'un observateur humain est à même de déterminer, pour tout produit de la classe de caractérisation, si la propriété s'applique ou pas, et, si elle s'applique, à quel aspect de produit elle correspond.

Note 2 à l'article: Le concept de propriété visible permet de partager la définition d'une propriété entre des classes de caractérisation de produits dans lesquelles cette propriété ne s'applique pas nécessairement.

EXEMPLE La propriété *longueur non filetée* a un sens pour toute classe de *vis*, mais elle ne s'applique qu'aux vis qui ont une partie non filetée. Elle peut être définie comme étant visible au niveau de la *vis*, tout en devenant applicable seulement dans certaines sous-classes.

Note 3 à l'article: Toutes les propriétés visibles d'une superclasse qui est une classe de caractérisation de produits sont également des propriétés visibles pour ses sous-classes.

Note 4 à l'article: Pour faciliter l'intégration des bibliothèques de composants et des catalogues électroniques selon l'ISO 13584-24:2003 et l'ISO 13584-25, ces parties de l'ISO 13584 demandent que seules les propriétés qui sont applicables à une classe soient utilisées pour caractériser leurs instances dans les bibliothèques de composants et les catalogues électroniques.

Note 5 à l'article: Cette définition d'une "propriété visible" annule et remplace la définition antérieure issue de l'ISO 13584-24:2003 qui était la suivante: "propriété définie pour une certaine famille de produits et qui ne s'applique pas nécessairement aux différents produits de cette famille de produits".

4 Vue d'ensemble du schéma de dictionnaire commun et de la compatibilité avec l'ISO13584_IEC61360_dictionary_schema

4.1 Généralités

Dans les paragraphes suivants, il sera présenté l'architecture du schéma de dictionnaire commun et il sera expliqué comment le même modèle d'informations doit être utilisé dans les Normes internationales pour assurer leur compatibilité.

Le schéma de dictionnaire commun combine les exigences de la série CEI 61360 et de la série ISO 13584. Par conséquent, il contient des ressources pour prendre en charge les exigences spécifiques des deux séries de Normes internationales. Ces ressources sont fournies soit comme capacités facultatives, soit comme sous-types des types définis pour satisfaire aux exigences communes.

4.2 Utilisation du schéma de dictionnaire commun pour échanger des données conformes à la CEI 61360-1

Le dictionnaire commun pour échanger des données conformes à la CEI 61360-1 doit être utilisé comme suit:

- a) les extensions spécifiques à la série ISO 13584 pour prendre en charge la capacité multilingue ne sont pas requises pour l'échange d'éléments de dictionnaire définis

conformément à la CEI 61360-1. Cependant, ces extensions qui sont `present_translations`, `translated_label` et `translated_text` doivent être utilisées dans la structure de l'échange, et ce, pour des raisons de compatibilité;

- b) si une classe de composants a une superclasse, le **coded_name** doit être défini comme étant un **value_code** dans le **domaine** du type d'élément des données de classification de la superclasse;
- c) si un type d'élément de données de classification existe dans une classe spécifique de composants, pour chaque **value** dans son **domain** une sous-classe et un **term** doivent être définis;
- d) un type d'élément de données de classification, facultatif dans la classe de conformité 2 dans le schéma de dictionnaire commun, doit toujours être fourni pour les classes des composants définies conformément à la CEI 61360-1;
- e) seules les unités SI doivent être utilisées, même si le schéma de dictionnaire commun permet l'utilisation de plusieurs types de systèmes d'unités. Cependant, lors de l'utilisation de ce schéma pour l'échange des données conformes à la série CEI 61360, seules les unités SI doivent être utilisées pour les types d'élément de données quantitatif.

4.3 Compatibilité avec l'ISO 13584-42

Une mise en œuvre conforme à la présente partie de la CEI 61360 doit prendre en charge toutes les entités, tous les types et toutes les contraintes associées qui appartiennent à la classe de conformité qu'elle prétend prendre en charge. Par conséquent, la conformité à la classe de conformité 1 de la présente partie de la CEI 61360 exige que toutes les entités, tous les types et toutes les contraintes associées définis dans le schéma de dictionnaire commun soient pris en charge. Les données ISO 13584 conformes au schéma de dictionnaire commun peuvent donc être traitées par une mise en œuvre conforme à la CEI 61360 qui se conforme à la classe de conformité 1 qui inclut toutes les caractéristiques intrinsèques de la classe de conformité 1. Dans l'ISO 13584, une classe de conformité 3 spécifique est destinée à contenir toutes les entités, tous les types et toutes les contraintes associées définis dans le schéma de dictionnaire commun. Une mise en œuvre conforme à l'ISO 13584 conforme à cette classe de conformité doit donc être capable de prendre en charge des données CEI qui appartiennent à la classe de conformité 1 de la présente partie de la CEI 61360.

4.4 Correspondance de dénomination entre la CEI 61360-1 et la CEI 61360-2

En raison des restrictions d'applications spécifiques (par exemple: le langage EXPRESS interdit les espaces dans les noms d'entités), un certain nombre de noms 'EXPRESS similaires sont créés en remplaçant le blanc dans un nom par un trait de soulignement (par exemple, "preferred name" est présenté comme étant "preferred_name").

En d'autres endroits, il est utilisé dans le modèle EXPRESS des noms qui s'écartent de ceux utilisés dans la CEI 61360-1. C'est la conséquence de l'effort visant à atteindre un seul modèle d'informations EXPRESS commun accompagné de Libraires de composants.

Le tableau ci-dessous présente une aide pour la mise en correspondance des noms utilisés dans les deux parties de la CEI 61360.

Tableau 1 – Table de références croisées

dénomination dans la CEI 61360-2	dénomination dans la CEI 61360-1
<code>component_class</code>	Component class (Classe de composants)
<code>condition_DET</code>	Condition data element type (Type d'élément de données de condition)
<code>dependent_P_DET</code>	Data element type (Type d'élément de données)
<code>det_classification</code>	Data element type class (Classe d'un type d'élément de données)

dénomination dans la CEI 61360-2	dénomination dans la CEI 61360-1
(DER)dic_identifier	Identifier (Identificateur)
dic_value	Value (Valeur)
material_class	Material class (Classe de matériaux)
meaning (signification)	Value meaning (Signification de la valeur)
non_dependent_P_DET	Data element type (Type d'élément de données)
preferred_symbol	Preferred letter symbol (Symbole littéral préférentiel)
revision (révision)	Revision number (Numéro de révision)
source_doc_of_definition	Source document of data element type definition (Document source de définition de type d'élément de données)
source_doc_of_definition	Source document of component class definition (Document source de définition de classe de composants)
synonymous_symbols	Synonymous letter symbol (Symbole littéral synonyme)
unit (unité)	Unit of measure (Unité de mesure)
value_code	Value code (Code de la valeur)
version	Version number (Numéro de version)

4.5 Structure principale du schéma de dictionnaire commun

Le présent paragraphe explique les principales constructions des ressources fournies par le schéma de dictionnaire commun:

- **dictionary_element** est n'importe quel élément défini dans le dictionnaire;
- **supplier_element** capture les données des fournisseurs des éléments du dictionnaire (classes, propriétés, types de données); la classe modélise l'élément du dictionnaire classes (familles) qui sont décrites par des propriétés;
- **property_DET** est l'élément de dictionnaire d'une propriété;
- **data_type** spécifie le type d'une propriété.

Ces composants du schéma de dictionnaire sont présentés plus en détail à l'Article 5: **ISO13584_IEC61360_dictionary_schema**.

Dans la présentation du schéma de dictionnaire commun, un certain nombre de diagrammes de vue d'ensemble sont fournis suivant le modèle de planification (voir Figure 1 à Figure 11). Ces modèles de planification utilisent la notation graphique EXPRESS-G pour le langage EXPRESS. Pour la clarté des diagrammes, certaines des relations qui sont définies dans le modèle EXPRESS sont omises. La Figure 1 ci-dessous présente sous forme de modèle de planification la structure principale du schéma du dictionnaire commun. La plupart de ces figures contiennent des modèles de vue d'ensemble (ou modèles de planification), mais montrent uniquement le niveau de détail qui est approprié à un certain endroit.

Pour la clarté des diagrammes, certaines des relations qui sont définies dans le modèle EXPRESS sont omises. La Figure F.1 ci-dessus présente, sous la forme d'un modèle de planification, les grandes lignes d'une structure principale du modèle de dictionnaire commun ISO13584/CEI61360.

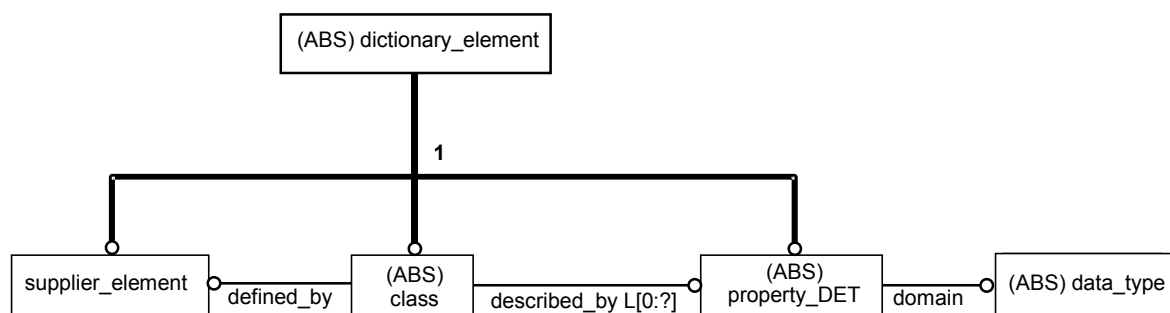


Figure 1 – Vue d'ensemble du schéma de dictionnaire

5 ISO13584_IEC61360_dictionary_schema

5.1 Généralités

Le présent article, qui constitue la partie principale du Modèle d'information commun de l'ISO 13584-42 et de la série CEI 61360, contient l'énumération EXPRESS complète du schéma de dictionnaire, annotée avec des commentaires et du texte explicatif. L'ordre du texte dans le présent article est déterminé en premier lieu par l'ordre imposé par le langage EXPRESS et en second lieu, par l'importance.

5.2 Schéma de dictionnaire

En premier lieu, le schéma a besoin d'être déclaré.

EXPRESS specification:

```

*)
SCHEMA ISO13584_IEC61360_dictionary_schema;
( *

```

5.3 Références à d'autres schémas conceptuels

Le présent paragraphe contient des références à d'autres schémas conceptuels EXPRESS qui sont utilisés dans le schéma du dictionnaire. Leur source est indiquée dans le commentaire respectif.

EXPRESS specification:

```

*)
REFERENCE FROM support_resource_schema(identifier, label, text);

REFERENCE FROM person_organization_schema(organization, address);

REFERENCE FROM measure_schema;

REFERENCE FROM ISO13584_IEC61360_language_resource_schema;

REFERENCE FROM ISO13584_IEC61360_class_constraint_schema;

REFERENCE FROM ISO13584_IEC61360_item_class_case_of_schema;

REFERENCE FROM ISO13584_external_file_schema
    (external_item,
     external_file_protocol,
     external_content,

```